
Abstract

Production data in today's industrial environments is often fragmented and demanding to use in operational contexts. The need for comprehensive solutions that can collect sensor data and production data in one system is increasing, especially in consideration of stricter requirements for environmental reporting and monitoring of production processes.

This report describes the development of *Factory pulse*, a full-stack web service developed on behalf of Innoveria AS. The name *Factory pulse* reflects the system's purpose of "taking the pulse of the factory" by collecting, storing, processing and visualizing sensor data from industrial LoRaWAN sensors, and combine them in context of production data obtained from the ERP system Monitor ERP.

The solution is built on a microservices architecture with Go as backend technology, React as frontend framework and PostgreSQL as database system. ChirpStack is used as a LoRaWAN network server for handling sensors and gateways. The system supports role-based access control and historical data visualization, management of sensors and gateways, as well as context-based merging of sensor and production data. The solution is designed with separation of responsibilities as a key design principle, so that one service still functions even if another is unavailable.

Sammendrag

Produksjonsdata i dagens industrimiljøer er ofte fragmentert og krevende å benytte i operasjonelle sammenhenger. Behovet for helhetlige løsninger som kan samle sensordata og produksjonsdata i ett system er økende, særlig i lys av strengere krav til miljørapportering og effektivisering av produksjonsprosesser.

Denne rapporten beskriver utviklingen av *Fabrikkpuls*, en fullstack-webtjeneste utviklet på oppdrag fra Innoveria AS. Navnet *Fabrikkpuls* reflekterer systemets formål om å "ta pulsen på fabrikken" ved å samle inn, lagre, behandle og visualisere sensordata fra industrielle LoRaWAN-sensorer, og sette disse i kontekst av produksjonsdata hentet fra ERP-systemet Monitor ERP.

Løsningen er bygget på en mikrotjenestearkitektur med Go som backend-teknologi, React som frontend-rammeverk og PostgreSQL som databasesystem. ChirpStack benyttes som LoRaWAN-nettverksserver for håndtering av sensorer og gateways. Systemet støtter rollebasert tilgangskontroll og historisk datavisualisering, administrasjon av sensorer og gateways, samt kontekstbasert sammenslåing av sensor- og produksjonsdata. Løsningen er utformet med separasjon av ansvar som et sentralt designprinsipp, slik at én tjeneste fortsatt fungerer selv om en annen er utilgjengelig.

Forord

Denne bacheloroppgaven er skrevet ved Institutt for Informatikk og Datateknikk ved NTNU i Gjøvik av Jonas Nygaard, Magnus Dybdal, Vinjar Berland og Marius Eilertsen.

Vi ønsker å takke vår veileder Tom Røise for meget god veiledning og samarbeid. Vi ønsker også å takke vår oppdragsgiver Innoveria og Magne Koren for god tilgjengelighet og entusiasme gjennom hele prosjektet.

Til slutt vil vi takke hverandre for et godt og konstruktivt samarbeid gjennom mange uker med gruppearbeid.

Table of Contents

Figurer	xii
Tabeller	xiv
1 Introduksjon	1
1.1 Bakgrunn	1
1.2 Fagområde	2
1.3 Oppgavebeskrivelse	3
1.4 Avgrensing	3
1.5 Målgrupper	4
1.6 Motivasjon	4
1.7 Gruppemedlemmer	5
1.8 Roller	5
1.9 Rapportens struktur	6
2 Teknologisk bakgrunn	7
2.1 LoRaWAN	7
2.2 Monitor ERP	8
2.3 Kontekst	8
3 Utviklingsprosess	10
3.1 Utviklingsmetodikk	10
3.2 Valg av metodikk	10
3.3 Arbeidsverktøy (utviklingsprosess)	11
3.4 Fordeling av arbeidsoppgaver	12
3.5 Gjennomføring	12

3.6	Bruk av KI	16
4	Kravspesifikasjon	18
4.1	Use Cases	18
4.2	Domenemodell	21
4.3	Operasjonelle krav	22
4.4	Sikkerhetskrav	23
4.5	Grensesnittkrav	24
5	Teknologianvendelse	26
5.1	Valg av teknologisk retning	26
5.2	Go som backend-teknologi	27
5.3	React for frontend-utvikling	28
5.4	Database	30
5.5	Nettverksprotokoller	31
5.6	ChirpStack	33
6	Systemarkitektur	34
6.1	Systemoversikt	34
6.2	Mikrotjenestearkitektur	35
6.3	Distribusjon og drift	39
7	Implementasjon	40
7.1	Frontend	40
7.1.1	Utfordringer	40
7.1.2	Wireframe	40
7.1.3	Material UI bibliotek	41
7.1.4	Feature-Sliced Design	42

7.1.5	Barrel exports	44
7.1.6	Hooks	46
7.1.7	Layout	48
7.2	API-struktur	49
7.3	API-gateway	51
7.4	JWT-basert autentisering	54
7.4.1	Innlogging	56
7.4.2	Tokenrotasjon	57
7.5	Enhetsadministrasjon og LoRaWAN-integrasjon	57
7.5.1	ChirpStack REST-klient	58
7.5.2	Administrasjon av sensorer og gateways	58
7.5.3	Sensorprofiler	60
7.5.4	Måletypevokabular og merking av rådata	61
7.5.5	Datakonsistens	63
7.6	Innhenting av sensordata	64
7.7	Monitor ERP integrasjon	67
7.8	Kontekstbasert sensordata	70
7.8.1	Tjenestens rolle i systemarkitekturen	70
7.8.2	Integrasjon mot eksterne tjenester	71
7.8.3	Aggregasjonsregler	72
7.8.4	Kalkulator og aggregasjonslogikk	73
7.8.5	Beregning av kontekstdata	75
7.8.6	Ordrekontekst og datafusjon	76
8	Kvalitetssikring	80
8.1	Gjennomgang av kode	80

8.2	Kodekvalitet	80
8.3	Testing	85
9	Diskusjon	93
9.1	Emneintegrasjon	93
9.2	Bærekraftsperspektiv	94
9.3	Refleksjon	96
9.3.1	Resultat	96
9.3.2	Læringsutbytte	96
9.3.3	Timebruk	99
9.3.4	Kunstig intelligens som verktøy	101
9.4	Forslag til videre utvikling	104
10	Kilder	105
11	Vedlegg	108
A	Prosjektplan	109
B	Gruppekontrakt	128
C	Wireframes	131
D	Møtereferater	133
E	Timeføring	143
F	Use Case Beskrivelser	146
G	Commit markdown	149
H	Sekvensdiagram context-service	150

I	API Endepunkter	151
J	sekvensdiagram	154

Ordliste

Agent En agent består av en underliggende språkmodell, egen isolert kontekst, en systemprompt, verktøy den kan benytte, samt eventuelle integrasjoner mot eksterne tjenester. Agenter egner seg særlig for langvarige og flertrinns oppgaver med begrenset menneskelig inngripelse.. [80](#)

Alfa test En type brukertesting hvor en bruker jobber med med utviklere for å vurdere om brukere vil ha funksjonalitetene som er planlagt [1]. [91](#)

AppKey En nøkkel som brukes for autentisering og sikker kommunikasjon i nettverket, som blir tilsendt av sensorprodusent med sensor. [8](#), [21](#)

Arbeidssenter Representerer en maskin eller ressurs hvor operasjonen utføres. Eksempel: Kutte-maskin i produksjonshallen. [8](#), [9](#), [69](#)

Beta test En type brukertesting hvor produktet distribueres til brukere for å få kommentarer, ofte fokusert på brukervennlighet og om det funker på brukerens miljø [1]. [92](#)

Boilerplate Kode som er mer eller mindre identisk på tvers av moduler eller tjenester, og som må inkluderes for at koden skal fungere, uten at den i seg selv inneholder særlig logikk. [17](#), [45](#), [97](#), [103](#)

ChirpStack En open-source LoRaWAN nettverkserver som håndterer kommunikasjon mellom [LoRa](#)-gateways og applikasjoner. [26](#), [33](#), [35](#), [36](#), [39](#), [57](#), [58](#), [60](#), [61](#), [63](#), [64](#), [98](#)

Device EUI En unik identifikator som står på sensor og gateway. [8](#), [21](#), [59](#), [62](#), [75](#), [76](#)

gateway En [LoRaWAN](#)-gateway er en enhet som mottar trådløse [LoRa](#)-signaler fra sensorer og videresender dem til en sentral server over IP-basert nettverk. [ix](#), [1](#), [3](#), [7](#), [18](#), [21](#), [24](#), [26](#), [30](#), [33](#), [34](#), [36](#), [45](#), [47](#), [51](#), [54](#), [57](#), [58](#), [63](#), [91](#), [94](#), [96](#), [104](#)

Hi-Fi wireframe En high-fidelity (hi-fi) wireframe inneholder detaljert informasjon, ofte inkludert interaktive elementer, som demonstrerer hvordan brukeren vil samhandle med grensesnittet [2]. [40](#)

LoRaWAN Nettverks Server Programvare som håndterer tilkoblinger, behandling og oversikt over [gateway](#) og sensorer. [3](#), [8](#), [27](#), [33](#), [35](#)

-
- monorepo** En programvareutviklingsstrategi der kode for flere prosjekter lagres i ett versjonskontrollert repository. [82](#), [96](#), [97](#)
- Operasjon** Beskriver et enkelt steg for et arbeidssenter i produksjonsprosessen som er knyttet til en produksjonsordre. Eksempel: Kutting av trepanel som en del av prosessen. [2](#), [8](#), [9](#), [22](#), [24](#), [27](#), [35](#), [37](#), [50](#), [52](#), [58](#), [59](#), [63](#), [68](#), [70](#), [71](#), [76](#)
- Operasjonsrapport** Inneholder registrering av utført arbeid knyttet til operasjon, som inkluderer tidsbruk og aktivitet. Eksempel: En operatør registrerer at kutting tok 1 time, med 30 min nedetid grunnet verktøyskifte. [8](#), [37](#), [68](#)
- PostgreSQL** Åpen kildekode relasjonsdatabasesystem som bruker SQL. [27](#), [30](#), [57](#), [58](#), [63](#), [72](#), [83](#), [93](#)
- Produksjonsordre** Representerer en spesifikk produksjon som skal gjennomføres. Eksempel: produksjon av 100 trepanel til kunden, med startdato og frist. [8](#), [9](#), [35](#), [37](#), [44](#), [47](#), [70](#), [76](#)
- SCRUM** Et smidig rammeverk for utviklingsmetodikk. [10](#), [13](#), [16](#), [97](#)
- Sensorprofil** Definisjonen på hvordan sensoren oppfører seg, inkludert hvilke data som sendes og hvordan det skal dekodes. [8](#), [21](#), [58–60](#), [62](#), [91](#)
- Skill** En skill er definert i en konfigurasjonsfil med instruksjoner. Den kan aktiveres av brukeren eller automatisk av KI-en når den vurderes som relevant for oppgaven.. [80](#)
- sprint** En kort, tidsbestemt periode hvor et scrum team jobber med å fullføre en bestemt mengde jobb.. [5](#), [10](#), [12](#), [14](#), [15](#), [91](#), [98](#)
- Stubb** En ekte avhengighet, som returnerer forhåndsdefinerte svar uten å utføre faktisk logikk. Formålet er å isolere det som testes fra eksterne avhengigheter.. [86](#)
- tenant** En isolert organisasjonsenhet i ChirpStack som representerer en kunde eller ett selskap. Hver tenant har egne gateways og sensorer som er logisk adskilt fra andre tenants. [13](#), [57](#), [58](#), [60](#), [66](#)
- type safety** Type safety er hvilken grad type error blir motvirket eller forhindret, som ofte skjer når man skal utføre operasjoner på verdier med feil datatype. [28](#), [45](#)
-

Forkortelser

API Application Programmable Interface. [1](#), [3](#), [12](#), [24–27](#), [33](#), [34](#), [36](#), [44](#), [46](#), [50](#), [51](#), [93](#), [99](#)

CD Continuous Development. [15](#), [104](#)

CI Continuous Integration. [80](#), [81](#), [84](#), [87](#), [104](#)

CSS Chirp Spread Spectrum. [7](#)

DTO Data Transfer Object. [50](#)

ERP Enterprice Resource Planning. [1–3](#), [7](#), [8](#), [13](#), [18](#), [20](#), [24](#), [27](#), [34](#), [35](#), [38](#), [41](#), [67](#), [68](#), [70](#), [76](#), [77](#), [86](#), [95](#), [96](#), [99](#), [104](#)

FSD Feature-Sliced Design. [40](#), [42](#), [102](#)

IIoT Industrial Internet of Things. [2](#), [10](#), [94](#)

KI Kunstig Intelligens. [16](#), [80](#), [85](#), [95](#), [101](#)

LoRa Long Range. [ix](#), [7](#)

LoRaWAN Long Range Wide Area Network. [ix](#), [3](#), [7](#), [10](#), [22](#), [26](#), [27](#), [33](#), [34](#), [39](#), [58](#), [60](#), [62](#), [65](#), [93](#), [96](#)

MAC Media Access Control. [7](#), [94](#)

MQTT Message Queuing Telemetry Transport. [8](#), [27](#), [32](#), [35](#), [36](#), [65](#), [89](#)

MR Merge Request. [12](#), [14](#), [16](#), [17](#), [27](#), [80](#), [100](#)

OTAA Over-The-Air Activation. [59](#), [63](#)

PBL Product Backlog. [14](#), [16](#)

PO Product Owner. [5](#), [10](#)

SBL Sprint Backlog. [11](#), [14](#), [16](#)

TDD Test-Driven Development. [85](#)

UDP User Datagram Protocol. [8](#), [35](#)

VCS Version Control System. [11](#)

Figurer

1	Fra sensor- og ERP-data til kontekst	2
2	Grafisk fremstilling av prosjektets sprinter (<i>Grafikk er generert med Claude Design</i>)	13
3	Issue board brukt i utviklingsprosessen (03.04.2026)	16
4	Use Case Diagram	19
5	Domenemodell	22
6	Wireframe — CO2-dashboard	25
7	Utkast av database diagram	30
8	Systemoversikt	34
9	Mikrotjenester	36
10	Kildekode struktur	39
11	Autocomplete brukt i to forskjellige UI-elementer	42
12	FSD hovedkonsept. Bilde hentet fra FSD-docs [22]	43
13	Innoveria sitt tidligere prosjekt	48
14	Nåværende layout	49
15	Backend-implementasjon	49
16	API Gateway middleware pipeline	52
17	sekvensdiagram for innlogging	56
18	Payload Schema Mapping	62
19	Sensorinnhenting	65
20	Dataavstemming av ERP-data	69
21	Overordnet dataflyt for Context-Service	70
22	Context regel i frontend	72
23	Strømsensor data	73

24	Trapesmetoden illustrert. Figur hentet fra hscprep.com.au [30]	75
25	Ordrekontekst i dashboard	77
26	Lokal- og CI-pipeline	84
27	HTTP svartid ved 300 samtidige brukere (produksjonsserver)	90
28	Tavlebruk i fysisk møte	98
29	Prosentandelvis fordeling av tid mellom temaer	100
30	Tomt dashboard	131
31	Med komponenter	131
32	CO2 dashboard v2	132
33	Sensors v2	132
34	Sprint 1 planning	133
35	Daily 5/2	134
36	Daily 12/2	135
37	Sprint 1 retrospective	136
38	Sprint 2 review	137
39	Daily 27/4	138
40	Sprint 4 retrospective	139
41	Sprint 4 planning	140
42	Daily 30/4	141
43	Sprint 5 planning	142
44	Detaljert sekvensdiagram	150
45	Context data flow	155

Tabeller

1	Scrum-roller i prosjektet	10
2	Sprintfordeling med datoer og hovedfokus	13
3	Faste møteformater i prosjektet	14
4	Standard commit-typer brukt i prosjektet	15
5	Kategorisering av labels brukt i prosjektets issue board	16
6	Utviklingsverktøy	27
7	FSD-lagene i prosjektet	43
8	Konfigurerte regler i <code>web/eslint.config.js</code>	82
9	Aktiverte lintere i <code>.golangci.yml</code>	83
10	Lint- og formateringsteknologi brukt i prosjektet	84
11	Resultater for sensormeldinger	89
12	Virtuelle brukere i HTTP-belastningstest	90
13	HTTP svartid ved 300 samtidige brukere (produksjonsserver)	91
14	API-endepunkter eksponert av systemet	151

1 Introduksjon

1.1 Bakgrunn

Innoveria AS [3] er et teknologiselskap som arbeider med å forenkle innsamling, bruk og visualisering av data i industrien. Selskapet har som mål å gjøre produksjon mer effektiv ved å motivere fabrikker til å ta i bruk teknologiske løsninger for overvåking av produksjonsprosesser. Innoveria ønsker å redusere kompleksiteten og tidsbruken knyttet til datainnsamling, slik at produksjonsbedrifter kan ta bedre beslutninger basert på historiske data.

Produksjonsdata i dagens industrimiljøer er ofte fragmentert og krevende å benytte i operasjonelle sammenhenger. Det finnes i begrenset grad felles standarder for hvordan slike data samles inn, struktureres og visualiseres [4]. Samtidig stilles det i økende grad krav til at fabrikker kan dokumentere og rapportere miljøavtrykket fra egne produksjonsprosesser. Et eksempel på dette er ISO 14067 [5], som legger til rette for rapportering av klimaavtrykk helt ned på per-enhet-nivå. Disse utfordringene skaper et behov for en helhetlig løsning som kan samle data fra maskiner, sensorer og eksisterende systemer, som for eksempel [Enterprise Resource Planning](#)-systemer, og sette disse i kontekst av produksjonsmiljøet gjennom visualisering av historiske data.

En sentral utfordring for fabrikker som ønsker å ta i bruk sensorbasert overvåking, er kompleksiteten knyttet til oppsett og konfigurering av nytt utstyr. Prosessen med å koble til nye [gateways](#) og sensorer krever i dag betydelig teknisk kompetanse og tar mye tid, noe som gjør terskelen for å komme i gang høy. Samtidig må Innoveria bruke betydelige ressurser på å få sensorovervåkingen i drift hos nye og eksisterende kunder.

Innoveria ønsker derfor å utvikle en helhetlig løsning bestående av en database for lagring av tidsseriedata med tilhørende metadata, et [Application Programmable Interface](#)-lag for datainnhenting og spørringer, samt et moderne dashboard med rollebasert tilgang for både fabrikkene og Innoveria. Løsningen skal støtte håndtering av historiske data og legge til rette for at data kan settes i kontekst av produksjonsmiljøet.

1.2 Fagområde

Industrien gjennomgår en gradvis digitalisering, der maskiner og produksjonsutstyr i økende grad kobles til sensorer for kontinuerlig overvåking av tilstand, forbruk og ytelse. Dette feltet omtales som [Industrial Internet of Things \(IIoT\)](#). Det er imidlertid stor variasjon i digitaliseringsgrad mellom fabrikker. Noen opererer med høy grad av sensorikk og automatisering, men mange produksjonsbedrifter er fortsatt preget av manuelle prosesser, der operatører logger tid, aktivitet og produksjonsdata manuelt [4]. ERP-systemet fungerer i slike miljøer primært som et register for disse manuelle registreringene, og ikke som en kilde til sanntidsdata. Det er denne typen fabrikker Fabrikkpuls er rettet mot.

En sentral utfordring innen IIoT er at sensordata alene gir begrenset verdi — en måling er først meningsfull når den settes i sammenheng med produksjonskonteksten den ble gjort i. Mer om dette beskrives i kapittel [2.3](#). Denne koblingen realiseres typisk gjennom integrasjon mot ERP-systemer som inneholder strukturert informasjon om ordrer, operasjoner og ressurser. En slik integrasjon er imidlertid ikke standardisert, og løsninger som håndterer dette på tvers av systemer finnes i begrenset grad [4]. I tillegg stilles det økende krav til at produksjonsbedrifter kan dokumentere energiforbruk og miljøavtrykk per produsert enhet. Standarder som ISO14067 [5] legger til rette for slik dokumentasjon. Fabrikkpuls er utviklet i dette skjæringsfeltet mellom sensorinfrastruktur, ERP-integrasjon og produksjonskontekst. De sentrale teknologiene som inngår i løsningen beskrives nærmere i kapittel [2](#).

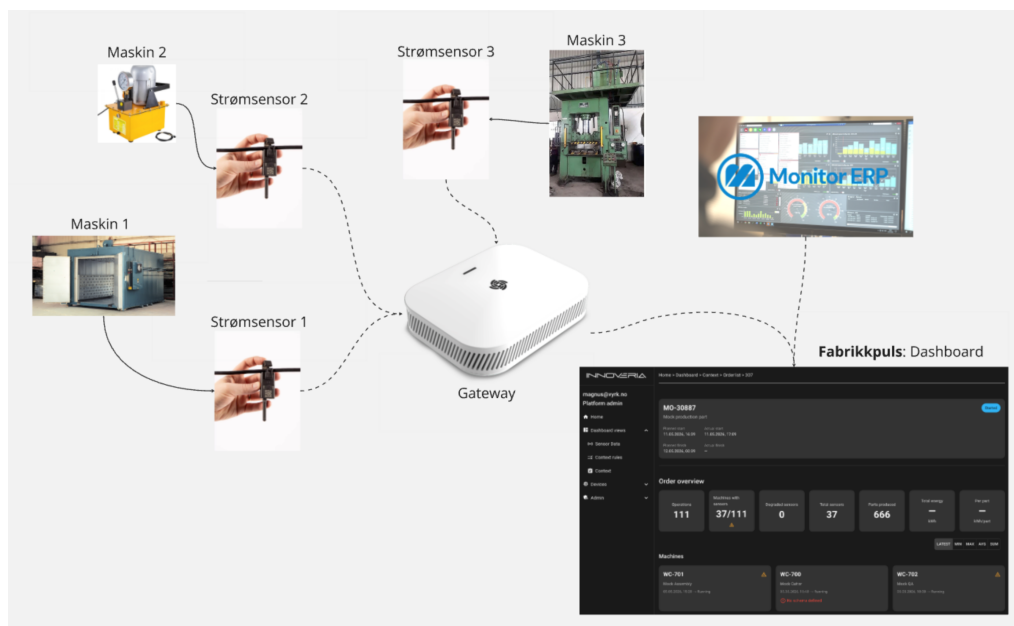


Figure 1: Fra sensor- og ERP-data til kontekst

1.3 Oppgavebeskrivelse

Oppgaven består av å utvikle en fullstack-løsning for mottak, lagring, behandling og visualisering av sensordata i kontekst av produksjonsdata fra industrielle sensorer tilknyttet et [Long Range Wide Area Network \(LoRaWAN\)](#)-nettverk, samt produksjonsdata fra ERP-systemer. Fullstack-løsningen skal bestå av et webgrensesnitt for visualisering av data, et backend-API for behandling av data, og en database for lagring. Den ferdige løsningen skal ha følgende funksjonalitet:

- Enkel onboarding av gateways og sensorer via webgrensesnitt med bekreftelse på at enheten er koblet til og mottar data.
- Motta og dekode sensordata.
- Lagre sensordata i databasen for videre behandling.
- Behandle data for analyse og visualisering.
- Innhente produksjonsdata fra ERP-systemer.
- Rollebasert autorisasjon for å sikre riktig tilgang til data.
- Visualisere sensordata satt i kontekst av produksjonsdata gjennom webgrensesnittet, inkludert:
 - Fremstilling av historiske sensordata.
 - Fremstilling av sensordata kombinert med produksjonsdata.

1.4 Avgrensing

Prosjektet ble avgrenset til å ikke utvikle en egen [LoRaWAN Nettverks Server](#), men i stedet benytte eksisterende tjenester for å forenkle oppsett av sensorer og gateways. Fokus skulle ikke ligge på å lage avanserte grafiske fremstillinger av data, men på systemets evne til å lagre, behandle og visualisere sensordata, samt sette disse dataene i kontekst med produksjonsdata.

Siden det finnes mange kilder til produksjonsdata, valgte gruppen å fokusere på leverandøren av ERP-systemer som kundene til Innoveria bruker, Monitor ERP¹, for innhenting av slike data. Videre ble prosjektet begrenset til å støtte et begrenset

¹<https://www.monitorerp.com/>

utvalg av sensorprodusenter og gateways, primært Milesight ² og RAK ³, for å sikre at prosjektets omfang var håndterbart.

1.5 Målgrupper

Interessen for dette prosjektet kan klassifiseres i to målgrupper basert på hvilket resultat de er interessert i: sluttproduktet eller rapporten.

Sluttproduktet

Sluttproduktets målgruppe er de som regelmessig vil benytte produktet. Hele produktet vil brukes administrativt av Innoveria, som videre gir tilgang til sine kunder.

Siden produktet visuelt er likt for alle målgrupper, og den tekniske kompetansen varierer basert på arbeidsprosess, alder og bakgrunn, kreves et brukervennlig grensesnitt som er enkelt å forstå uten at funksjonalitet går tapt. Produktet er utviklet med engelsk språk gjennom hele applikasjonen for å sikre at alle brukere forstår det.

Rapporten

Rapporten er primært rettet mot de som skal evaluere prosjektet, altså sensorer og gruppens veileder. Målgruppen inkluderer også personer som skal videreutvikle produktet eller bygge videre på det, samt prosjektets oppdragsgiver og eventuelle arbeidsgivere som ønsker innsikt i utviklingsprosessen.

1.6 Motivasjon

Oppgaven ble valgt fordi den gir mulighet til å arbeide med teknologi gruppen ikke hadde erfaring med fra tidligere. Sammenhengen mellom programvareutvikling og praktisk anvendelse er tydelig, og prosjektet har potensial til å skape reell verdi i faktiske produksjonsmiljøer. Samarbeidet med Innoveria AS og deres tilknytning til lokale bedrifter og fabrikker gjør prosjektet ekstra relevant og motiverende.

²<https://www.milesight.com/>

³<https://www.rakwireless.com/>

1.7 Gruppemedlemmer

Teamet består utelukkende av nåværende studenter på bachelorstudiet i programmering. Gruppen har derfor bred kompetanse og kunnskap, med unntak av enkelte valgfag. Gjennom studieløpet har gruppen opparbeidet erfaring innen flere områder av programvareutvikling. Dette inkluderer sikkerhet, webutvikling med HTML, CSS og JavaScript, mobilutvikling med Kotlin, cloud-teknologi, grafikk, programvareutvikling, databaser, algoritmer og integrasjonsprosjekt. I tillegg hadde to av gruppemedlemmene erfaring fra emnet robuste og skalerbare systemer.

For dette prosjektet var særlig kunnskap innen webutvikling, databaser og backendutvikling, opparbeidet gjennom flere emner, den mest relevante kompetansen gruppen tok med seg inn i prosjektet. Når det gjelder arbeidsprosessen, var erfaring fra programmeringsvareutvikling, integrasjonsprosjekter og ulike gruppeprosjekter viktig for å sikre god fremdrift og effektivt samarbeid.

1.8 Roller

På grunn av et variert arbeidsløp med tekniske aspekter som i stor grad var avhengige av hverandre, var det ikke hensiktsmessig å tildele permanente oppgaver gjennom hele prosjektet. Gruppen etablerte likevel en oppdeling med varierende roller innen backend og frontend.

Frontendoppgavene var i stor grad avhengige av hvilke funksjonaliteter som til enhver tid var utviklet i backend og måtte implementeres i den pågående [sprinten](#).

Referenten hadde ansvar for å ta notater under møter internt i gruppen, med [Product Owner \(PO\)](#) eller med veileder. I tillegg skrev vedkommende møtereferater for hver type møte. Møtereferatene finnes i vedlegg [D](#).

Rombookeren hadde ansvar for å reservere møterom på campus til møter.

Veileder og gruppen hadde møter torsdager klokken 14:30 de fleste uker. Veileder bidro med å sikre at prosjektet utviklet seg i en realistisk retning og kvalitetsvurderte tekster.

Produkteieren representerte Innoverias [3] ønsker og krav til produktet. Gruppen og produkteieren hadde møter annenhver uke for å oppdatere fremdrift og prioritere oppgaver for neste sprint.

1.9 Rapportens struktur

I starten av rapporten finnes en ordliste og en liste med forkortelser. Disse er inkludert for å gjøre det enklere for leseren å forstå begreper og forkortelser som ikke forklares i detalj i selve rapporten. Ved lesing i PDF-format vil det også være mulig å klikke på ord markert i blått for å navigere direkte til ordlisten. På slutten av rapporten finnes tabeller, figurer og andre vedlegg som er relevante for prosjektets helhet. Rapporten er delt inn i ni kapitler, hvor hvert kapittel inneholder flere spesifikke seksjoner. Kapitlene og en kort forklaring av innholdet er som følger:

1. **Introduksjon:** En introduksjon til oppdragsgiveren, oppgaven som skal utføres, utviklingsgruppen og annen informasjon som er relevant for å forstå rapporten.
2. **Teknologisk bakgrunn:** Faglige felt som oppgaven har stort fokus på.
3. **Utviklingsprosess:** Planen og prosessen utviklingen er basert på.
4. **Kravspesifikasjon:** Beskriver systemets funksjonelle og ikke-funksjonelle krav, og definerer hva sluttproduktet skal oppfylle.
5. **Teknologianvendelse:** Valg av teknologi innenfor de ulike delene av utviklingen.
6. **Systemarkitektur:** En beskrivelse av hvordan systemet er bygget opp og hvordan data flyter mellom komponentene.
7. **Implementasjon:** Hvordan spesifikke komponenter ble implementert, samt utfordringer underveis.
8. **Kvalitetssikring:** Oppgaver utført for å forbedre produktet under utvikling.
9. **Diskusjon:** Refleksjoner rundt tekniske aspekter, hvordan de kunne vært videreutviklet, og hvordan de fremstår i sluttproduktet.

2 Teknologisk bakgrunn

Dette kapittelet beskriver sentrale objekter som senere blir nevnt i rapporten. Formålet er å gi nødvendig bakgrunnsinformasjon for å forstå hvordan sensordata behandles og settes i kontekst med produksjonsdata. Kapittelet introduserer derfor *LoRaWAN*-teknologi, Monitor *ERP* og begrepet kontekst.

2.1 LoRaWAN

LoRaWAN består av flere komponenter som samarbeider for å samle inn, overføre og behandle sensordata. Nettverket er designet for lavenergi- og langdistansekommunikasjon. Av den grunn egner det seg godt for produksjonsbedrifter, ettersom teknologien gjør det mulig å overvåke maskiner og produksjonsmiljøer over store arealer med minimalt energiforbruk. Det passer samtidig godt i miljøer med tykke vegger, da signalet har sterk penetreringsevne [6].

Long Range og LoRaWAN utgjør en sentral teknologi i denne løsningen. I følge The-Things-Network [6] beskrives LoRa som:

*Trådløs modulasjonsteknikk som er avledet fra *Chirp Spread Spectrum (CSS)*-teknologi. Den koder informasjon på radiobølger ved hjelp av chirp-pulser på samme måte som delfiner og flaggermus kommuniserer. LoRa-modulert overføring er robust mot forstyrrelser og kan mottas over svært lange avstander (egen oversettelse).*

Videre beskriver The-Things-Network [6] LoRaWAN som:

**Media Access Control (MAC)*-lagsprotokoll bygget oppå LoRa-modulasjon. Protokollen fungerer som et programvarelag som definerer hvordan enheter benytter LoRa-maskinvare, blant annet når enheter kan sende data, samt hvordan meldinger struktureres og formateres (egen oversettelse).*

Sensor

Sensorer er enheter som registrerer måledata som temperatur, fuktighet, avstand og lignende. Disse enhetene er som oftes batteridrevne og designet for lavt strømforbruk. Enkelte sensorer kan ha en levetid på opptil 10 år [6]. Dataene som samles inn kodes med Base64-encoding og sendes videre til gateway. Ulike sensorer fungerer forskjellige når det gjelder hvordan data sendes og akkumuleres. Sensorer sender data til gatewayen med lengre tidsintervaller - intervaller på 1-10 minutter er ikke uvanlig - men en sensor kan samtidig samle inn målinger hyppigere enn selve dataoverføringen skjer.

Gateway

En gateway fungerer som et mellomledd mellom sensor og internett. Den mottar signaler fra flere sensorer og videresender disse til en nettverksserver via internett, ofte ved bruk av protokoller som [Message Queuing Telemetry Transport \(MQTT\)](#) eller [User Datagram Protocol \(UDP\)](#). Gatewayen utfører ikke avansert databehandling, noe som innebærer at den videresender data umiddelbart når sensordata mottas. For å håndtere mottak og dekoding av sensordata kreves derfor en nettverksserver.

LoRaWAN Nettverks Server

LoRaWAN-nettverket består av to fysiske enheter: sensor og gateway. Begge er koblet til LoRaWAN-nettverket. Gatewayen har ansvar for å sende data over internett, mens sensoren sender data som må dekodes for å kunne tolkes på en meningsfull måte. For å håndtere denne kommunikasjonen kreves en LoRaWAN-nettverksserver. Dette er programvare som må driftes enten på gateway eller på en server. Nettverksserveren har ansvar for å motta rådata før den dekker sensordata og videreformidler dem til lesbar visning.

For å registrere og ta i bruk en sensor i nettverket, gitt at gatewayen er konfigurert til å peke mot nettverksserveren, kreves følgende felter: [device EU](#), [appkey](#) og [sensorprofil](#).

2.2 Monitor ERP

Produksjonsbedrifter benytter ERP-systemer for å samle data om produksjonsprosesser, som produksjonsorder, maskinbruk og tidsintervaller. Monitor ERP⁴ er et slikt ERP-system som flere produksjonsbedrifter benytter. Det er et større informasjonssystem som samler og strukturerer data fra ulike deler av virksomheten, som produksjon, logistikk, økonomi og personell. I denne løsningen hentes kun nødvendige felter relatert til produksjon. Disse er: [produksjonsordre](#), operasjoner, [operasjonsrapport](#) og [arbeidssenter](#).

2.3 Kontekst

Rå sensormålinger er i seg selv ikke meningsfull informasjon. En måling forteller lite uten kjennskap til produksjonskonteksten den ble gjort i. Det er først når slike målinger settes i sammenheng med produksjonsprosessen at dataene får verdi. Dette er kjernen i det som i denne oppgaven betegnes som *kontekst*.

⁴<https://www.monitorerp.com/no/>

I Fabrikkpuls er kontekst definert som sammenstillingen av rå sensordata og tilhørende produksjonsinformasjon fra ERP-systemet. For et gitt tidsvindu kobles sensormålinger til den produksjonsordren og operasjonen som var aktiv på det aktuelle arbeidssenteret. Resultatet er det som i applikasjonen omtales som *ordrekontekst*: en strukturert representasjon av en produksjonsordre, dens operasjoner og sensoravlesningene knyttet til hver operasjon i perioden. Et konkret eksempel er strømsensorer montert på en produksjonsmaskin, hvor akkumulerte amperemålinger kan omregnes til watt og wattimer, slik at energiforbruket kan knyttes direkte til en spesifikk produksjonsordre eller til hver enkelt enhet som produseres innenfor den aktuelle ordren. Dette gjør det mulig å besvare spørsmål som hvor mye energi det krever å produsere én enhet av et gitt produkt.

Det er verdt å understreke at det i dag ikke finnes en komplett løsning som håndterer denne typen kontekstualisering på tvers av systemer. Integrasjon mot ERP-systemer er i stor grad leverandørspesifikk, og Fabrikkpuls er hovedsaklig utviklet mot MonitorERP. Arkitekturen i Fabrikkpuls er imidlertid utformet med tanke på at andre ERP-systemer også skal kunne støttes. For å integrere et nytt ERP-system kreves et tilpasset adapter som oversetter det aktuelle systemets datastruktur til Fabrikkpuls sitt interne format. Kjerne-logikken for kontekstualisering, aggregering og presentasjon forblir uendret.

3 Utviklingsprosess

3.1 Utviklingsmetodikk

En utviklingsmetodikk er overordnet sett et rammeverk for hvilke prosesser man anvender i utviklingen av et produkt. Det finnes mange forskjellige metodikker, og per dags dato er agile metoder bransjestandard[7].

3.2 Valg av metodikk

Ved valg av utviklingsmetodikk stod gruppen overfor to sentrale utfordringer: behovet for fleksibilitet til å endre retning underveis, og manglende erfaring, spesielt med IIoT- og LoRaWAN-domenet og tilhørende teknologier som oppgaven omhandlet. Dermed falt utviklingsmetodikker som fossefallsmodellen bort, da de ikke støtter den fleksibiliteten prosjektet krevde.

Gruppen vurderte *Kanban* som et alternativ, men forkastet det til fordel for *SCRUM*. Kanban egner seg godt for kontinuerlig flyt uten faste iterasjoner, men gir mindre struktur rundt planlegging og prioritering — noe gruppen anså som ugunstig gitt prosjektets omfang og behovet for regelmessig avstemming med oppdragsgiver. Scrum ga derimot muligheten til å starte med den kunnskapen gruppen hadde, justere kursen mellom sprinter og levere funksjonalitet i kortere iterasjoner.

Rammeverket ble tilpasset prosjektets størrelse og ressurser. Elementer som støttet smidig arbeid og hyppige iterasjoner ble beholdt, mens elementer som ikke ble ansett som hensiktsmessige for et lite studentteam ble nedprioritert. Sprint-relaterte møter ble holdt i skjæringspunktet mellom avsluttet og ny sprint, mens daily standup ble gjennomført de dagene det var forventet fysisk oppmøte av gruppemedlemmene.

Fordelingen av roller innenfor Scrum så slik ut:

Rolle	Antall
Scrum Master	- 1
Intern Product Owner	- 1
Ekstern Product Owner	- 1
Utvikler	- 2

Table 1: Scrum-roller i prosjektet

Det ble gjort et bevisst valg om å ha to PO-er - en intern og en ekstern. Oppdragsgiver (fra Innoveria [3]) ønsket ikke selv å ha full kontroll over product backlog, og

gruppen så derfor behovet for en intern PO som tok hånd om selve styringen av prosjektets Scrum-board på vegne av oppdragsgiver. Oppdragsgiver var til stede på alle sprint review- og sprint planning-møter, og var med på å styre den overordnede retningen for hver sprint, selv om sprint backlog ble håndtert og organisert av den interne PO-en.

Det at gruppen hadde to PO-er fungerte godt for de involverte. Oppdragsgiver hadde ikke bakgrunn innen *agil-programmering*, og hadde ikke et ønske om å ha kontroll over sprintboard og dens [Sprint Backlog \(SBL\)](#). Dette førte også til at utviklerne hadde en intern person å forholde seg til når det oppstod diskusjoner rundt hva som burde prioriteres i løpet av en sprint.

3.3 Arbeidsverktøy (utviklingsprosess)

Teamet har primært brukt syv verktøy for å opprettholde versjonskontroll, kommunikasjon og overordnet design:

- Miro** - Design og idémyldring
- Figma** - Frontend wireframe/UI-design
- GitLab** - Versjonskontroll ([Version Control System](#))
- Discord** - Intern kommunikasjon ⁵
- Draw.io** - Diagram verktøy
- TeamUp** - Felles kalender

Daglig kommunikasjon mellom teamets medlemmer foregikk på en intern *Discord*-server, der hovedkommunikasjonen med oppdragsgiver også fant sted. En slik lavterskel kommunikasjonskanal forenklet prosessen med å avklare tekniske og domenespesifikke spørsmål uten å måtte bruke tid på å formulere formelle e-poster. At oppdragsgiver stilte seg svært tilgjengelig bidro ytterligere til at kanalen fungerte godt i praksis.

GitLab fungerte som hovedverktøy for prosjektstyring. Bruk av GitLabs Scrum-board bidro til en strukturert arbeidsflyt, mens *Miro* og *Figma* ga en lett tilgjengelig referanse for tidlig designarbeid. Alle diagrammer tilhørende applikasjonens dokumentasjon ble laget i *Draw.io*. *TeamUp* ble introdusert som et felles kalenderverktøy der møter, varslet fravær og øvrige hendelser ble registrert og gjort tilgjengelige i nettleser eller via kalenderabonnement.

⁵Bachelorgruppe og oppdragsgiver omtales her som "intern"

3.4 Fordeling av arbeidsoppgaver

Tidlig i prosjektet ble det klart at full oversikt over hele kodebasen for samtlige gruppe-medlemmer ikke var realistisk. Den opprinnelige tanken var at alle skulle jobbe på tvers av kodebasen, men etter hvert som prosjektet utviklet seg, så gruppen nødvendigheten av å spesialisere seg for å opprettholde fremdriften. Basert på erfaringer fra integrasjonsprosjektet (PROG2052) foregående semester - der gruppen jobbet så spesialisert at sykdom hos ett gruppe-medlem utgjorde en reell prosjektrisiko - valgte gruppen en mer moderat tilnærming denne gangen.

Rollefordelingen i gruppen var flytende i praksis. Behovet for arbeidskraft på de ulike områdene varierte gjennom prosjektets faser, og det var ikke alltid hensiktsmessig å arbeide strengt innenfor sin primærrolle. Et eksempel på dette var integrasjonen av backend-funksjonalitet i frontend, der det var naturlig at backend-utviklere bidro for å sikre korrekt bruk av API-ene. Gruppen anser denne fleksibiliteten som en styrke som bidro til jevn fremdrift gjennom hele prosjektet.

For å motvirke risikoen ved spesialisering ble det lagt vekt på grundige code reviews gjennom GitLabs [Merge Request \(MR\)](#)-funksjonalitet. Dette sikret at flere enn én utvikler hadde kjennskap til endringer i kodebasen og bidro til å opprettholde en felles forståelse av systemet på tvers av roller.

3.5 Gjennomføring

Prosjektet fulgte en iterativ arbeidsform (Scrum) der innholdet i hver sprint varierte med prosjektets fase. I de tidlige sprintene dominerte arkitekturbeslutninger, teknologivalg og oppsett av infrastruktur. Etter hvert som grunnlaget var lagt, skiftet fokus mot implementasjon av kjernefunksjonalitet og integrasjon mellom tjenestene. Mot slutten av prosjektet ble prioriteringene dreid mot testing, feilretting og ferdigstillelse av dokumentasjon.

Utviklingsperiode

Utviklingsperioden bestod av syv sprinter à to uker, med unntak av den siste sprinten som var ni dager lengre. Totalt utgjorde perioden 107 dager fra oppstart til innlevert rapport. I forkant av første sprint gjennomførte gruppen en research-periode fra 05.01 til 01.02. Denne perioden er ikke inkludert i den grafiske fremstillingen under, ettersom arbeidsmetodikken på dette tidspunktet ennå ikke var fastslått.



Figure 2: Grafisk fremstilling av prosjektets sprinter (*Grafikk er generert med Claude Design*)

Sprint	Dato	Hovedfokus
1	03.02 – 16.02	Utviklermiljø, teknologistack og sensor-pipeline
2	17.02 – 02.03	Database og collection-service
3	03.03 – 16.03	Autentisering, tenants og sensor-onboarding
4	17.03 – 30.03	ERP -service og context-service
5	31.03 – 13.04	ERP-agent, payload-schema og Context-UI
6	14.04 – 27.04	Ordre, work centers og tenant-isolasjon
7	28.04 – 20.05	Feilretting og rapportskrivning

Table 2: Sprintfordeling med datoer og hovedfokus

Møter

Gruppen etablerte to faste dager med fysisk oppmøte. Dette kan sammenlignes med kontortid i arbeidslivet. Fasilitering av SCRUM-relaterte møter ble dekket under disse dagene:

Møtetype	Beskrivelse av gjennomføring
Daily standup	Kort statusoppdatering per medlem: hva er gjort, hva er planlagt, og eventuelle blokkerende problemer. Lengre diskusjoner ble utsatt til etter standup.
Sprint review	Ble gjennomført ved slutten av hver sprint, med oppdragsgiver tilstede.
Sprint planning	Gjennomført i to deler: først et review av foregående sprint med oppdragsgiver, deretter intern planlegging av hvilke issues som skulle trekkes fra Product Backlog til SBL.
Fysiske arbeidsdager	Felles arbeid og diskusjon utover standup. Særlig verdifullt i startfasen for å etablere felles forståelse av løsningen som skulle utvikles.

Table 3: Faste møteformater i prosjektet

Gruppen erfarte at selve programvareutviklingen stagnerte noe på møtedagene sammenlignet med individuelle arbeidsdager. Dette ble likevel ansett som en styrke. De fysiske møtene ga rom for grundige diskusjoner som gjorde de individuelle arbeidsdagene desto mer effektive.

Timeføring

Gruppen satte seg et mål om å utføre 30 arbeidstimer per uke. Kravet var aldri satt opp til at man måtte oppnå 30 timer hver uke, men heller et mål om å nå omtrent 30 timer i gjennomsnitt innen slutten av bacheloroppgaven. For å kunne styre hvor mye man selv og resten av gruppen hadde jobbet, ble det opprettet et dokument for føring av timer. Her skulle man føre opp antall timer, fordelingen mellom typer arbeid og et kort notat for konkret hva man jobbet med hver dag. Dette førte til at man hadde oversikt over hva man hadde gjort og hvor mye man burde jobbe pågående uken. I tillegg gjorde det vanskeligere å feilrapportere om utført arbeid. Timeloggen ligger i vedlegg [E](#). Diskusjon av timeloggen ligger i kapittel [9](#).

Branch-logikk

Prosjektet benyttet en *feature branch*-basert arbeidsflyt, der all kodeendring ble gjort i en egen branch og deretter merget inn i hovedbranchen gjennom en MR. Dette gjorde det enkelt å isolere endringer og opprettholde god versjonskontroll.

Det ble tidlig etablert standarder for commit-meldinger, basert på et system beskre-

vet i [8] og tilpasset prosjektets behov:

Type	Beskrivelse
feat	Ny feature du legger til i applikasjonen
fix	En bugfix
style	Feature og oppdateringer relatert til styling
refactor	Refaktoring av en spesifikk seksjon av kodebasen
test	Alt relatert til testing
docs	Alt relatert til dokumentasjon
chore	Vedlikehold av kode

Table 4: Standard commit-typer brukt i prosjektet

En feilretting ville for eksempel få følgende commit-melding:

```
git commit -m "fix: calculation of kwh"
```

Dokumentasjonen av commit-standardene ble lagt i repositoriets `docs`-mappe slik at den var tilgjengelig for teamet under utviklingen [G](#).

Repositoriet var delt opp i to hovedbrancher: *main* og *dev*. *main* representerte produksjonsklar kode, mens *dev* fasiliterte aktiv utvikling og var branchen det ble branchet ut fra under arbeidet. Ettersom det blant annet ikke ble satt opp en [Continuous Development](#)-pipeline for prosjektet, ble heller ikke separate servermiljøer for de to branchene etablert. *dev*-branchen ble derfor benyttet som den aktive branchen på prosjektets server, og all testing mot server foregikk på *dev*-kode — noe som i praksis reduserte nytten av *main* som dedikert stabil branch. For et prosjekt uten reelle sluttbrukere i produksjon var dette en akseptabel løsning. I et scenario med aktive brukere ville det derimot vært nødvendig med både en CD-pipeline og dedikerte miljøer for begge branchene, slik at ny funksjonalitet kunne testes isolert uten å påvirke et stabilt produksjonsmiljø.

GitLab - issues og labels

I GitLab-repositoriet ble *labels* brukt til å gi rask og tydelig informasjon om en oppgaves tilstand, forventet arbeidsmengde og type. Bruken av labels hjalp gruppen med å kategorisere oppgavene på en strukturert måte og var særlig nyttig i planleggingen av kommende sprinter. *Milestones* ble brukt for å knytte issues opp mot spesifikke sprinter.

Utviklingsstadium	Prioritet	Størrelse	Type	Øvrig
PBL	pri::high	size::large	type::feat	backend
SBL	pri::medium	size::medium	type::bug	frontend
in progress	pri::low	size::small	type::admin	
in review			type::rapport	
blocked			type::research	

Table 5: Kategorisering av labels brukt i prosjektets issue board

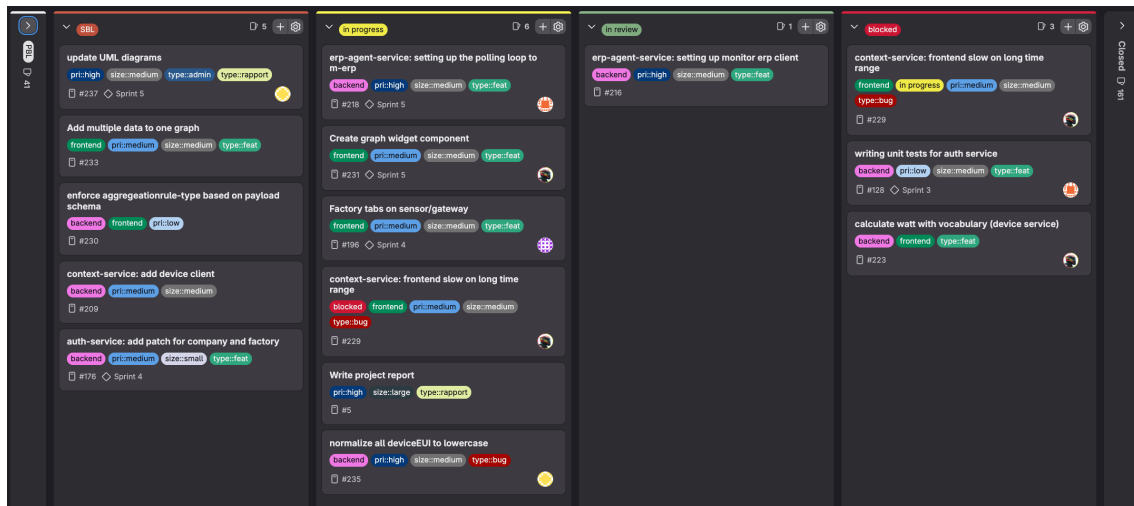


Figure 3: Issue board brukt i utviklingsprosessen (03.04.2026)

Ved prosjektets avslutning hadde gruppen totalt 390 issues registrert i prosjektets SCRUM-board. Totalt har prosjektet 2034 commits og 220 gjennomførte MRer. Dette inkluderer hele spekteret - fra PBL til issues markert som *closed*. Fordelingen så slik ut:

PBL	4 issues
SBL	10 issues
In progress	0 issues
In review	0 issues
Blocked	4 issues
Closed	372 issues

3.6 Bruk av KI

Prosjektets omfang, med utvikling av flere mikrotjenester, et administrasjonspanel og tilhørende infrastruktur innenfor en begrenset tidsramme, gjorde at teamet valgte å benytte Kunstig Intelligens (KI) aktivt og systematisk gjennom hele utviklingsprosessen. KI ble ikke anvendt som en erstatning for faglig vurdering, men som

et verktøy for å effektivisere arbeidet og redusere tid brukt på oppgaver som ikke krevde dyp domenekunnskap.

Før og under utviklingen ble potensielle løsninger diskutert med KI, noe som ga gruppen rask orientering i domener og teknologier teamet hadde begrenset erfaring med fra før. Selv om KI ikke alltid ga de mest presise svarene, viste denne bruken seg å ha stor verdi som et innledende verktøy for å kartlegge ukjente fagområder.

KI bidro også til tidsbesparelser ved repetitive oppgaver. Oppsett av nye mikrotjenester krever store mengder boilerplate-kode som i stor grad er identisk på tvers av tjenestene. Ved aktiv bruk av KI gjorde gruppen det mulig å generere slik kode raskt uten at utviklerne måtte skrive det manuelt fra bunnen av.

Kodeagenter

Under utviklingen ble kodeagenter som *OpenCode*[9] og *Claude Code*[10] benyttet aktivt. I motsetning til en vanlig chatbot har kodeagenter tilgang til hele kodebasens filstruktur, noe som gjorde agentene i stand til å bistå i mer komplekse oppgaver som refaktorering, feilsøking og generering av tester. Det vil si at utvikleren ikke manuelt trengte å lime inn relevante kodeseksjoner. Valget mellom de to verktøyene var i hovedsak et spørsmål om personlig preferanse, da begge dekket de samme behovene.

MR-review og dokumentasjon

KI ble benyttet som et supplement i gjennomgang av MR, samt til generering av kodekommentarer og teknisk dokumentasjon. En nærmere vurdering av disse erfaringene, samt refleksjoner rundt bruk av KI i programvareutvikling generelt, diskuteres i kapittel 9.

4 Kravspesifikasjon

Gjennom dialog med Innoveria ble det tidlig tydelig at tre krav stod sentralt: enkel onboarding av gateways og sensorer, tilbakemelding på at disse fungerer og mottar data, en meningsfull fremstilling av sensordata satt i kontekst med produksjonsdata fra *ERP*-systemet, samt skalerbarhet. Innoveria understreket at terskelen for å legge til nye sensorer og gateways måtte være lav, slik at en person på fabrikken uten dyptgående teknisk kompetanse skulle kunne koble til en ny sensor og verifisere at den fungerer uten å være avhengig av betydelig støtte fra Innoveria eller utviklere. I tillegg ble det tidlig klart fra Innoverias side at hovedfokuset ikke skulle ligge på å utvikle et visuelt avansert frontend, men heller prioritere funksjonalitet.

4.1 Use Cases

For å visualisere de viktigste funksjonelle kravene til applikasjonen har gruppen valgt å benytte et use case-diagram. Vi har erfaring med denne fremstillingen fra tidligere prosjekter og opplever at den fungerer godt for å gi en strukturert og oversiktlig fremstilling av de viktigste handlingene en bruker kan utføre i applikasjonen.

Under use case-diagrammet finnes også et utvalg av use casene beskrevet nærmere i form av utvidede use cases. Disse er inkludert for å gi en mer detaljert beskrivelse av enkelte use cases. Samtlige use cases har minst én high-level use case-beskrivelse i vedlegg F. I løpet av prosjektperioden har use-case diagrammet utviklet seg. En versjon av diagrammet slik det så ut i starten av prosjektet finnes i den vedlagte Prosjektplanen

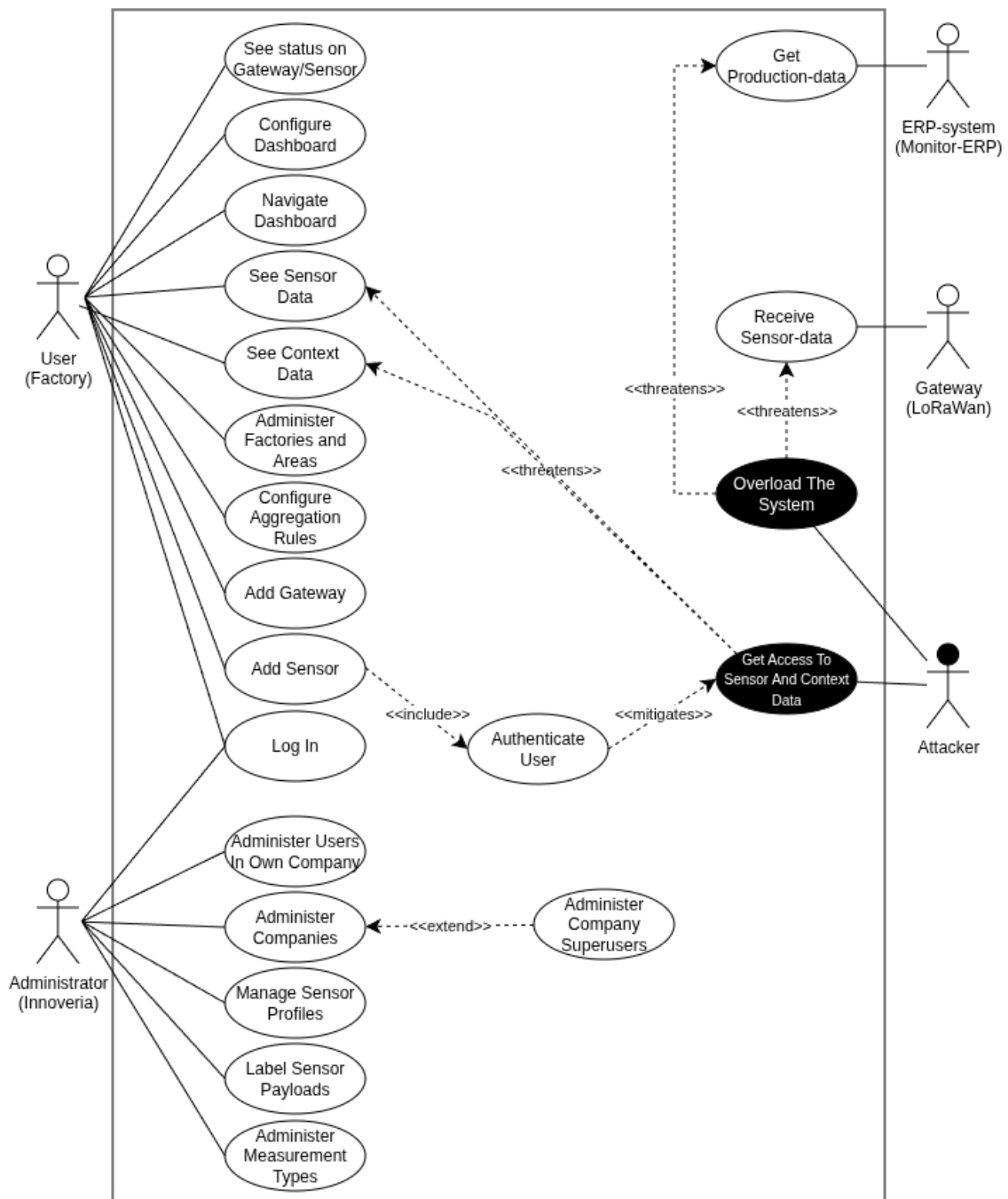


Figure 4: Use Case Diagram

Use case-beskrivelser

Følgende er to av de mer komplekse use casene beskrevet nærmere som utvidede use case-beskrivelser.

Use case: Se kontekstdata

Omfang: Web-applikasjon / frontend

Aktører: Bruker, Administrator

Mål: Se data satt i kontekst av produksjons- og sensordata

Beskrivelse: Brukeren navigerer til kontekst-dashbordet. Her får brukeren mulighet til å velge hvilken ordre det ønskes å se kontekstdata fra, og hvilke utregninger (aggregering) som skal benyttes. Når en ordre er valgt, vises utregningen av energiforbruk per ordre, maskin og produsert enhet. Individuelle datapunkter for ordre-operasjoner vises i form av grafer basert på valgt operasjon tilhørende ordren.

Type: Essensiell

Pre-betingelser:

1. Brukeren er logget inn.
2. Aktuell strømsensor er lagt til og konfigurert korrekt.
3. Eventuelle produksjonsdata er lagret i fabrikkens *ERP*-system.

Programflyt:

1. Brukeren navigerer til kontekst-dashbordet.
2. Brukeren velger ønsket ordre.
3. Kontekstdata vises i et dashboard på en ny side.
4. Brukeren *kan* videre trykke på "*Show graphs*" for å vise individuell sensordata per ordre.
5. Brukeren kan videre holde musepekeren over sensorens graf for å lese av individuelle datapunkter.

Use case: Legg til sensor

Omfang: Web-applikasjon / frontend

Aktører: Bruker, Administrator

Mål: Legge til og verifisere en ny sensor i systemet

Beskrivelse: Fabrikkarbeideren registrerer en ny sensor via webgrensesnittet og får bekreftelse på at sensoren er tilkoblet og sender data korrekt.

Type: Essensiell

Pre-betingelser:

1. Brukeren er logget inn
2. En gateway er allerede registrert og aktiv i systemet
3. Sensoren er fysisk plassert innenfor gatewayens rekkevidde

Post-betingelse: Sensoren vises i systemet, og brukeren har bekreftet at data mottas.

Programflyt:

1. Brukeren navigerer til sensoroversikten.
2. Brukeren trykker på “Legg til sensor” og fyller inn påkrevd informasjon (navn, device EUI, application key, fabrikk/fabrikkområde og sensorprofil).
3. Systemet registrerer sensoren og viser den i oversikten.
4. Brukeren kan se at sensoren mottar data ved at siste måletidspunkt oppdateres i grensesnittet.

4.2 Domenemodell

Domenemodellen ble opprettet tidlig i planleggingsfasen av prosjektet. I starten brukte gruppen betydelig tid på å fastsette kravene til applikasjonen. Dette har medført at domenemodellen har utviklet seg underveis i prosjektet. Det er derfor forskjeller mellom domenemodellen som presenteres her og den som ligger som vedlegg i prosjektplanen A.

De største forskjellene er at implementasjonsdetaljer er fjernet fra modellen for å gjøre den enklere å lese. I tillegg har flere klasser blitt lagt til i systemet. Ag-

gregeringene mellom enkelte klasser har også blitt endret etter hvert som gruppen har fått bedre forståelse for teknologien som benyttes i prosjektet, spesielt LoRaWAN-teknologi.

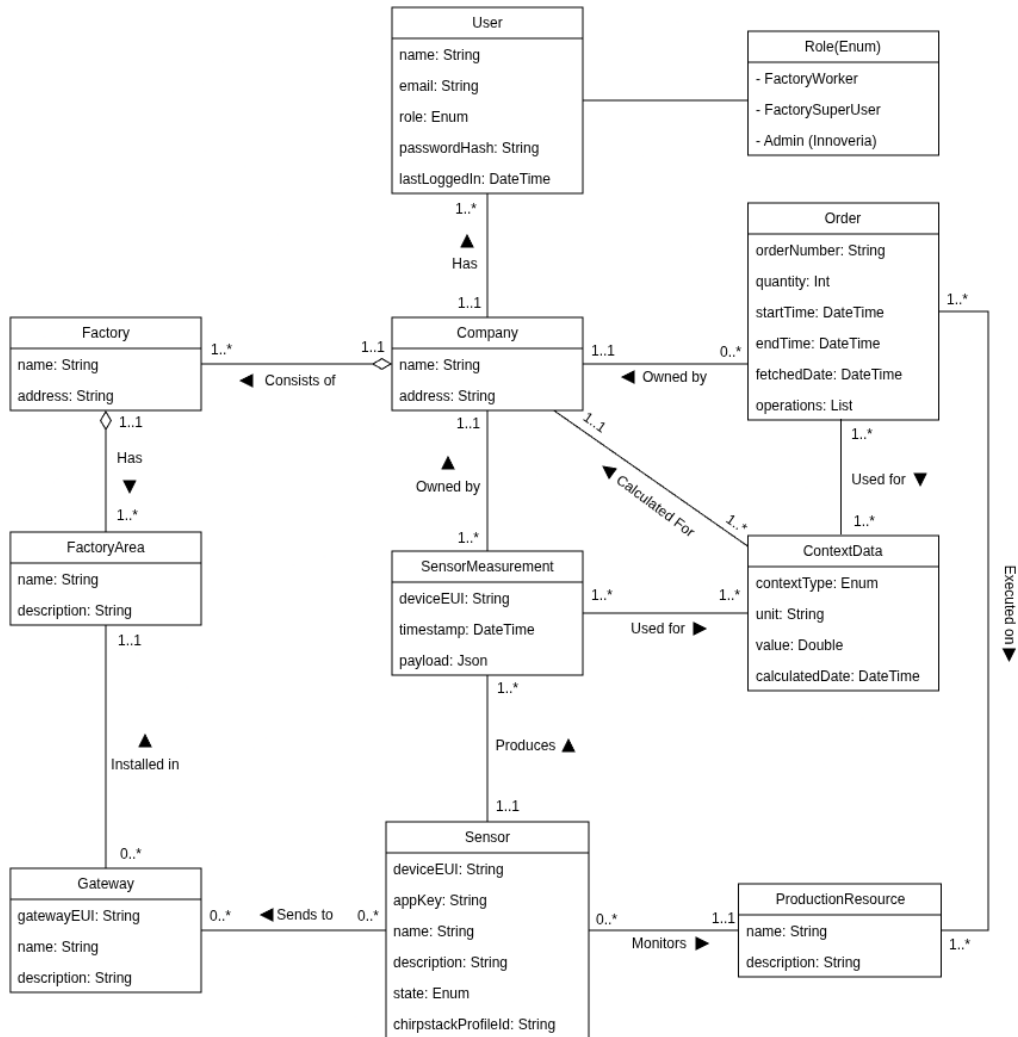


Figure 5: Domenemodell

4.3 Operasjonelle krav

De operasjonelle kravene ble etablert i samarbeid med oppdragsgiver. Kravene er utarbeidet basert på oppdragsgivers ønsker, kombinert med våre anbefalinger og innspill, forankret i vår teknologiske kompetanse og innsikt. Gjennom diskusjon av ulike scenarier, utforsking av muligheter og avklaring av hva som er akseptabelt for prosjektet, har gruppen kommet frem til følgende krav:

1. Systemet skal kunne håndtere 300 samtidige brukere.

-
- Systemet er tiltenkt brukt av små og mellomstore bedrifter med titalls ansatte. Systemet er heller ikke et primærsystem hos kundene; for eksempel benyttes andre systemer til ordrebehandling i fabrikkene. Bruken av vårt system vil derfor være sporadisk, og antall samtidige brukere vil være betydelig lavere enn antall ansatte på jobb. Den største trafikken vil komme fra sensorene.
2. Systemet skal kunne motta 600 sensormålinger per time.
 - Sensorene sender målinger med ulike tidsintervaller. Enkelte sensorer sender for eksempel én gang i timen, mens andre sender hvert minutt. Av den grunn er sensormålinger per time valgt fremfor antall aktive sensorer som krav.
 3. Svartid fra backend under normale forhold skal være under ett sekund. Med normale forhold menes at brukeren har tilgang til en nettverksforbindelse med gjennomsnittlig hastighet og uten større feil. Dersom svartiden overstiger ett sekund, kan brukeren oppleve avbrudd og ventetid, noe som gir en dårligere brukeropplevelse. [11]
 4. Det er viktig for Innoveria at systemet har høy oppetid, da det skal brukes i den daglige driften i fabrikkene. Fabrikker opererer ofte med både dag- og kveldsskift, og enkelte har også nattskift som kan ha behov for systemet. Gruppen har derfor kommet frem til at oppetiden skal være på 99,5%. Oppetid er særlig viktig på dagtid. Dersom nedetid er nødvendig for oppdateringer, bør dette planlegges til nattestid.

4.4 Sikkerhetskrav

I løpet av prosjektperioden har det blitt gjort enkelte valg som har påvirket sikkerhetskravene. Innledningsvis var målet å implementere rollebasert innlogging med tre roller: administrator, superbruker og bruker, der brukere skulle ha færre tilganger enn superbrukere ved hver fabrikk. På grunn av tidsbegrensninger i prosjektet har gruppen valgt å implementere en forenklet versjon av dette med to roller: administrator og bruker. Dette er de mest sentrale sikkerhetskravene i applikasjonen.

1. Det skal ikke være mulig for uvedkommende å få tilgang til systemet. Alle brukere må ha en brukerkonto opprettet av Innoverias administratorbruker.

-
2. Selskaper som bruker applikasjonen skal kun ha tilgang til data som tilhører deres eget selskap. Dette gjelder blant annet brukere, fabrikker, sensorer og sensormålinger.
 3. Administratoroperasjoner, som behandling av brukere og selskaper, skal være beskyttet ved rollebasert tilgang.
 4. Innloggingen skal være tidsbegrenset ved bruk av kortlevde access tokens og lengrelevde refresh tokens.
 5. All datatrafikk skal skje over TLS for å forhindre at kriminelle aktører innsiserer konfidensiell bedriftsdata over nettverket. Her benyttes den eksterne aktøren Let's Encrypt.
 6. Applikasjonen skal lagre så lite personlig informasjon som mulig. Det skal kun benyttes intern ID, brukernavn, e-post og passord til identifisering av brukere.
 7. Applikasjonen skal kun innhente nødvendig informasjon fra ERP-systemet den er koblet til, for å unngå lagring av unødvendige data.

For å imøtekomme kravene 1–3 har gruppen implementert autentisering på alle endepunkter gjennom backends API-gateway. Dette innebærer at alle operasjoner bak disse, inkludert databasebehandling, er sikret. Mer om implementasjonen er beskrevet i 7.3. For mer detaljer om implementering av tidsbegrenset innlogging med access tokens, se 7.4.

Kravene 5–6 er ivarettatt gjennom datamodellene og implementasjonen av mikroservicene i applikasjonen som behandler data, spesielt auth-service 7.4 og erp-service 7.7.

4.5 Grensesnittkrav

Brukergrensesnitt

Applikasjonen utvikles med engelsk som språk. Bakgrunnen for dette er at det arbeider personer uten gode norskkunnskaper i industrien. Ved å benytte engelsk sikres det at flest mulig kan bruke applikasjonen. Støtte for flere språk vil ikke bli utviklet, da dette ikke er prioritert i prosjektet.

Applikasjonen skal fungere som en webapplikasjon og er beregnet for bruk på PC-skjermer. Grensesnittet bør være responsivt og tilpasse seg ulike skjermstørrelser. Det er ikke prioritert å gjøre applikasjonen responsiv for mindre, mobile enheter.

Figur 6 viser et representativt eksempel på en av wireframsene som ble utviklet tidlig i prosjektet. Øvrige skjermbilder fra designfasens wireframes finnes i Vedlegg C.

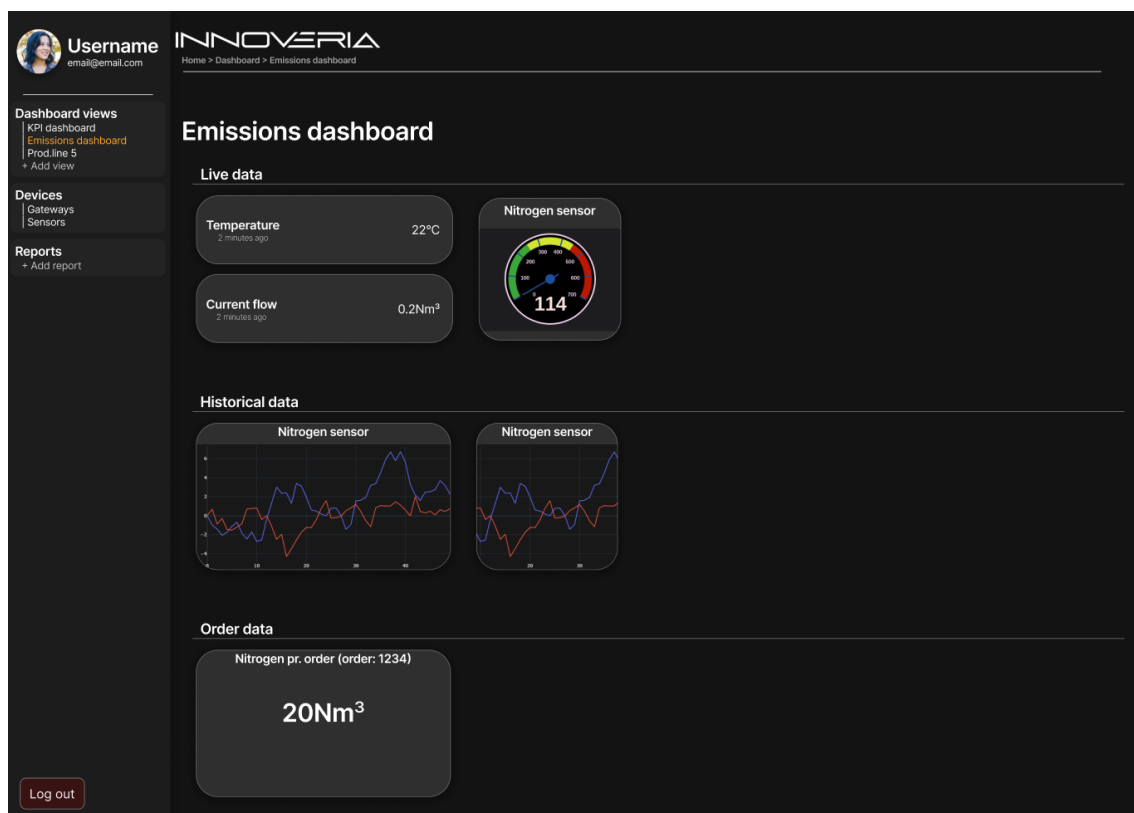


Figure 6: Wireframe — CO2-dashboard

Systemgrensesnitt

Hver mikrotjeneste eksponerer et API dokumentert med Swagger. Dokumentasjonen genereres automatisk og er tilgjengelig som en samlet oversikt når systemet kjøres lokalt på port 8081. En fullstendig oversikt over alle endepunkter som er dokumentert i Swagger finnes som en tabell i Vedlegg H.

5 Teknologianvendelse

5.1 Valg av teknologisk retning

Valg av teknologi var en krevende del av prosjektet, fordi applikasjonen skal dekke et bredt spekter av funksjonaliteter, hvor blant annet datainnhenting via LoRaWAN-sensorer, behandling av historiske data og en sammenslåing av produksjonsdata og sensordata representerer et nytt fagfelt for gruppen.

I tillegg benytter Innoveria et eksternt konsultentselskap som allerede har utviklet deler av en tilsvarende løsning. Denne løsningen er basert på andre teknologivalg, blant annet bruk av Node.js i backend og en innebygd LoRaWAN-nettverksserver på gateway. Det finnes flere grunner til at plassering på gateway-nivå er utfordrende. En sentral begrensning er at det gjør det vanskelig å legge til nye sensorer og gateways på en fleksibel måte, ettersom hver sensor må konfigureres og håndteres manuelt på gateway. Innoveria la ekstra vekt på håndtering av LoRaWAN-enheter, og løsningen er derfor spesielt fokusert på dette området.

Plattformvalg

Ettersom prosjektet skal videreutvikles, førte dette til diskusjon rundt valg av teknologi, spesielt knyttet til håndtering av LoRaWAN-enheter og valg av backend-språk. Innoveria var samtidig åpne for nye tilnærminger og ga oss i stor grad frihet til å gjøre egne teknologiske valg.

Løsningen vår ble utviklet som en fullstack webapplikasjon, bestående av en frontend implementert i React og TypeScript, samt en backend i Go for håndtering av API-er og databehandling. I tillegg benyttet ChirpStack for administrasjon og håndtering av LoRaWAN-enheter. Til drift brukes Docker som kjører på NTNU sine private servere, OpenStack. Tabellen under gir en oversikt over de sentrale teknologiene benyttet i utviklingsmiljøet.

Teknologi	Bruksområdet
React-typescript	Frontend programmeringsspråk
Material UI	UI rammeverk
Golang	Backend programmeringsspråk
ChirpStack	LoRaWAN Nettverks Server
Mosquitto	MQTT broker
PostgreSQL/TimeScaleDB	Datalagring/Datalagring av tidsseriedata
Swagger	API dokumentasjon
Tilt	Samlet visning av mikrotjenester
Postman	Testing av API endepunkter
Golangci-lint	Linting
Bun	Typescript toolkit
VSCoDe/Nvim	Kode-editor
Opencode/Claude code	MR Review, kodeforståelse, koding
Git/Gitlab	Versjonskontroll/prosjektstyring
Docker	Isolert kjøring av applikasjoner i utvikling og drift
Openstack	Drift av applikasjon

Table 6: Utviklingsverktøy

5.2 Go som backend-teknologi

Valget av backend-programmeringsspråk stod mellom Node.js og Go. Det ble tidlig avklart at prosjektet i stor grad ville bestå av utvikling av et omfattende API. For API-utvikling har begge språkene god dokumentasjon og er robuste teknologier for å bygge API-er. Likevel falt valget på Go, basert på erfaring og kjennskap til språket fra studieløpet, samt hvordan det håndterer samtidighet.

Golang-arkitekturen er designet for samtidighet, noe som gjør det godt egnet for systemer som håndterer flere samtidige LoRaWAN-enheter [12]. Gjennom bruk av "goroutines", som er lette tråder for parallell kjøring, kan flere operasjoner utføres effektivt samtidig.

Samtidighet ble en viktig faktor i løsningen, ettersom systemet består av flere tjenester og prosesser som opererer parallelt. Løsningen håndterer blant annet kommunikasjon fra LoRaWAN-enheter og ERP-synkronisering med dataavstemming var noen sentrale problemer. Disse operasjonene må kunne behandles parallelt uten å blokkere resten av systemet.

På den andre siden er Node.js en event drevet arkitektur. Det er en ikke-blokkerende arkitektur basert på en enkelttrådet event loop. Selv om det er utfordrerne ut å kjøre Node.js parallelt er det fortsatt mulig med modulen "worker threads", men dette introduserer økt kompleksitet i håndtering av samtidighet. Go håndterer samtidighet

på en mer direkte og strukturert måte, noe som ble en avgjørende faktor i valg av backend-løsning.

5.3 React for frontend-utvikling

En rekke teknologier ble valgt for frontend. Enkelte av disse ble valgt på bakgrunn av ønske fra oppdragsgiver, mens andre valg er diskutert og begrunnet nedenfor:

React

*React*⁶ er et open source-bibliotek utviklet av Meta⁷, som nylig overførte eierskapet av prosjektet til *React Foundation*[13]. React har etablert seg som et av de mest utbredte bibliotekene for utvikling av brukergrensesnitt på web- og native-plattformer. Ifølge *Stack Overflow Developer Survey 2025*[14], hvor respondentene kunne oppgi flere rammeverk, oppga 44,7% av 23 678 utviklere at de hadde benyttet React det siste året og/eller ønsket å benytte det i det kommende året.

React ble valgt på bakgrunn av ønske fra oppdragsgiver i tillegg til bibliotekets modne økosystem og utbredte industriadopsjon, noe som gir grunnlag for å anta at det vil forbli en relevant teknologi i overskuelig fremtid.

TypeScript

*TypeScript*⁸ er et sterkt typet programmeringsspråk som bygger på JavaScript. TypeScript ble valgt som hovedspråk i frontend-utviklingen, da det tilfører funksjonaliteter utover det JavaScript tilbyr. Feilhåndteringen er mer presis og gir muligheten til å definere syntaks som bidrar til bedre type safety. Generell bruk av typer gjør det enklere å definere hvordan variabler skal brukes, og fører til en mer strukturert utviklingsprosess som bidrar til at gruppen får bedre forståelse for hva variabler representerer. TypeScript var i tillegg et språk gruppen ikke var like kjent med, i motsetning til JavaScript som ble lært i tidligere semestre, og utgjorde dermed også en god læringsmulighet.

⁶<https://react.dev/>

⁷<https://opensource.fb.com/projects/react/>

⁸<https://www.typescriptlang.org/>

Bun

*Bun*⁹ er en JavaScript-runtime og pakkebehandler utviklet med ytelse som hovedprioritet. Sammenlignet med *npm*¹⁰ er Bun betydelig raskere ved installasjon av avhengigheter og kjøring av skript. Gruppen anser pakkehåndteringen i Bun som en betydelig fordel, ettersom verktøyet kombinerer runtime, pakkebehandler og bundler i én enhet. Dette reduserer antall avhengigheter i utviklingsmiljøet og forenkler oppsett og konfigurasjon, noe som effektiviserer den daglige utviklingsflyten.

Tailwind

*Tailwind CSS*¹¹ er et utility-first CSS-rammeverk som muliggjør rask stilsetting direkte i markup. Tailwind var i utgangspunktet ment som hovedløsning for CSS-styling i applikasjonen, men ettersom MUI har sitt eget styling-system og ikke benytter Tailwind-klasser, ble bruken gradvis redusert. I praksis ble Tailwind hovedsakelig brukt til overordnet sideoppsett og strukturering - eksempelvis flex-containerer, spacing og høyde- og breddejusteringer - mens selve komponentstylingen ble overlatt til MUI.

MUI

*Material UI (MUI)*¹² er et React-komponentbibliotek basert på Googles Material Design-system. MUI ble valgt som hovedkilde for UI-komponenter på bakgrunn av det brede utvalget av ferdige og tilgjengelige komponenter, som reduserer behovet for å utvikle grensesnitt fra bunnen av. Biblioteket var særlig interessant grunnet enkle integrasjonen med React, åpen kildekode og muligheter for å tilpasse komponenter med egendefinerte egenskaper. MUI X (et MUI-bibliotek for grafvisualisering) er et *open code*-bibliotek med MIT-lisens for grunnfunksjonalitet, mens enkelte avanserte funksjoner (for eksempel zoom-mulighet i grafer) krever en kommersiell lisens¹³.

Andre biblioteker som *Shadcn*¹⁴ ble også vurdert, men gruppen vurderte MUI-bibliotekets forhåndsdefinerte stilsett som nærmere det ønskede utseendet for prosjektets frontend. Bruken av et UI bibliotek medførte at mindre tid måtte settes av til å definere egne komponenter, slik at fokuset kunne rettes mot utvikling av

⁹<https://bun.com/>

¹⁰<https://www.npmjs.com/>

¹¹<https://tailwindcss.com/>

¹²<https://mui.com/material-ui/>

¹³<https://mui.com/x/introduction/licensing/>

¹⁴<https://ui.shadcn.com/>

funksjonalitet og øvrige UI-aspekter.

5.4 Database

PostgreSQL

For vedvarende lagring valgte gruppen å benytte en SQL-basert database. Etter å ha brukt mye tid på å bygge forståelse av domenet, ble det tydelig at dette egnet seg godt i en relasjonsbasert database.



Figure 7: Utkast av database diagram

Figur 7 viser et utkast for et domene gruppen jobbet med i tidlig utvikling fase. Diagrammet illustrer noen sentrale felter for sensorer, gateways og selskaper, samt relasjonene mellom disse. Gjennom domene forståelsen ble det tydelig at dataene i stor grad er strukturert og består av tydelige relasjoner. Det gjorde en relasjons database til et naturlig valg.

Videre ble PostgreSQL valgt som databasesystem, blant annet på grunn av god støtte i utviklingsmiljøet og pålitelige drivere for Go. En viktig kvalitet er også støtten for ulike typer databehandling. Oppgaven omfatter flere datatyper og bruksområder, inkludert lagring av tidsseriedata, håndtering av autentiseringsdata, samt et komplekst system for å håndtere avhengigheter gjennom dataavstemming. Mer om dette i kapittel 7.7. PostgreSQL har støtte for disse bruksområdene og ble valgt for å kunne håndtere databehandlingen i en plattform, noe som reduserer behovet

for tredjepartsløsninger og bidrar til en mer oversiktlig systemarkitektur.

TimescaleDB

Sensordataene i applikasjonen er tidsseriedata, det vil si målinger som kontinuerlig akkumuleres over tid og primært leses som tidsintervaller. En vanlig relasjonsdatabase som PostgreSQL kan håndtere dette, men ytelsen vil gradvis reduseres etter hvert som datamengden øker. Tabeller i en tradisjonell relasjonsdatabase er ikke optimalisert for dette tilgangsmønsteret.

Gruppen valgte derfor TimescaleDB [15] som løsning. TimescaleDB er en åpen kildekode-utvidelse av PostgreSQL som introduserer såkalte *hypertables*. Dette er tabeller som automatisk partisjoneres i mindre segmenter kalt *chunks*. Dette gjør det mulig å utføre spørringer på tidsintervaller betydelig raskere, ettersom databasen kun trenger å skanne relevante partisjoner fremfor hele tabellen.

En sentral fordel med TimescaleDB er at det er en utvidelse av PostgreSQL, og ikke et separat databasesystem. Dette gjorde at gruppen kunne aktivere funksjonaliteten i den eksisterende PostgreSQL-databasen uten å introdusere ny infrastruktur, samtidig som standard SQL kunne benyttes med minimalt ekstra oppsett.

```
1 SELECT create_hypertable('coll.sensor_measurement', 'timestamp');
```

Listing 1: Sette opp hypertable

Som vist i 1 er det tilstrekkelig med ett funksjonskall for å konvertere en eksisterende tabell til en hypertable. TimescaleDB tar deretter over ansvaret for partisjonering og komprimering av data over tid. Dette reduserer behovet for lagringsplass og bidrar til lav spørretid selv ved store datamengder.

5.5 Nettverksprotokoller

HTTP og HTTPS

Systemet følger en standard klient-tjener-modell der HTTP fungerer som nettverksprotokoll. For løsningen er datasikkerhet viktig, spesielt på grunn av behandling av reelle produksjonsdata. Dermed benyttes HTTPS via Caddy for ekstern kommunikasjon mellom brukerens nettleser og serveren. Caddy fungerer som en omvendt proxy og håndterer TLS-kryptering på lik linje med Nginx. Grunnen til at gruppen valgte Caddy er dens automatiske håndtering av TLS-kryptering, og at oppsett av omvendt proxy krever minimal konfigurasjon, som vist under.

```
handle {  
    reverse_proxy frontend:3000  
}
```

Listing 2: Oppsett av omvendt proxy

Internt i systemet er det flere tjenester som kommuniserer med hverandre. Her valgte gruppen å ikke sikre HTTP med TLS-kryptering. Dette skyldes at systemet kjører innenfor et lukket nettverk på NTNU sine servere, og at det ville krevd betydelig mer tid å utvikle et sikkert internt kommunikasjonsnettverk mellom tjenestene. For å imøtekomme OWASP-kravet om at all kommunikasjon skal være sikker uavhengig av nettverks plassering [16], må dette implementeres i videreutviklingen.

MQTT

For kommunikasjonsprotokoll for sensordata valgte gruppen å benytte MQTT. Dette er en lettvekts meldingsprotokoll designet for ressursbegrensede enheter og nettverk med lav båndbredde [17]. MQTT følger en publiser/abonner-arkitektur, der sensorer publiserer data til en MQTT-broker. For å motta data er det implementert en dedikert tjeneste som abonnerer på relevante emner.

Et MQTT-emne fungerer som en kanal for meldinger. En publiser oppretter et emne for å identifisere hva slags data som sendes. Klienter kan deretter abonnere på de emnene de er interessert i, noe som gjør det mulig å motta kun relevante meldinger uten å håndtere all trafikk i systemet. Dette passer godt for sensordata, siden emnebasert kommunikasjon gjør det mulig for tjenester å hente kun de dataene de trenger.

En sentral fordel med MQTT er den lave overheaden, ettersom protokollen er designet for små datamengder som blant annet LoRaWAN-packets. I dette systemet flyter data hovedsakelig én vei, fra sensor til backend i en asynkron flyt. Sensoren bestemmer selv når data sendes og er ikke avhengig av at systemet responderer. Dette gjør MQTT godt egnet, ettersom det reduserer belastningen på nettverket og skalerer godt med økende antall sensorer.

Til sammenligning kunne websockets vært brukt, ettersom det også muliggjør asynkron kommunikasjon. Likevel introduserer websockets høyere overhead ved å opprettholde datastrøm i begge retninger, noe som er unødvendig i dette tilfellet hvor kommunikasjonen i hovedsak går en vei [18].

5.6 ChirpStack

Gruppen valgte å bruke ChirpStack som LoRaWAN Nettverks Server. Da gruppen undersøkte ulike nettverksservere for LoRaWAN, fremstod ChirpStack som den mest utbredte gratisløsningen. Den hadde god dokumentasjon og støtte for integrasjon gjennom et API. Det var viktig for løsningen å ha et programmeringsgrensesnitt i samme nettverk som resten av løsningen. Dette gjorde det mulig for gruppen å bruke kode til å legge til nye sensorer og gateways på en sentralisert plattform, noe som ellers ville vært umulig dersom nettverksserveren lå på gatewayen. Den håndterer også duplisering av sensordata, som ellers ville vært et problem dersom en produksjonsbedrift benytter flere gateways [19].

Det finnes to løsninger som nevnt i 5.1: Nettverksserveren kan kjøres lokalt på gateway eller driftes som en del av systemets infrastruktur. Å drifte en slik løsning på gatewayen gjør det utfordrende å behandle LoRaWAN-enheter. Dersom en nettverksserver hadde vært installert på gatewayen, ville løsningen blitt sårbar ved feil eller tap av gateway. Å drifte en ChirpStack-instans gjør det enklere å skalere løsningen med flere gateways og sensorer over tid.

Selv om ChirpStack dekker de fleste behov, ble det identifisert enkelte utfordringer knyttet til biblioteket for sensordekodere. Plattformen tilbyr et begrenset utvalg, noe som førte til at gruppen måtte lete etter sensordekodere på produsentens nettside. Et alternativ kunne vært The Things Stack, som tilbyr et omfattende LoRaWAN-nettverks-økosystem med ferdigdefinerte dekodere. Dette ble likevel valgt bort på grunn av kostnader og et ønske om å unngå å basere løsningen på en ekstern tredjepartsplattform.

6 Systemarkitektur

6.1 Systemoversikt

Løsningen følger en mikrotjenestearkitektur, der ulike komponenter er separert som egne tjenester. På et overordnet nivå er arkitekturen sammensatt av tre hoveddeler: brukergrensesnittet og inngangspunktet til systemet, de interne tjenestene, samt de eksterne LoRaWAN-enhetene og produksjonsdata fra Monitor ERP.

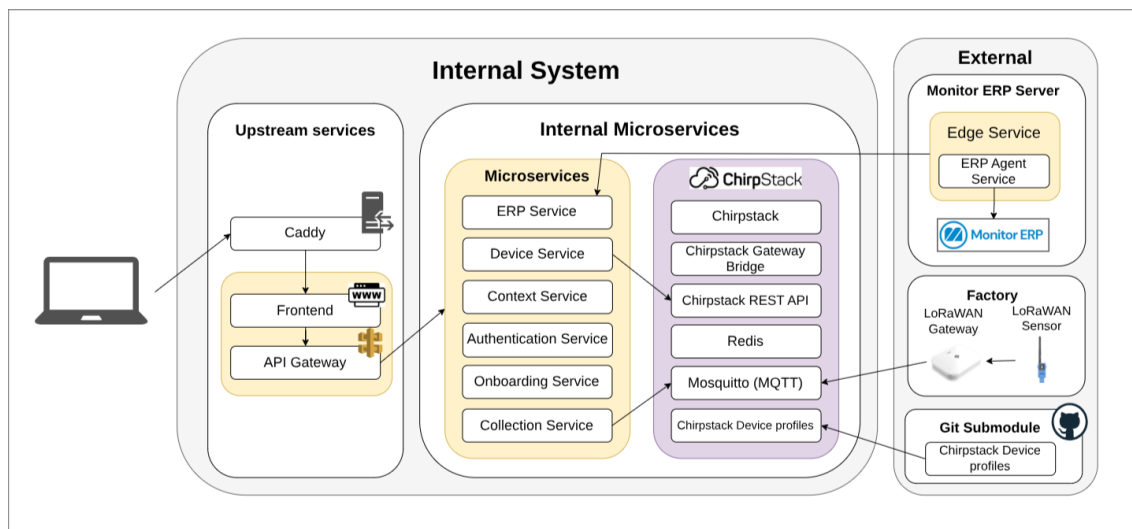


Figure 8: Systemoversikt

Figur 8 viser hvordan brukertilgang gjennom systemet starter med Caddy, som fungerer som en omvendt proxy til å motta klient forespørsler i en nettleser. Trafikken rutes videre til frontend-applikasjonen, som kommuniserer med API-gateway. Den fungerer som knyttetpunkt mellom frontend og backend, og abstraherer de interne tjenestene og sørger for at klienten ikke trenger direkte tilgang til hver tjeneste. De interne tjenestene er satt opp til å håndtere ulike deler av systemets forretningslogikk. Det vil si at autentisering, LoRaWAN-integrasjon og produksjonsdata har egne programmeringsgrensesnitt som API-gateway delegerer forespørselen til.

På den andre siden finnes data tilhørende fabrikker. For å hente produksjonsdata fra fabrikkene brukes en tjeneste som er installert i Monitor ERP-miljøet. Tjenesten gjør kall til Monitor ERP sitt API og videresender dataen til systemets interne ERP-tjeneste. Av tidsmessige årsaker ble dette et konsept for løsningen ettersom det ikke var nok tid til å drifte en slik tjeneste på serveren til en av fabrikkene. Tjenesten er likevel implementert og kjører internt i systemet med mock-data. Dermed kan vi simulere hvordan systemet ville fungert med produksjonsdata satt sammen med sensordata.

For håndtering av sensordata benyttes ChirpStack, som fungerer som systemets LoRaWAN Nettverks Server. Den er avhengig av korrekt konfigurasjon slik at Gateway er innstilt til sende til riktig IP-adresse, der ChirpStack er tilgjengelig. Gateways mottar LoRaWAN-packets å videresender sensordata via internett til systemet.

Innenfor ChirpStack kan data behandles på to måter. ChirpStack Gateway Bridge kan motta data gjennom UDP, som deretter publiseres til et MQTT-emne i Mosquitto. Alternativt kan gateway bli innstilt på å sende MQTT direkte til Mosquitto. Uavhengig av hvordan Gateway er konfigurert, blir data presentert som MQTT til ChirpStack. Nettverksserveren prosessere deretter data hentet fra Mosquitto, ved å dekode sensordata basert på enhetsprofiler og publisere den strukturerte dataen tilbake til MQTT-broderen på et spesifikt MQTT-emne. Videre abonnerer Collection Service på MQTT-emne og mottar ferdig prosessert sensordata. Sensordata lagres i en tids-seriedatabase og hentes deretter av andre tjenester.

Context Service henter deretter data fra Collection Service og kombinerer disse med data fra ERP Service. Dette innebærer å knytte relevante produksjonsobjekter som produksjonsordre, operasjoner og maskiner med sensordata. Denne berikelsen er kjernen i løsningen og muliggjør en mer avansert analyse for produksjonsbedrifter.

6.2 Mikrotjenestearkitektur

Valget av mikrotjenestearkitektur i denne løsningen er knyttet til systemets behov for å håndtere flere ulike domener, som autentisering, LoRaWAN-enheter, sensordata og integrasjon med ERP-systemer. Disse områdene har ulike krav til databehandling, bruksfrekvens og eksternt integrasjon, noe som gjorde det hensiktsmessig å dele systemet opp i separate tjenester. I utformingen av arkitekturen er det tatt utgangspunkt i etablerte designmønstre fra microservice.io, som har bidratt til å strukturere tjenestene og deres ansvarområder [20].

Figur 9 viser en oversikt over det interne systemet, samt hvordan tjenestene kommuniserer med hverandre.

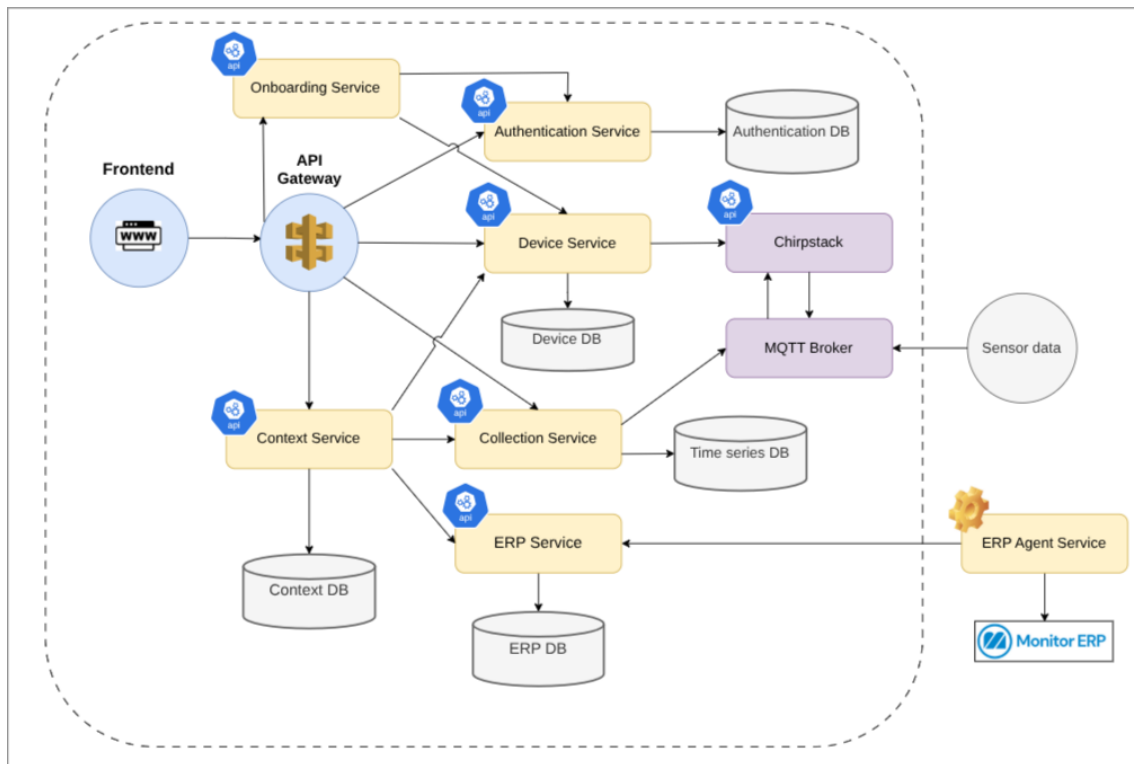


Figure 9: Mikrotjenester

Måten gruppen har delt opp tjenestene og deres ansvarsområder er som følger:

- **API Gateway** er inngangspunktet for klienten og ruter forespørsler til de interne tjenestene. Tjenestens ansvar er å beskytte det interne systemet ved å validere JWT-tokens for autentisering og rate limiting, slik at systemet ikke overbelastes av for mange HTTP-forespørsler.
- **Authentication Service** har ansvar innlogging samt registrering av nye brukere, selskaper, produksjonsbedrifter og tilhørende domener i systemet.
- **Device Service** håndterer konfigurasjon av sensorer og abstraherer funksjonaliteten i ChirpStack, slik at avanserte oppgaver som oppsett av LoRaWAN-innstillinger og enhetsprofiler forenkles for brukeren. Tjenesten utfører API-kall mot ChirpStack for å opprette og konfigurere sensorer og gateway.
- **Collection Service** har ansvar for å motta og håndtere sensordata. Tjenesten abonnerer på ChirpStack sitt MQTT-emne og fanger opp prosessert data. Dataene lagres deretter i en tidsseriedatabase og gjøres tilgjengelige for videre bruk av andre tjenester i systemet.
- **Collection simulator** brukes i utviklingsmiljøet for å simulere sensordata til Collection Service, slik at systemet kan testes og utvikles uten tilgang til ekte

sensorer.

- **Context Service** er bindeleddet mellom IoT-sensordata og ERP-produksjonsdata. Servicen beregner aggregerte målinger i kontekst av utførte produksjonsordrer.
- **Onboarding Service** har ansvar for oppsett av nye selskaper i systemet og fungerer som en orkestrator mellom flere tjenester.
- **ERP Service** håndterer interne produksjonsdata i systemet og har ansvar for relasjoner mellom produksjonsordre, operasjoner, operasjonsrapporter og maskiner. Tjenesten fungerer som en sentral kilde for produksjonsdata og eksponerer disse til Context Service.
- **ERP Agent Service** har ansvar for å hente data fra Monitor ERP og sende dette videre til systemet. Tjenesten kjører på Monitor ERP sine servere og fungerer ikke som et tradisjonelt API, men som en worker som periodisk henter data og gjør POST-forespørsler til ERP Service for videre behandling og lagring.

En viktig begrunnelse for å anvende mikrotjenester er robustheten i systemet. Ved å strukturere løsningen som uavhengige tjenester kan systemet fortsatt fungere selv om én tjeneste er utilgjengelig. For eksempel kan Collection Service fortsatt tilby sensordata selv om ERP Service er nede. Dette er spesielt viktig i en løsning hvor sensordata ikke bør gå tapt. I en monolittisk arkitektur ville en feil i én del av systemet i større grad påvirket hele løsningen.

Mikrotjenestearkitektur introduserer samtidig en rekke utfordringer sammenlignet med en monolittisk arkitektur. Ved utviklingen av systemet måtte det tas hensyn til hvordan tjenester kommuniserer med hverandre. Dette gjelder spesielt Onboarding Service, ettersom den orkestrerer registrering av nye selskaper i de fleste tjenestene. Når en operasjon feiler, må derfor transaksjonen annulleres på tvers av tjenestene. Microservice.io beskriver dette som "Saga Patterns", hvor en distribuert transaksjon deles opp i flere lokale transaksjoner. Hver tjeneste utfører sin del av operasjonen, og dersom en feil oppstår, utføres en kompensasjonsoperasjon for å reversere tidligere steg [21].

For å håndtere disse utfordringene er løst koblede tjenester (loose coupling) et sentralt designprinsipp, hvor tjenester kommuniserer via selvstendige API-er fremfor å dele data direkte. Et sentralt prinsipp fra microservice.io er at hver tjeneste har ansvar for egne data og kun eksponerer disse gjennom sitt API [20]. Løsningen anvender dette gjennom et annet mønster kalt "Database per Service", hvor hver

tjeneste eier sin egen database. Dette forhindrer direkte tilgang mellom tjenester og bidrar til tydelig dataseparasjon.

Utfordringer ved integrasjon med ERP-systemer

Uthenting av ERP-data skapte utfordringer både for implementasjonen og for systemet. Selve uthenting av produksjonsdata ble komplisert ettersom det er relasjoner mellom ulike produksjonsobjekter. Dette skapte utfordringer knyttet til konsistens i databasen til ERP Service. Mer om dette i kapittel 7.7.

På system nivå førte det til utfordringer knyttet til skalerbarhet og distribusjon. Siden Monitor ERP ikke er direkte tilgjengelig over internett, ble løsningen å distribuere en egen ERP Agent service hos hvert selskap. Tjenesten må derfor installeres manuelt i samme nettverksmiljø som ERP-systemet for å kunne hente ut data og sende disse videre til det interne systemet. Dette er en tungvint løsning med tanke på skalerbarhet, men det er noe løsningen ikke kommer seg unna. Årsaken er at produksjonsbedrifter ofte drifter ERP-systemene sine på interne servere som er bak VPN og interne nettverk. Dette gjør systemet utilgjengelige direkte over internett og krever autorisert tilgang.

ERP Agent Servicen er derfor plassert internt i systemet med mock-data under utvikling. Gruppen valgte denne tilnærmingen for å prioritere et fungerende system med sikker kommunikasjon før løsningen kobles mot faktiske ERP-systemer.

6.3 Distribusjon og drift

Løsningen kjøres i containere med Docker for å sikre at tjenestene kan driftes isolert på tvers av utviklings- og produksjonsmiljø. I kildekoden er det skilt mellom egne Dockerfiler for utvikling og produksjon. For utviklingsmiljøet benyttes Tilt for å overvåke og administrere tjenestene, samt samle logger fra containerne i ett grensesnitt. Dette gjorde det enklere å følge med på kommunikasjonen mellom tjenestene under utvikling. I tillegg bruker mikrotjenestene Air og Frontend Vite via Bun for hotcode reloading, slik at endringer i koden automatisk bygger og starter tjenestene på nytt.

I utviklingsmiljøet mountes kildekoden inn som volumer i containeren for rask gjenoppbygging av Docker-images. I produksjonsmiljøet benyttes derimot egne bygg- og kjøresteg i Dockerfilene, der tjenestene konfigureres gjennom miljøvariabler fra `.env.production`.

En viktig del av infrastrukturen var oppsettet av ChirpStack for håndtering av LoRaWAN-enheter og sensordata. ChirpStack krever samspill mellom flere tjenester som vist i figur 8. For å forenkle oppsettet, så tok gruppen utgangspunkt i det offisielle Docker-baserte eksempelet fra Chirpstack¹⁵. Dette ga løsningen et grunnlag for infrastruktur og kommunikasjon mellom ChirpStack komponenter.

For distribusjon valgte gruppen å bruke Docker Compose. Den opprinnelige planen var å bruke Kubernetes, men mye tid gikk med til å sette opp tjenestene for håndtering av LoRaWAN-enheter og innhenting av sensordata. Siden dette viste seg å være mer krevende enn forventet, valgte gruppen å prioritere funksjonalitet fremfor mer avansert infrastruktur med Kubernetes. Docker Compose ble derfor valgt som en mer praktisk løsning.

Samtidig er infrastrukturen strukturert med tanke på en mulig overgang til Kubernetes. Under `infra/` er oppsettet delt inn i `development/`, `production/`, og hver tjeneste har egne Dockerfiles per miljø. Måten løsningen skiller utvikling og produksjon, vil legge til rette for senere migrasjon til Kubernetes.

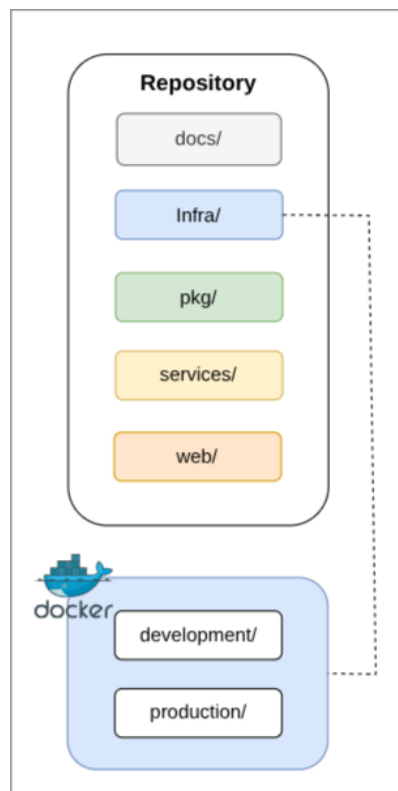


Figure 10: Kildekode struktur

¹⁵<https://github.com/chirpstack/chirpstack-docker>

7 Implementasjon

7.1 Frontend

Frontend omfatter den visuelle presentasjonen av applikasjonen og fungerer samtidig som den primære grensesnittflaten for administrasjon av systemet. Løsningen er utviklet ved bruk av React-rammeverket, supplert med eksterne biblioteker.

7.1.1 Utfordringer

I starten av frontend-utviklingen jobbet to utviklere parallelt uten et etablert sett med felles komponenter. Begge utviklerne skrev store, sidedekkende komponenter, noe som førte til at like UI-elementer ble implementert to ganger med små forskjeller i stil og oppførsel. Da behovet for refaktorering ble synlig, oppstod det videre koordineringsutfordringer: Mens én utvikler refaktorerte eksisterende komponenter, jobbet den andre videre med nye sider uten å kunne bruke komponentene, da de ikke var ferdig refaktorert. Dette skapte en periode med inkonsistens i kodebasen og noe duplisert arbeid som måtte skrives om i etterkant.

Tidlig i prosjektet ble alle komponenter plassert i en stor `components/`-mappe uten særlig videre struktur. Etter hvert som antall filer økte, ble det vanskelig å lokalisere relatert kode og forstå hvilke komponenter som hørte sammen. Litt før midtveis i prosjektet ble det derfor besluttet å refaktorere kodebasen til Feature-Sliced Design (mer om FSD i kapittel 7.1.4).

7.1.2 Wireframe

For å få et perspektiv på hvordan frontend kunne se ut, lagde gruppen Hi-Fi wireframe av enkelte sider. Det ble valgt å gå direkte til Hi-Fi fremfor å starte med Low-Fi, fordi det var allerede en klar idé om hvordan løsningen skulle se ut. Disse ble utviklet med Figma ¹⁶ og fungerte som et grunnlag for frontend-layouten og var en viktig del av visualiseringen. Siden det var to utviklere som jobbet med frontend i starten, var det også nyttig å ha et felles utgangspunkt som begge hadde tilgang til. I starten ble frontend utviklet til å ligne wireframene mest mulig, som gjorde det enklere å vite hva som skulle plasseres hvor. Det ble seinere utviklet en oppdatert wireframe som videre utvikling tok utgangspunkt i, men deretter utviklet frontend seg mer selvstendig. Hver av wireframe-sidene ligger i vedlegg C.

¹⁶<https://www.figma.com/>

7.1.3 Material UI bibliotek

MUI-pakkene som ble benyttet er delt inn i tre hovedbiblioteker: `@mui/material` utgjør kjernen og inneholder grunnleggende UI-komponenter som *Box*, *Typography*, *Button*, *TextField*, *Dialog* og *Card*. `@mui/icons-material` tilbyr et omfattende ikonbibliotek som ble brukt til statusindikatorer, navigasjon og knapper, der funksjonaliteten visualiseres gjennom ikoner. `@mui/x-charts` leverer avanserte diagramkomponenter, blant annet *BarChart* og *LineChart*. Disse ble brukt til fremvisning av rå sensordata, samt på Context-order-siden der all data settes i kontekst av ERP-data.

```
1 import ... from "@mui/material/...";
2 import ... from "@mui/x-charts/...";
3 import ... from "@mui/icons-material/...";
```

Listing 3: MUI imports

En sentral del av MUI-bruken er det egendefinerte temaet som ble implementert for å støtte både lys og mørk visning. Temaet definerer applikasjonens fargepalett, typografi og komponentstiler sentralt, slik at alle MUI-komponenter automatisk arver riktig utseende uavhengig av valgt modus. Tema-preferansen lagres i `localStorage` og gjenopprettes ved neste økt.

For å fremme gjenbrukbarhet og redusere duplisert kode ble det utviklet et eget lag med abstraherte UI-komponenter i `shared/ui`. Disse bygger på MUI-komponenter, men eksponerer et forenklet grensesnitt tilpasset applikasjonens behov. Eksempler inkluderer `AppSnackBar` for systemvarsler, `LoadingIndicator` for lastestatus og `DropDownSelect` (markert gul i figur 11), en `Autocomplete`-basert nedtrekksmeny med søkefunksjonalitet. Denne tilnærmingen sikret visuell og funksjonell konsistens på tvers av alle sider og gjør det enklere å endre utseendet på tvers av hele applikasjonen ved å endre én komponent.

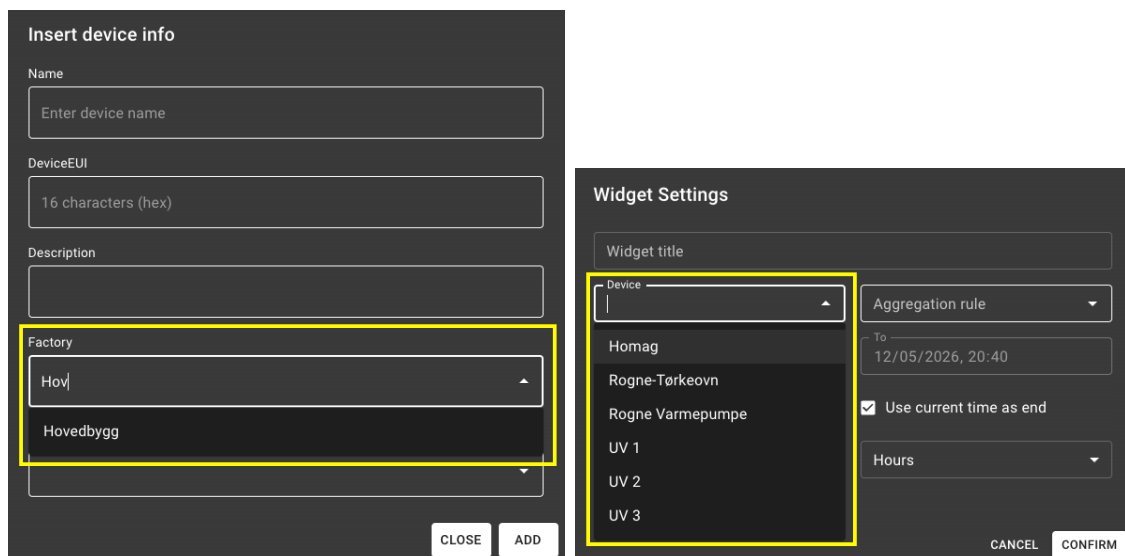


Figure 11: Autocomplete brukt i to forskjellige UI-elementer

7.1.4 Feature-Sliced Design

I frontend ble det benyttet en struktur kjent som FSD¹⁷. FSD er ikke et rammeverk, men en arkitekturmetodikk som definerer organisering av en kodebase. I FSD organiseres kode etter domene og ansvar, ikke etter filtype. Gruppen startet med å samle all logikk i generiske mapper som `components/`, `hooks/` og `utils/`, men dette skalerer dårlig. Når kodebasen vokser, blir det vanskelig å finne ut hvilke filer som hører sammen, og det kan enkelt oppstå sirkulære avhengigheter mellom komponenter.

FSD deler kodebasen inn i lag med et strengt importhierarki. Et lag kan kun importere fra lagen under seg:

```
app -> pages -> features -> entities -> shared
```

¹⁷<https://feature-sliced.design/>

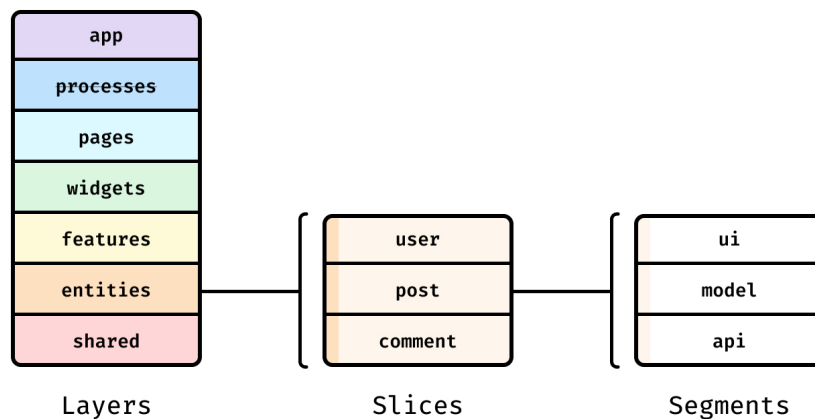


Figure 12: FSD hovedkonsept. Bilde hentet fra FSD-docs [22]

Altså kan en side (`pages`) bruke komponenter fra `widgets`, `features`, `entities` og `shared`. En `feature` kan bruke `entities` og `shared`, men aldri importere fra en side eller widget over seg. Denne regelen eliminerer sirkulære avhengigheter og gjør det enkelt å resonnerer rundt hva en gitt del av kodebasen kan og ikke kan bruke.

I praksis har prosjektet følgende lag:

Lag	Formål
<code>app</code>	Inngangspunkt, ruting, globale stiler og providers
<code>pages</code>	Helsidekomponenter sammensatt av lagene under
<code>widgets</code>	Store, selvstendige UI-blokker som gjenbrukes på tvers av sider
<code>features</code>	Brukerinteraksjoner — skjemaer og handlinger
<code>entities</code>	Domenemodeller med data, logikk og visning
<code>shared</code>	Domeneuavhengige hjelpefunksjoner, UI-primitiver og konfigurasjon

Table 7: FSD-lagene i prosjektet

Segmentkonvensjoner

Hvert domene (`slice` i figur 12) er videre delt inn i fire segmenter med definerte ansvarsområder. Sensorer er et godt eksempel på en fullstendig slice:

```

1 entities/sensor/
2 |-- api/      HTTP-kall mot backend (getSensors, postSensor, deleteSensor...)
3 |-- model/    Tilstandslogikk og TypeScript-typer (useSensors, sensorSchema)
4 |-- lib/      Pure hjelpefunksjoner uten sideeffekter (sortSensors)
5 |-- ui/       React-komponenter for sensordomenet (SensorMainInfo, ...)
6 |-- index.ts  Offentlig API - det eneste andre lag importerer fra

```

Listing 4: Segmentstruktur for `entities/sensor`

`index.ts` fungerer som sensor-slicens offentlig API og eksporterer kun det andre lag trenger. Konsumenter importerer alltid fra slice-roten, aldri fra interne stier:

```
1 // Korrekt - importerer fra slice-roten
2 import { useSensors, SensorMainInfo } from "@entities/sensor";
3
4 // Unngå - eksponerer intern struktur
5 import { useSensors } from "@entities/sensor/model/useSensors";
```

Listing 5: Import fra slice-rot via offentlig API

Features og sidekomposisjon

Brukerhandlinger som å opprette en sensor, legge til en bruker eller konfigurere en regel ligger i `features/-laget`. Dette er typisk skjemaer eller dialoger som kaller API-funksjoner fra `entities`. Sider i `pages/` komponerer deretter disse lagene sammen til en hel side. For eksempel kan siden `Sensors.tsx` bruke `useSensors` fra `entities/sensor` og `AddDevice` fra `features/addDevice` uten selv å inneholde API-logikk.

Shared-laget

`shared/` er det nederste laget i hierarkiet og kan importeres fra alle andre lag. Her ligger kode som ikke tilhører noe spesifikt domene: den generiske API-hjelpfunksjonen, Axios-instansen, MUI-temakonfigurasjon, navigasjonslisten og UI-komponenter som er svært gjenbrukbare, som for eksempel knapper og lasteindikatorer. Skillet mellom `shared` og `entities` er at `shared` ikke kjenner til noen forretningslogikk. En knapp i `shared/ui` vet ingenting om sensorer eller ordrer og kan derfor brukes overalt uten å introdusere avhengigheter.

Feature-Sliced Design-strukturen gjør det enkelt å lokalisere kode, siden alt som tilhører et domene ligger samlet på ett sted. Nye domener legges til ved å opprette en ny slice med de samme fire segmentene, uten at eksisterende kode berøres.

7.1.5 Barrel exports

Etter hvert som kodebasen vokste og det ble utviklet flere funksjoner og objekter som måtte importeres, ble det tydelig at importene krevde en bedre struktur enn å navigere til toppmappen og ned gjennom alle undermappene til man kommer til eksporten. Gruppen valgte derfor å benytte *barrel exports*, hvor eksportene skjer mappevis oppover i strukturen. For å oppnå dette plasseres hver komponent i sin egen mappe, sammen med en *index*-fil som eksporterer alt i mappen, inkludert andre

mapper med egne *index*-filer.

```
1  /*
2  src/components/
3  menuBox/
4  menuBox.tsx
5  index.ts
6  */
7
8  import Menu from "../../components/Menu.tsx"; // Uten barreling
9  import { MenuBox } from "@components/menuBox"; // Med barreling
```

Listing 6: Barrel eksport

Som vist i listing 4 benyttes `@`, som peker til toppmappen *app* i frontend. Dermed importeres alt fra en hovedmappe som maksimalt krever to nivåer ned for å hente nødvendige komponenter. Alle eksporter skal importeres med navn i stedet for å bruke `export default`, slik at hele kodebasen forblir konsistent. Dette fører til langt renere importstier og ekstra type safety.

Axios

*Axios*¹⁸ er et promise-basert HTTP-klientbibliotek for JavaScript som ble benyttet til all kommunikasjon mellom frontend og backend. Hele applikasjonens API-trafikk rutes gjennom én sentral Axios-instans definert i `shared/api/apiClient.tsx`, hvor alle forespørsler sendes via API-gatewayens `/api`-prefiks.

Axios ble valgt fremfor det innebygde `fetch`-API-et på grunnlag av bibliotekets støtte for *interceptors*, som gjør det mulig å definere felles logikk for autentisering og feilhåndtering ett sted i kodebasen. Axios håndterer også JSON-serialisering og deserialisering automatisk, og avviser promises ved HTTP-feilkoder. Dette står i kontrast til `fetch`, som krever manuell parsing og kun feiler ved nettverksfeil. Samlet sett gjør dette det enklere å implementere robust kommunikasjon med backend.

En generisk `apiRequest`-funksjon ble implementert. Den tar en Axios-instans, URL, HTTP-metode og data som parametere, og returnerer et typet `Promise` basert på generikk i TypeScript. Dette gir et enhetlig grensesnitt for alle API-kall i applikasjonen og reduserer mengden boilerplate-kode ved kommunikasjon mot backend.

¹⁸<https://axios-http.com/>

```
1   export const apiRequest = async <T,>(  
2     client: AxiosInstance,  
3     url: string,  
4     method: "GET" | "POST" | "PUT" | "DELETE",  
5     data?: unknown,  
6   ): Promise<T> => {  
7     const response: AxiosResponse<T> = await client({  
8       method,  
9       url,  
10      data,  
11    });  
12  
13    return response.data;  
14  };
```

Listing 7: API handler

7.1.6 Hooks

React tilbyr et sett med innebygde hooks (blant annet `useState` og `useEffect`) som kan kombineres til egne *custom hooks* for å trekke ut og gjenbruke tilstandslogikk fra komponenter. En hook er en vanlig funksjon som følger konvensjonen om å starte med `use`, og som kan kalle andre hooks internt. Uten hooks ville henting av data, lastestatus og feilhåndtering måtte implementeres direkte i hver enkelt komponent. Dette fører raskt til duplisert kode og komponenter med for mange ansvarsområder. Ved å samle denne logikken i en hook kan komponenten begrense seg til å vise data, uten å forholde seg til hvordan de ble hentet. Samme hook kan gjenbrukes av flere komponenter uten at logikken kopieres, og eventuelle endringer i API-strukturen trenger kun å gjøres ett sted.

To typer hooks

Prosjektet bruker to typer hooks: hooks som henter data fra backend, og hooks som håndterer lokal UI-tilstand. `useSnackbar` er et eksempel på sistnevnte - den holder styr på om et varsel skal vises, hvilken melding som skal vises og alvorlighetsgraden. Denne komponenten har kun tilstandslogikk, ingen API-kall eller `useEffect`. Ved å samle dette i en hook kan alle komponenter som trenger varslingsatferd ta i bruk `useSnackbar`-hooken uten å reimplementere logikken hvert sted den skal benyttes.

Datahentende hooks

Den viktigste gruppen hooks er de som henter data fra backend. Disse følger alle samme struktur: de returnerer dataene, et `isLoading`-flagg, et `error`-objekt og en `refetch`-funksjon. Denne konsistensen gjør det forutsigbart for alle på teamet å ta

i bruk en hook da returverdiene er de samme uavhengig av domene. I en applikasjon med mange domener som sensorer, gateways, fabrikker, produksjonsordre og brukere ville implementering av datahentingslogikk per komponent raskt ført til en uoversiktlig kodebase og logikk. Med et felles mønster kan en utvikler som kjenner én hook umiddelbart forstå alle andre.

```
1  const refetch = useCallback(() => {
2    setIsLoading(true);
3    setRefetchIndex((i) => i + 1);
4  }, []);
5
6  useEffect(() => {
7    let cancelled = false;
8    getCompanies()
9      .then((data) => {
10       if (!cancelled) {
11         setCompanies(data);
12         setIsLoading(false);
13       }
14     })
15     .catch((err: unknown) => {
16       if (!cancelled) {
17         setError(err instanceof Error ? err : new Error(String(err)));
18         setIsLoading(false);
19       }
20     });
21   return () => { cancelled = true; };
22 }, [refetchIndex]);
23
24 return { companies, isLoading, error, refetch };
```

Listing 8: Datahentende hook med lastestatus, feilhåndtering og refetch

`refetchIndex` er en teller som inkrementeres hver gang `refetch()` kalles. Siden `useEffect` avhenger av denne verdien, vil effekten kjøre på nytt ved hver oppdatering, noe som tvinger frem en ny henting av data uten at komponenten trenger å vite om det. `cancelled`-flagget løser synkronisering mellom flere fetch-kall som kan returnere i feil rekkefølge og dermed overskrive tilstanden med utdaterte data [23]. Dersom komponenten avmonteres mens et kall er i gang, vil svaret ignoreres og tilstanden ikke oppdateres på en komponent som ikke lenger eksisterer.

Avledede tilstander for presis lastestatus

For mer sammensatte hooks hvor en parameter kan endre seg, er en enkel `isLoading`-bool ikke alltid presis nok. En hook kan da i stedet *avlede* lastestatusen fra selve

datatilstanden fremfor å holde den som en separat tilstand:

```
1  const isLoading =
2      orderId !== null &&
3      error === null &&
4      orderContext?.order.id !== orderId;
```

Listing 9: Avledet lastestatus i useOrderContext

Her er hooken i lastestatus så lenge vi forventer data (`orderId` er satt), det ikke har oppstått en feil og dataen vi har ikke tilhører den forespurte ordren ennå. Dette hindrer at UI-et kort viser gammel data som "ferdig lastet" når en ny ordre velges.

7.1.7 Layout

Applikasjonens layout er basert på en eksisterende løsning utviklet i et tidligere prosjekt av Innoveria. Strukturen består av to sentrale komponenter: en *vertikal navigasjonsmeny* og et *hovedområde*. Denne inndelingen er valgt for å legge til rette for effektiv navigasjon, uavhengig av hvor i applikasjonen brukeren befinner seg. Navigasjonsmenyen er kontinuerlig synlig, noe som understøtter prinsippet om høy tilgjengelighet og enkel tilgang til applikasjonens funksjonalitet. Hvert menyelement er markert med et relevant ikon for å forenkle den visuelle navigasjonen.

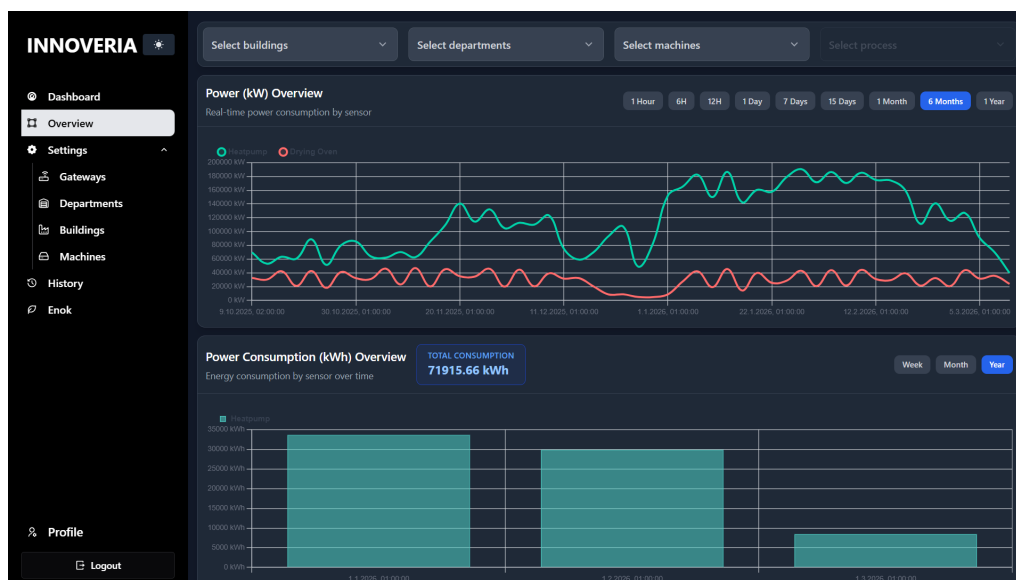


Figure 13: Innoveria sitt tidligere prosjekt



Figure 14: Nåværende layout

Navigasjonsmeny

Navigasjonsmenyen presenterer en oversikt over alle tilgjengelige sider i applikasjonen og muliggjør navigasjon mellom disse ved hjelp av en rutingsmekanisme. Når en bruker velger et element i menyen, dirigeres vedkommende til den tilhørende siden. For å håndtere et større antall navigasjonsvalg er menyen strukturert med undermenyer i form av kollapsbare rulleelementer. Dette bidrar til bedre plassutnyttelse og en mer oversiktlig presentasjon av tilgjengelige funksjoner. Hvert av menyelementene er markert med et relevant ikon for å forenkle den visuelle navigasjonen.

Hovedområde

Hovedområdet fungerer som applikasjonens primære visningsflate, hvor innholdet på de ulike sidene presenteres. Dette området oppdateres dynamisk basert på brukerens trykk på navigasjonsmenyen, stien på toppen eller videregående sider.

7.2 API-struktur

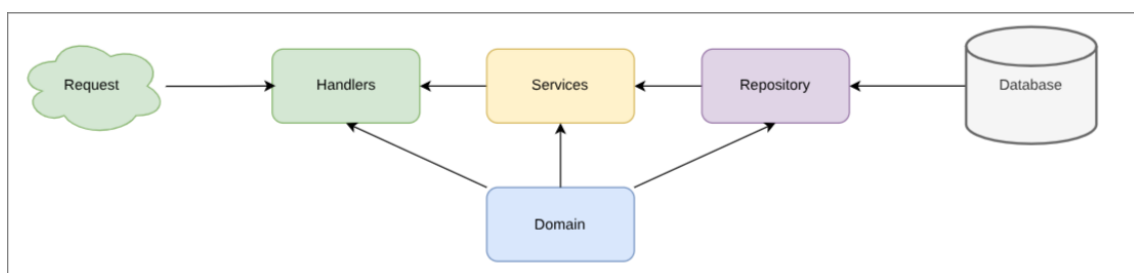


Figure 15: Backend-implementasjon

Ettersom backend består av flere tjenester som håndterer ulike domener, valgte gruppen å legge stor vekt på prosessen før en tjeneste ble implementert. Alle tjenester er dermed systematisk bygget og følger en standard lagdelt struktur med tydelig separasjon mellom API-lag, forretningslogikk og datalagring.

Ved utformingen av en tjeneste startet prosessen med diskusjoner rundt hva tjenesten skulle håndtere, og hvordan domenet skulle se ut. Deretter ble domenemodeller og nødvendige funksjoner etablert. Dette innebærer å etablere forretningslogikk ved bruk av Go sine interfaces. Det reduserer koblingen mellom komponenter og gjør det enklere å lage enhetstester for hver komponent. Avslutningsvis blir nødvendige databaseoperasjoner, selve forretningslogikken og handler-laget implementert.

Hver tjeneste er hovedsakelig strukturert rundt fire sentrale komponenter:

- **Handler-laget** håndterer HTTP-forespørsler og fungerer som inngangspunktet til tjenesten. Laget er ansvarlig for validering av input, autentisering og mapping mellom Data Transfer Object (DTO)-modeller og interne modeller. DTO-modeller er måten tjenesten formaterer domenestrukturen over HTTP på.
- **Domain** inneholder tjenestens regler for hvordan forretningslogikk og databaseoperasjoner skal se ut, samt hvordan domenet skal være strukturert.
- **Service-laget** er tjenestens implementasjon av forretningslogikk. Her implementeres operasjoner knyttet til domenet.
- **Repository-laget** håndterer kommunikasjon med databasen.

En unik egenskap ved Go er måten språket håndterer interfaces på. I Go trenger man ikke å skrive at en struktur implementerer et interface. Hvis typen har de riktige metodene, implementerer den interfacet automatisk. Designet bak språket gjør at man får en ryddig kildekode, og med denne idiomatikken kunne gruppen raskt forstå hvordan en tjeneste fungerte ved å undersøke hvilke operasjoner domenet eksponerte. Kodeutsnitt 10 viser en slik implementasjon. Videre vil en CompanyService-struktur bare trenge å følge kontrakten for å implementere interface-kontrakten.

```

1 // Company is the domain model for company
2 type Company struct {
3     ID          string
4     Name        string
5     Address     string
6     CreatedAt   time.Time
7     UpdatedAt   time.Time
8 }
9
10 // CompanyRepo defines the company repository needed by the auth domain.
11 type CompanyRepo interface {
12     Create(ctx context.Context, company Company) (Company, error)
13     FindAll(ctx context.Context) ([]Company, error)
14     FindByID(ctx context.Context, companyID string) (Company, error)
15     DeleteByID(ctx context.Context, companyID string) error
16 }
17
18 // CompanyService defines company use-cases exposed by the service layer.
19 type CompanyService interface {
20     RegisterCompany(ctx context.Context, payload Company) (Company, error)
21     GetOneCompany(ctx context.Context, companyID string) (Company, error)
22     GetAllCompanies(ctx context.Context) ([]Company, error)
23     DeleteCompany(ctx context.Context, companyID string) error
24 }

```

Listing 10: domene etablering for selskaper

7.3 API-gateway

Klienten kommuniserer ikke direkte med de interne tjenestene, men sender forespørsler gjennom API-gateway. Den fungerer som en omvendt proxy, implementert ved hjelp av Go sitt innebygde bibliotek. Gatewayen omskriver innkommende forespørsler før de sendes videre til interne tjenester. Før videresending blir forespørselen validert ved å kontrollere om den inneholder et gyldig JWT-token i headeren. Dersom tokenet er ugyldig eller mangler, stoppes forespørselen, og gatewayen returnerer status *uautorisert* til klienten. Videre beskrivelse av token-flyten er beskrevet i 7.4.

Hele poenget med tjenesten er å kjøre en rekke mellomvareledd for å kontrollere om klienten er autorisert og ikke misbruker systemet. Etter autentisering vil den fylle forespørselen med nødvendig informasjon til videresending. En typisk flyt vises i figur 16.

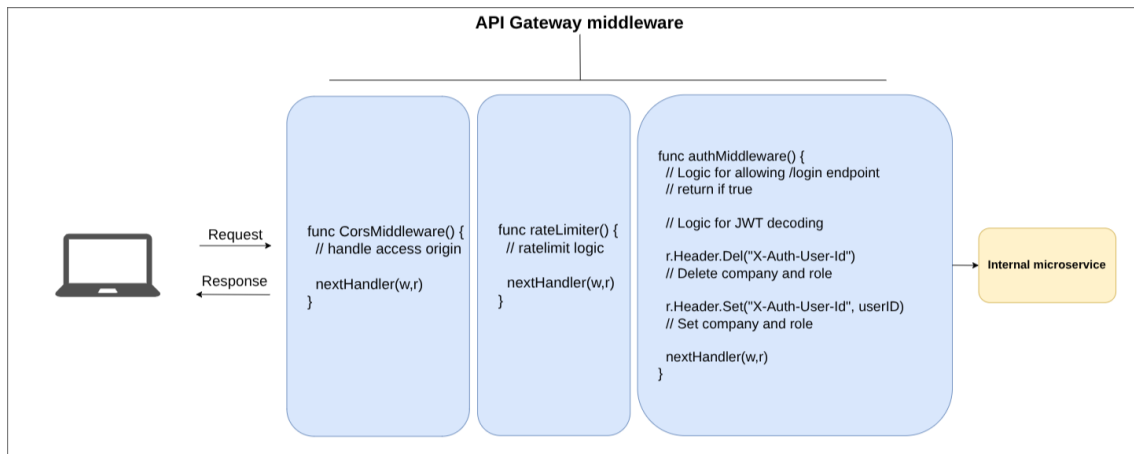


Figure 16: API Gateway middleware pipeline

Den nyttigste funksjonen med gateway skjer under validering av forespørsler. Da henter den ut brukerens-ID, rolle og selskaps ID fra JWT-tokenet og bruker disse i de interne HTTP-forespørslene. Dette gjør at hver tjeneste kan utføre databaseoperasjoner basert på denne informasjonen. Man kunne implementert dette per tjeneste, men da måtte hver tjeneste håndtert validering. Med en API-gateway får løsningen en sentralisert plattform for å beskytte systemet.

Multi-tenancy og Rollebasert tilgang

Løsningen ble implementert med multi tenancy for å sikre at data tilhørende bedrifter ikke vises til andre bedrifter. Det blir håndtert av API Gateway gjennom HTTP headers, som de interne tjenestene henter. Innkommende forespørsler fra API Gateway bærer tre headere: `X-Auth-Company-Id`, `X-Auth-User-Id` og `X-Auth-Role`. Disse hentes via funksjonen i kodeutsnitt 11, og brukes i databaseoperasjoner for å sikre at hver tjeneste kan behandle data innenfor riktig bedrift.

```

1 func FromRequest(r *http.Request) (Auth, error) {
2     userID := r.Header.Get(headerUserID)
3     // return unauthorized
4     companyID := r.Header.Get(headerCompanyID)
5     // return unauthorized
6     role := r.Header.Get(headerRole)
7     // return unauthorized
8
9     return Auth{
10         UserID:    userID,
11         CompanyID: companyID,
12         Role:       roles.RoleType(role),
13     }, nil
14 }

```

Listing 11: Uthenting av X-Auth-* headere

Rollen til brukeren sendes også videre fra API Gateway, slik at de interne tjenestene kan håndheve spesifikke autorisasjonsregler ved behov. Løsningen skiller mellom tre roller: **Platform admin**, som representerer ansatte hos Innoveria med tilgang på tvers av bedrifter, **superbruker**, som fungerer som administrator for én bestemt bedrift, og **bedriftsansatt**, som er en standard bruker med mer begrenset tilgang. Gruppen planla opprinnelig å implementere en egen rolletabell for mer fleksibel tilgangskontroll. På grunn av tidsbegrensninger ble det valgt en enklere løsning, der rollebasert tilgang kun sjekker **Platform admin**, som håndteres gjennom manuelle sjekker i de relevante tjenestene.

```

1 func (a Auth) IsAdmin() bool {
2     return a.Role == roles.PlatformAdmin
3 }

```

Listing 12: Rollebasert tilgang

Ratelimit

For å beskytte systemet mot overbelastning og misbruk er det også implementert ratelimiting. Gateway begrenser antall forespørsler en bruker kan sende innenfor et gitt tidsvindu. Dersom grensen er nådd, returneres en feilmelding om at for mange forespørsler er nådd. En grunnleggende implementasjon av ratelimiting ville gitt sikkerhetsproblemer, spesielt knyttet til innlogging. Løsningen håndterer dette med unike regler for innloggingsendepunkter.

```
1 func bruteForcePolicy(r *http.Request) (rateLimitPolicy, bool) {
2     path := r.URL.Path
3
4     switch {
5     case r.Method == http.MethodPost && path == AUTHENTICATION_ROUTE+"/login":
6         return rateLimitPolicy{Limit: 5, Burst: 5, Window: time.Minute}, true
7     case r.Method == http.MethodPost && path == AUTHENTICATION_ROUTE+"/refresh":
8         return rateLimitPolicy{Limit: 20, Burst: 10, Window: time.Minute}, true
9     default:
10        return rateLimitPolicy{}, false
11    }
12 }
```

Listing 13: bruteforce-regel for ratelimiting

Figur 13 viser den sentrale funksjonen for ratelimiting. Endepunktet `/login` er begrenset til fem forespørsler per minutt med en "burst" verdi på fem minutter. Det betyr at en klient maksimalt kan sende 5 innloggingforsøk innenfor ett minutt før neste forespørsel blokkeres midlertidig. Denne begrensningen er viktig for automatiserte passordangrep mot systemet.

7.4 JWT-basert autentisering

I systemet er autentisering delt mellom API gateway og Auth Service. API Gateway har ansvar for å validere innkommende forespørsler før de rutes videre til det interne systemet. Dette innebærer at API Gateway ikke selv autentiserer brukeren med e-post eller passord. Håndtering av innlogging og utlevering av JWT-tokens foregår i Auth Service. Denne ansvarsfordelingen gjør at sensitiv innloggingslogikk holdes samlet i Auth Service, samtidig som API Gateway kan fungere som sikkerhetslaget foran de interne tjenestene. Årsaken til denne løsningen er å unngå forretningslogikk i API Gateway. Dette gir en tynnere tjeneste som kun fokuserer på å være mellomledd mellom klient og interne tjenester.

Gruppen valgte å ikke basere autentisering på tredjepartstjenester som OAuth. Årsaken er at Innoveria jobber med bedrifter, og av den grunn blir kun nødvendig brukerdata relatert til bedriften lagret i databasen. Dette gjorde gruppen ved å implementere en egen autentiseringstjeneste, dermed blir løsningen ikke avhengig av eksterne leverandører, noe som sikrer at innloggingslogikk og brukerdata håndteres innenfor systemet.

```
1  claims := jwt.MapClaims{}
2  token, err := jwt.ParseWithClaims(
3      tokenStr, claims, func(t *jwt.Token
4      ) (any, error) {
5      if t.Method != jwt.SigningMethodHS256 {
6          return nil, errors.New("invalid signing method")
7      }
8      return []byte(cfg.JWTSecret), nil
9  }, jwt.WithIssuer(cfg.JWTIssuer))
10 if err != nil || !token.Valid {
11     // return status unauthorized,
12 }
13
14 userID, _ := claims["sub"].(string)
15 companyID, _ := claims["company_id"].(string)
16 role, _ := claims["role"].(string)
```

Listing 14: dekoding av JWT-token i API Gateway

For løsningen valgte gruppen å bruke JSON Web Tokens for autentisering. JWT har et kompakt format og er godt egnet i mikrotjenestearkitektur der flere tjenester må kunne verifisere brukerens identitet uten å gjøre databaseoppslag for hver forespørsel. JWT er definert i RFC 7519 som en måte å representere claims mellom to parter som et JSON-objekt, der innholdet kan verifiseres ved hjelp av en digital signatur [24]. Kodeutsnitt 14 viser måten API Gateway dekker en JWT-token utstedt av Auth Service.

Autentisering i Auth Service er implementert med kortlevde access tokens og langvarige refresh tokens. Formålet er å gi klienten en effektiv måte å autentisere API-kall. For å håndtere langvarige innloggingsøkter vil refresh token oppdatere access token når det er utløpt.

7.4.1 Innlogging

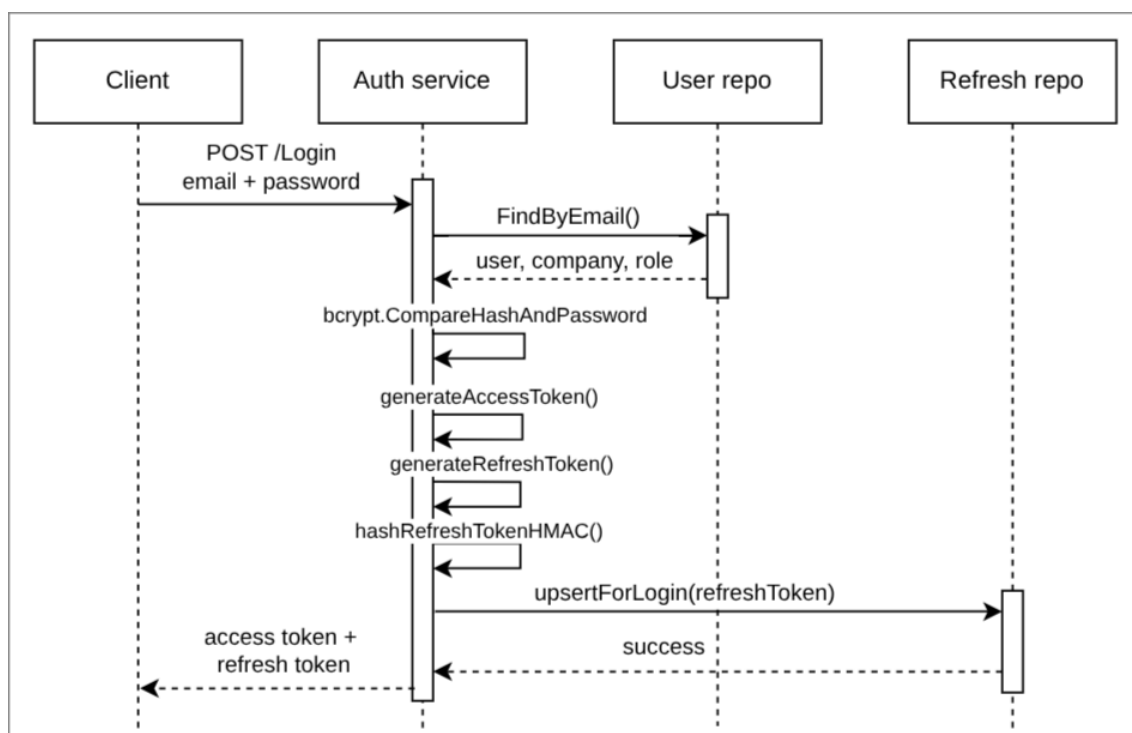


Figure 17: sekvensdiagram for innlogging

Figur 17 viser dataflyten ved innlogging fra endepunktet `/login`. Klienten starter prosessen med å sende en POST-forespørsel til Auth Service med e-post og passord. Auth Service henter deretter brukeren fra databasen. Dersom brukeren finnes, sammenlignes passordet med en lagret passordhash ved hjelp av rammeverket `bcrypt`. I denne sammenhengen sjekker den passordet med det krypterte passordet fra databasen. Auth0 anbefaler `bcrypt`-rammeverket for kryptering av lagrede passord i databasen. Ettersom det er designet for å være tregt og benytter salting, passer det godt i løsningen ved å gi ekstra beskyttelse mot brute-force-angrep [25].

Dersom passordet er gyldig, genererer Auth Service først en kortlevd access token. Det inneholder bruker-id, rolle, selskaps-id og utløpstidspunkt, som blir brukt i API Gateway. I tillegg genererer Auth Service en refresh token. I motsetning til access token er refresh token ikke en JWT, men en tilfeldig generert byte-verdi. Før det blir lagret i databasen blir det enkodet med HMAC-SHA256 og en egen JWT-secret. Systemet lagrer ikke refresh token i klartekst, men som en hashverdi som senere brukes for oppslag. Dette reduserer risiko dersom en database lekkasje skulle oppstå. Avslutningsvis returnerer Auth Service access token og det rå refresh tokenet til klienten.

Det er spesielt viktig at refresh token ikke er direkte tilgjengelig for klienten. Derfor

lagres refresh token som en HttpOnly-cookie. En HttpOnly-cookie kan ikke leses eller manipuleres gjennom JavaScript i nettleseren, noe som reduserer risikoen for at refresh token eksponeres ved XSS-angrep.

7.4.2 Tokenrotasjon

Når access token utløper bruker klienten refresh token på nytt til å be om en ny access token på endepunktet /refresh. Auth Service hasher da det mottatte refresh tokenet med samme HMAC-enkoding og søker etter hashverdien i Refresh repo. Dersom tokenet finnes og ikke er utløpt hentes brukeren fra User repository. Auth Service genererer deretter et nytt access token og et nytt refresh token.

Det nye refresh tokenet erstatter det gamle i databasen. Denne prosessen kalles tokenrotasjon. Hensikten med tokenrotasjon er å redusere levetiden til hvert enkelt refresh token. Dersom et refresh token skulle bli lekket vil det gamle tokenet bli ugyldig etter at et nytt token er utstedt. På den måten begrenses muligheten for gjenbruk av gamle refresh tokens.

7.5 Enhetsadministrasjon og LoRaWAN-integrasjon

Administrasjon av enheter som sensorer og gateways er samlet i en mikrotjeneste kalt *Device-Service*. Dens oppgave er å holde gateways, sensorer og tenants synkronisert mellom vår egen PostgreSQL-database og ChirpStack [19], og er samtidig kilde for den metadataen som ikke hører hjemme i ChirpStack. Device Service er i tillegg ansvarlig for vokabularet som beskriver hvilke måletyper systemet kjenner, samt mappingen mellom de rå nøklene ChirpStack sender og disse definerte måletypene.

ChirpStack og databasen er to separate systemer uten distribuerte transaksjoner. Hver gang en enhet opprettes, oppdateres eller slettes må applikasjonen sørge for at begge systemene er synkronisert. Dette har ført til at gruppen måtte håndtere konsistens gjennom saga-mønsteret med kompenserende transaksjoner i applikasjonen. Mer om datakonsistens er beskrevet senere i 7.5.5. De følgende delene beskriver de viktigste delene av implementasjonen med særlig fokus på sensor- og gateway-administrasjon og payload schema-systemet, da disse utgjør de mest sentrale delene av tjenesten.

7.5.1 ChirpStack REST-klient

All kommunikasjon mellom Device-Service og ChirpStack går gjennom en dedikert REST-klient. Autentisering mot klienten skjer ved at et Bearer-token leses fra disk ved hvert kall, fremfor å caches ved oppstart. Dette gjør at hemmeligheter kan roteres i containermiljøet uten omstart. En ulempe ved å lese fra disk ved hvert kall er at det medfører en liten ytelseskostnad som kan akkumuleres i et høytrafikkscenario. I tillegg vil kallet feile dersom filen er utilgjengelig i et kort øyeblikk under rotasjon.

Et alternativ kunne vært å cache tokenet i minnet, samtidig som systemet lytter etter filsystemendringer og oppdaterer cachen når filen endres. Dette ville gitt lavere ytelseskostnad samtidig som det støtter rotasjon av hemmeligheter uten omstart.

Gruppen implementerte en egen feilhåndteringsmekanisme som omformer ChirpStacks strukturerte JSON-feil til en egen ChirpStackError-type, slik at applikasjonen kan reagere ulikt på eksempelvis en manglende ressurs kontra en autorisasjonsfeil, med fallback til HTTP-statuskode og rå feilmelding dersom svaret ikke er strukturert JSON.

```
1 func handleChirpStackError(err error) error {
2     // ...
3     if json.Unmarshal(httpErr.Body, &apiErr) == nil {
4         return &ChirpStackError{
5             Code: apiErr.Code, Message: apiErr.Message
6         }
7     }
8     return fmt.Errorf("chirpstack error (%d): %s",
9         httpErr.StatusCode, httpErr.Body)
10 }
```

Listing 15: Parsing av feil fra ChirpStack

Klienten dekker nødvendige CRUD-operasjoner for tenants, applikasjoner, gateways, sensorer og sensorprofiler — listing, oppslag, opprettelse, oppdatering og sletting — der hver metode er implementert som et tynt HTTP-kall.

7.5.2 Administrasjon av sensorer og gateways

Dobbel lagring

Sensorer og gateways lagres to steder i applikasjonen vår: i vår egen PostgreSQL-database og i ChirpStack. Alle LoRaWAN-enheter har en 64-bits hardwarenøkkel

kalt EUI, brent inn av produsenten, som brukes som koblingspunkt mellom systemene. Alt forretningsmessig som ChirpStack ikke kjenner til — fabrikk, produksjonsressurser, navn og beskrivelse — lagres i vår database, mens ChirpStack eier alt på radio- og nettverkssiden (nøkler, profiler, status). Uten dette skillet ville det ikke vært mulig å besvare spørsmål som "Hvilke sensorer hører til hvilken fabrikk?".

Operasjoner mot ChirpStack

Operasjoner på sensorer og gateways krever kommunikasjon med ChirpStack i alle ledd. Ved opprettelse av en sensor gjøres to separate kall til ChirpStack — først opprettes sensoren, deretter settes Over-The-Air Activation-nøkkelen. Gateways er enklere og krever kun ett kall. Ved oppdatering hoppes ChirpStack-kallet over dersom kun database-felt er endret, noe som reduserer unødvendig ekstern kommunikasjon og gir lavere latens for de vanligste oppdateringene.

```
1 // Only call ChirpStack if a ChirpStack-stored field changed.
2 // DB-only fields (factory_id, factory_area_id, production_resource)
3 // skip ChirpStack entirely.
4 chirpstackChanged := sensor.Name != oldSensor.Name ||
5     ptrutil.Deref(sensor.Description) !=
6     ptrutil.Deref(oldSensor.Description) ||
7     sensor.ChirpstackProfileID != oldSensor.ChirpstackProfileID
8
9 if chirpstackChanged {
10     // ...
11     if err := s.cc.UpdateSensor(ctx, newReq); err != nil { ... }
12     if err := s.sensorRepo.Update(ctx, companyID, sensorID, sensor);
13         err != nil {
14         // Compensate: revert ChirpStack to old values.
15         if compErr := s.cc.UpdateSensor(ctx, oldReq); compErr != nil {
16             slog.Error("saga compensation failed",
17                 "id", sensorID, "error", compErr)
18         }
19         return fmt.Errorf("update sensor: update in database: %w", err)
20     }
21 } else {
22     // No ChirpStack fields changed - update DB only.
23     if err := s.sensorRepo.Update(ctx, companyID, sensorID, sensor);
24         err != nil { ... }
25 }
```

Listing 16: ChirpStack hoppes over ved oppdatering av database-felt

Berikelse på lesetid

Når en bruker leser sensorer eller gateways, slår applikasjonen først opp i databasen

og beriker deretter hver enhet med live-status fra ChirpStack, som viser når enheten sist ble sett og hvilken status den har. Statusen lagres ikke lokalt da den fort ville blitt utdatert. I tillegg gjør denne tilnærmingen at applikasjonen kan levere meningsfull data selv om ChirpStack er utilgjengelig — brukeren kan fortsatt bruke systemet, men uten live-status. Dette mønsteret kalles gjerne *graceful degradation* i distribuerte systemer [26].

7.5.3 Sensorprofiler

For at en LoRaWAN-sensor skal kunne kommunisere med ChirpStack trenger den en sensorprofil. En sensorprofil er en mal for hvordan en bestemt sensormodell oppfører seg på LoRaWAN-nettverket — den inneholder bestemmelser om frekvensbånd, LoRaWAN-versjon og payload-decoder, og forteller ChirpStack hvordan meldinger fra modellen skal dekodes og rutes.

ChirpStack ble satt opp med et bibliotek av globale profiler hentet fra et community-drevet repository som dekker mange vanlige sensormodeller [27]. Større repoer finnes, men opererer under lisenser som ikke tillater fri gjenbruk. De globale profilene er tilgjengelige som maler for alle brukere.

ChirpStack krever at en enhet knyttes til en profil eid av samme tenant — globale profiler kan ikke kobles direkte til enheter. Gruppen implementerte derfor en mekanisme som ved behov oppretter en tenant-eid kopi av den globale profilen før enheten registreres: systemet sjekker først om tenanten allerede har en kopi, og oppretter én fra den globale malen dersom den ikke finnes.

```

1  profile, err := s.cc.GetSensorProfile(ctx, profileID)
2  // ...
3  // Profile is already owned by this tenant - nothing to do.
4  if profile.TenantID == tenantID {
5      return profile.ID, nil
6  }
7  // Profile is global - check if a tenant-level copy exists by name.
8  existing, err := s.cc.GetTenantSensorProfiles(ctx, tenantID, 1000)
9  // ...
10 for _, p := range existing.Result {
11     if p.Name == profile.Name {
12         return p.ID, nil
13     }
14 }
15 // No tenant-level copy found - create one from the global profile.
16 return s.cc.CreateTenantSensorProfile(ctx, /* ... */)

```

Listing 17: Tenant-kopi av global sensorprofil opprettes ved behov

På sensoren lagres både ID-en til tenant-kopien (brukt mot ChirpStack) og ID-en til den opprinnelige globale profilen. Den globale ID-en gjør det mulig å gruppere sensorer på tvers av tenants i admin-grensesnittet, siden ulike tenants ellers ville hatt ulike kopier av samme profil uten noen felles referanse.

7.5.4 Måletypevokabular og merking av rådata

Problemet og vokabularet

ChirpStack leverer sensordata som JSON-payloads der nøklene varierer per sensor-modell. En strømsensor fra én produsent sender ikke nødvendigvis data med samme nøkler som en tilsvarende sensor fra en annen produsent. For at resten av systemet skal kunne tolke og fremstille målingene på en meningsfull måte, implementerte gruppen et måletypevokabular: et sett med målingstyper som fungerer som en kontrakt på tvers av tjenestene, og som administreres av systemadministrator. Hver målingstype har en unik nøkkel, et visningsnavn, en beskrivelse og en standardenhet. Tjenester som konsumerer sensordata forholder seg til dette vokabularet.

To nivåer av konfigurasjon

I løpet av implementeringen forstod gruppen at ikke alle sensorer kan behandles likt når det gjelder mapping mellom vokabularet og nøklene i målingene. De fleste sensormodeller sender alltid den samme datastrukturen. Kjøper man eksempelvis

hundre strømsensorer av samme modell, vil alle sende identiske payload-nøkler. For disse er det tilstrekkelig å konfigurere en felles mapping på profilnivå som gjelder for alle sensorer av den modellen.

Andre sensorer er derimot konfigurerbare og kan sende ulik data avhengig av installasjonen. Et eksempel fra prosjektet er Milesights UC300 – en Modbus-til-LoRaWAN-bro som leser data fra andre sensorer og videresender dem over LoRaWAN-nettverket. Hvilke nøkler den sender og hva de betyr, avhenger av hvilke sensorer den er koblet til i det aktuelle området, og hvilke kanaler den er konfigurert til å sende. Én UC300 kan lese nitrogendata, en annen CO₂-data. For disse sensorene er mapping på profilnivå utilstrekkelig, og mappingen må derfor konfigureres per sensor.

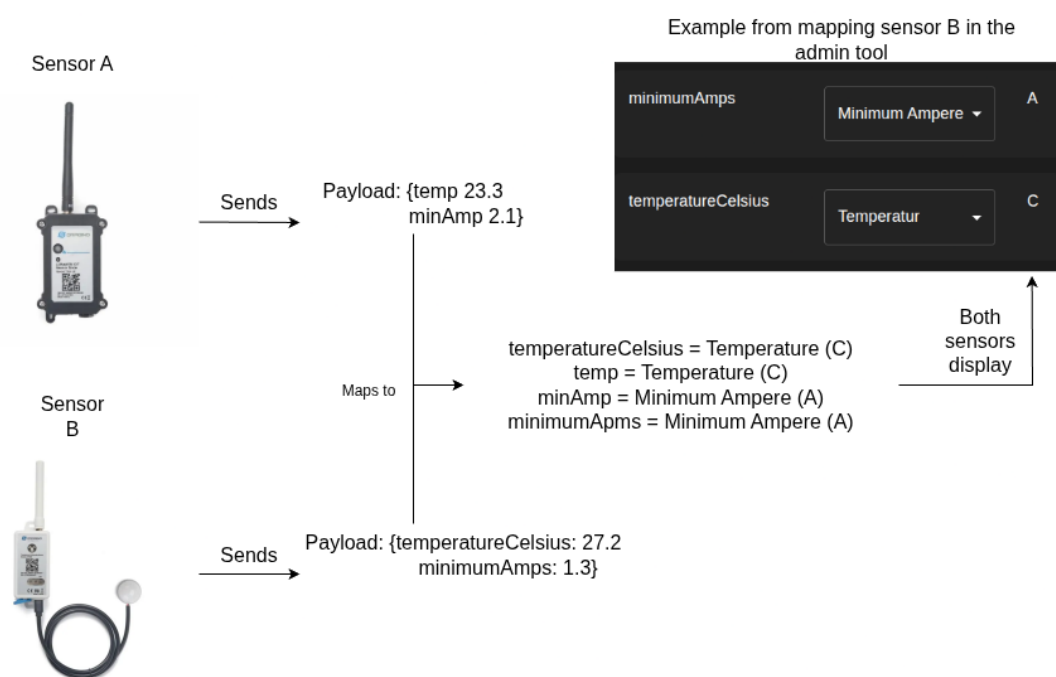


Figure 18: Payload Schema Mapping

Konfigurering og mapping

For begge tilfellene følger konfigureringen samme prinsipp: En administrator knytter rå payload-nøkler til målingstyper fra vokabularet. For å vite hvilke nøkler en sensormodell faktisk sender, implementerte gruppen et eget endepunkt som returnerer EUI-en til en sensor av en gitt profil. Administratoren kan bruke dette til å slå opp nøklene så lenge sensoren allerede har sendt data til systemet, uten at systemet eksponerer de faktiske måleverdiene; kun nøklene vises. En strømsensor vil da eksempelvis kunne returnere `{amp, battery}`, som kobles mot vokabularets verdier: ampere og batteri. Mappingen lagres enten på profilnivå, slik at den gjelder for alle

sensorer av samme modell, eller på sensornivå for konfigurerbare sensorer.

Fallback og oppslag

Når en annen tjeneste spør om hvilke målingstyper en sensor er konfigurert for, sjekker systemet sensornivået først. Hvis ingen mapping er definert der, faller det tilbake til profilmnivå. Dersom begge konfigurasjonene er tomme, betyr dette at sensoren ennå ikke er konfigurert og dermed ikke har noen meningsfull kobling mot vokabularet.

```
1 func (s *SensorMetricServiceImpl) GetEffectiveMetrics(  
2     ctx context.Context, deviceEUI string) ([]domain.SensorMetric, error) {  
3     sensor, err := s.sensorRepo.FindByEUI(ctx, deviceEUI)  
4     // ...  
5  
6     // Check per-sensor metrics first (configurable sensors).  
7     metrics, err := s.sensorMetricRepo.FindBySensorID(ctx, sensor.Id)  
8     // ...  
9     if len(metrics) > 0 {  
10         return metrics, nil  
11     }  
12  
13     // Fall back to profile-level payload schema (fixed-schema sensors).  
14     schemas, err := s.payloadSchemaRepo.FindByProfileID(  
15         ctx, sensor.ChirpStackProfileID)  
16     out := make([]domain.SensorMetric, len(schemas))  
17     // ...  
18     return out, nil  
19 }
```

Listing 18: Fallback fra sensornivå til profilmnivå ved oppslag av målingstyper

7.5.5 Datakonsistens

ChirpStack og PostgreSQL er to separate systemer uten støtte for distribuerte transaksjoner. Enhver operasjon som å opprette, redigere eller slette en sensor eller gateway krever at begge systemene blir berørt, og applikasjonen må selv sørge for at de forblir i samsvar med hverandre selv om et av systemene feiler underveis.

Gruppen valgte å håndtere dette ved å implementere saga-mønsteret med kompensere transaksjoner [21]. ChirpStack kalles alltid først — hvis dette kallet feiler er databasen fortsatt ren og ingenting trenger å rulles tilbake. Feiler en påfølgende operasjon gjøres den motsatte operasjonen i ChirpStack for å reversere endringen. Opprettelse av en sensor er det mest komplekse tilfellet, da det krever to separate kall til ChirpStack: først opprettes sensoren, deretter settes OTAA-nøkkelen.

```

1 sensorReq := mappers.MapChirpStackSensorRequest(
2     payload, cfg.ChirpStackApplicationID)
3 // ...
4 // Set AppKey in ChirpStack
5 sensorKeyReq := mappers.MapChirpStackSensorKeyRequest(payload)
6 if err := s.cc.SetSensorKey(ctx, sensorKeyReq); err != nil {
7     // Compensate:
8     if compErr := s.cc.DeleteSensor(ctx, payload.DeviceEUI);
9         compErr != nil {
10         slog.Error(...)
11     }
12     return fmt.Errorf("create sensor: set app key: %w", err)
13 }
14 // If successfully stored in ChirpStack, try to store in database.
15 sensor, err := s.sensorRepo.Create(ctx, payload)
16 if err != nil {
17     // Compensate:
18     if compErr := s.cc.DeleteSensor(ctx, payload.DeviceEUI);
19         compErr != nil {
20         slog.Error(...)
21     }
22     return fmt.Errorf("create sensor: add to database: %w", err)
23 }

```

Listing 19: Saga-mønster med kompenserende transaksjoner ved opprettelse av sensor

Kompensering med dette mønsteret er best-effort. Hvis selve kompenseringen feiler — for eksempel fordi ChirpStack er utilgjengelig — logger applikasjonen feilen, men systemene kan ha havnet i en inkonsistent tilstand. Dette er en kjent begrensning ved saga-mønsteret. Et alternativ som ble vurdert er en hendelsesdrevet arkitektur, der tjenestene publiserer hendelser som de andre tjenestene reagerer på asynkront [28]. Dette ville gitt bedre frakobling, men introduserer en vesentlig mer kompleks programmeringsmodell. Gruppen valgte derfor saga-mønsteret for å prioritere fremdrift på kjernefunksjonalitet.

7.6 Innhenting av sensordata

For å håndtere innhenting av sensordata implementerte gruppen en egen mikroservice, Collection-Service. Tjenesten abonnerer på sensordata fra ChirpStack, lagrer disse i databasen og eksponerer dem for andre deler av applikasjonen.

Sensordata over MQTT

Sensordata mottas fra ChirpStack over MQTT. ChirpStack publiserer sensormeldingene til en MQTT-broker etter at dataene er dekodet av sensorprofilens dekode. Gruppen valgte å bruke Mosquitto som MQTT-broker fordi den er open source, gratis og relativt enkel å konfigurere. Collection-Service abonnerer på et emne som kan konfigureres, og mottar alle meldingene fra ChirpStack.

Mottak av sensordata med goroutines

Ved implementasjon av tjenesten ble det tydelig at to utfordringer måtte håndteres. Systemet måtte kunne ta imot potensielt mange sensormeldinger uten å blokkere MQTT-klienten. For å løse dette implementerte gruppen en worker pool bestående av ti goroutines, der hver worker har en buffret kanal med kapasitet på 1000 meldinger. Dette viste seg å være mer enn robust nok til å håndtere store mengder sensormeldinger. Mer om dette kan leses i 8.3.

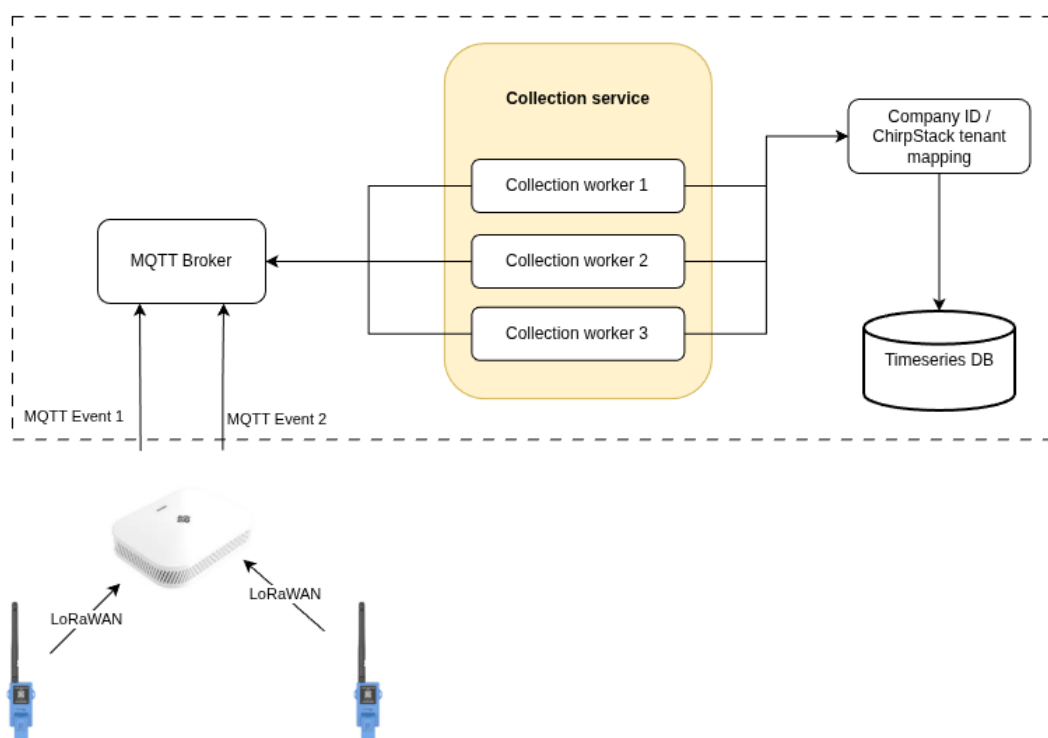


Figure 19: Sensorinnhenting

Når en melding ankommer MQTT-handleren, beregnes en hash fra meldingens innhold for å bestemme hvilken worker som skal behandle den. Dette er gjort for å sikre at meldinger fra samme sensor alltid blir behandlet av samme worker, og eliminerer behovet for låsing ved skriving. Dersom en worker har nådd sin fulle kapasitet på 1000 meldinger, logges en advarsel og meldingen forkastes fremfor å blokkere MQTT-tråden. Faren for at dette skjer i systemet er svært liten, selv om applikasjonen skaleres betydelig opp, da det vil kreve flere tusen tilkoblede LoR-

aWAN-sensorer.

```
1 func (c *Collector) MQTTHandler(client mqtt.Client, msg mqtt.Message) {
2     var event ChirpstackUpEvent
3     if err := json.Unmarshal(msg.Payload(), &event); err != nil {
4         slog.Error("invalid uplink", "error", err)
5         return
6     }
7     workerindex := hash(event.DeduplicationID) % len(c.workers)
8     select {
9     case c.workers[workerindex] <- event:
10    default:
11        slog.Warn("MQTT collection is full")
12    }
13 }
```

Listing 20: Worker pool med hashing av sensordata

Lagring i TimescaleDB

Som beskrevet i avsnitt 5.4 benyttet gruppen TimescaleDB for lagring av sensordata. Tabellen er satt opp med en hypertable som er partisjonert på tidsstempelen for målingen. Dette gir mer effektiv komprimering og raskere spørringer på målinger basert på tidsintervaller, som brukes av Context-Service.

Payloaden som kommer fra sensorene lagres som JSONB, noe som gjør at applikasjonen kan lagre data fra sensorene uavhengig av sensormodell og hvordan payloaden ser ut. Ulike sensorer kan sende ulike payload-strukturer uten at databaseskjemaet må endres, og Collection-Service lagrer dem slik de kommer inn.

For å knytte sensordata til riktig selskap i applikasjonen har gruppen implementert en mapping-tabell som kobler ChirpStacks tenantId til riktig intern company_id i applikasjonen. ChirpStack opererer med sin egen tenantId, som ikke kjenner til eller er direkte tilknyttet applikasjonens interne company_id-er. Ved lagring slår Collection-Service derfor opp i denne mappingen og tagger alle målinger med riktig company_id for å sikre separasjon av måledata mellom selskaper. Dersom ingen mapping finnes, forkastets målingen.

Collection-simulator

For å kunne utvikle og teste systemet tidlig uten at gruppen hadde tilgang til fysiske sensorer implementerte gruppen en egen collection-simulator. Simulatoren er en separat tjeneste som lager en goroutine per konfigurerbar enhet og publiserer realistiske ChirpStack-meldinger til MQTT-broderen med et konfigurerbart tidsintervall. Det ble satt opp simulerte enheter for måling av temperatur, fuktighet og

strømmålinger. Simulatoren ble aktivt brukt under stresstesting av systemet, som er nærmere beskrevet i avsnitt 8.3.

7.7 Monitor ERP integrasjon

Integrasjon med Monitor ERP skjer ved hjelp av to tjenester. Den ene tjenesten står for å periodisk hente Monitor ERP-data og sende dette videre, slik at den andre kan motta data og lagre dem internt i en egen database. Denne løsningen ble valgt fordi det er tryggere å sende data ut fra tjenestene i stedet for å gjøre direkte kall mot serveren til ERP-systemet. Dette er også gunstig med tanke på sikkerhet, siden produksjonsbedriftene ikke trenger å åpne portene til internett.

Implementasjonen viste seg å være betydelig mer utfordrende enn gruppen opprinnelig forventet, spesielt når det gjelder dataauthenting. Dette skyldes at det ikke er konsistens i hvordan ulike bedrifter bruker Monitor ERP. Dette kom frem gjennom møter med Innoveria og ulike produksjonsbedrifter. Av den grunn kan ikke systemet avgjøre om produksjonsdata som tjenesten henter er konsistente med løsningen, og innhenting skjer derfor under forutsetningen om at Monitor ERP er brukt i henhold til dokumentasjonen.

ERP Agent Service

For løsningen er innhentingstjenesten implementert som en periodisk "worker", der en loop kjører en funksjon som både henter og sender data. Den krever fire parametere som vist i figur 21.

```
1 type Worker struct {  
2     interval      time.Duration  
3     cycleTimeout  time.Duration  
4     maxBackoff    time.Duration  
5  
6     runner domain.Runner  
7 }
```

Listing 21: Struktur for ERP Agent worker

Intervall bestemmer hvor ofte tjenesten henter ut data, slik at systemet unngår å kontinuerlig legge press på Monitor ERP. Videre brukes parameteren `cycleTimeout` for å sikre at klientkall mot både Monitor ERP og det interne systemet ikke blir hen-

gende dersom noe går galt. Dette håndteres ved hjelp av Go sitt innebygde context-bibliotek, som gjør det mulig å sette tidsbegrensninger på klientkall og avbryte dem ved timeout. I tillegg er det implementert et eget backoff-system som hindrer tjenesten i å fortsette med kontinuerlige klientkall dersom feil oppstår. Ved feil vil systemet gradvis vente lenger mellom hvert nye forsøk, noe som reduserer belastningen på Monitor ERP. Selve kjøringen av denne syklusen håndteres av en generisk funksjon for å støtte alle typer produksjonsdata.

```
1 func syncRows[T any](
2     ctx context.Context,
3     monitor domain.MonitorHandler,
4     erpSvc domain.ERPIngestClient,
5     monitorPath monitor.MonitorERPEndpoint,
6     opts url.Values,
7     erpPath erpserviceclient.Endpoint,
8 ) error {
9     rows := make([]T, 0)
10    if err := monitor.Query(ctx, monitorPath, opts, &rows); err != nil {
11        // Error log
12    }
13
14    if len(rows) == 0 {
15        return nil
16    }
17
18    if err := erpSvc.Post(ctx, erpPath, rows); err != nil {
19        // Error log
20    }
21    return nil
22 }
```

Listing 22: Generisk GET- og POST-funksjon

Figur 22 viser bruken av generisk programmering. Den tar typen T som en parameter, der funksjonen spesifiseres hvilke type produksjonsdata som skal hentes. Basert på dette kan Monitor-klienten hente riktig datasett. Videre vil ERP-klienten sende data per dataobjekt til den interne ERP-tjenesten for databehandling.

ERP Service

Under utvikling av den interne ERP-tjenesten oppstod det et problem knyttet til relasjoner mellom mottatt data. Produksjonsdata fra Monitor ERP var uavhengige av hverandre når det gjelder relasjoner fra ordre til operasjon og rapportering. Siden

dataen sendes separat oppstod det situasjoner hvor relaterte produksjonsdata ikke eksisterte i databasen på tidspunktet dataene ble lagret.

En diskutert løsning var å la ERP Agent Service samle alle datastrukturer i ett større HTTP-kall før dataene ble sendt videre til den interne ERP-tjenesten. Dette ville ikke løst problemet, ettersom hvis en feil oppstod i dataobjektet, ville samme problem med relasjoner oppstå. I tillegg ville HTTP-forespørselen blitt svært stor, noe som hadde ført til høy belastning på nettverket.

For å håndtere dette valgte gruppen å implementere en form for dataavstemming. Dette innebærer å sammenligne og slå sammen data fra ulike systemer for å sikre korrekt struktur mellom relaterte datasett. Formålet er å verifisere at relasjoner finnes og korrigere dem slik at dataene representerer en konsistent tilstand [29].

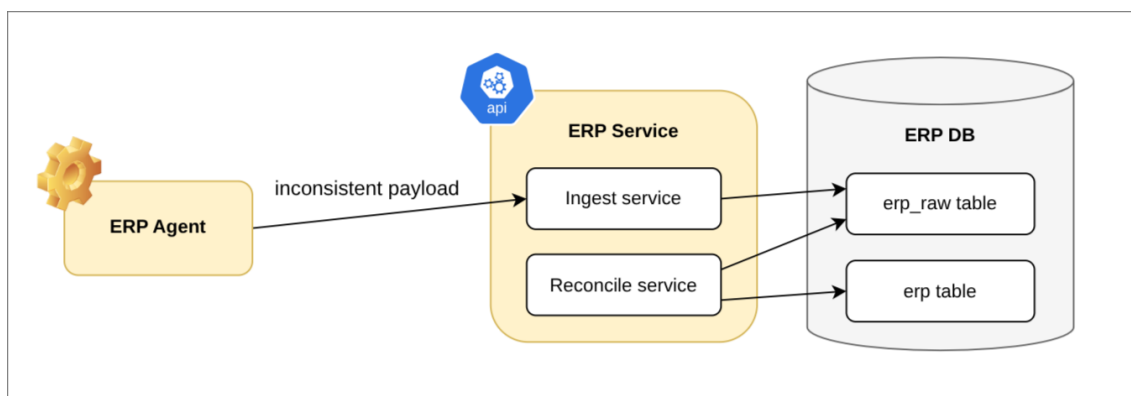


Figure 20: Dataavstemming av ERP-data

I praksis er løsningen implementert med to SQL-tabeller. Den ene tabellen fungerer som et mellomlager for usynkroniserte ERP-data mottatt fra ERP Agent Service, mens den andre inneholder ferdig avstemte data som brukes av resten av systemet. Dette gjør at data kan mottas og lagres selv om nødvendige relasjoner ennå ikke eksisterer i databasen.

Dataavstemmingen utføres av en egen tjeneste som periodisk kjører SQL-operasjoner for å kontrollere at nødvendige relasjoner eksisterer før data flyttes videre til den ferdig avstemte tabellen. Dersom avhengigheter som ordre eller arbeidssenter mangler, blir datasettet utsatt og forsøkt synkronisert igjen senere. Når alle relasjoner er tilgjengelige, blir dataene overført til den andre tabellen gjennom en UPSERT-operasjon, som oppdaterer hvis den finnes eller setter inn hvis den ikke finnes.

7.8 Kontekstbasert sensordata

7.8.1 Tjenestens rolle i systemarkitekturen

Context-Service er mikrotjenesten som setter produksjonsdata i kontekst. Der de øvrige tjenestene i systemet har et klart avgrenset domene - innsamling av målinger, administrasjon av enheter eller håndtering av produksjonsordre - har Context-Service en helt annen rolle: den henter data fra flere domener, utfører beregninger og setter resultatet sammen til en meningsfull, kontekstuell helhet.

Tjenesten har tre avhengigheter. Fra **Collection-Service** henter den rå måledata fra sensorene. Fra **Device-Service** henter den metadata om sensorene, herunder hvilke produksjonsressurser de er knyttet til og hvilke målingstyper de er konfigurert for. Fra **ERP-Service** henter den produksjonsordre og arbeidsoperasjoner. Context-Service eier ingen av disse datakildene, men er den eneste tjenesten som kombinerer dem.

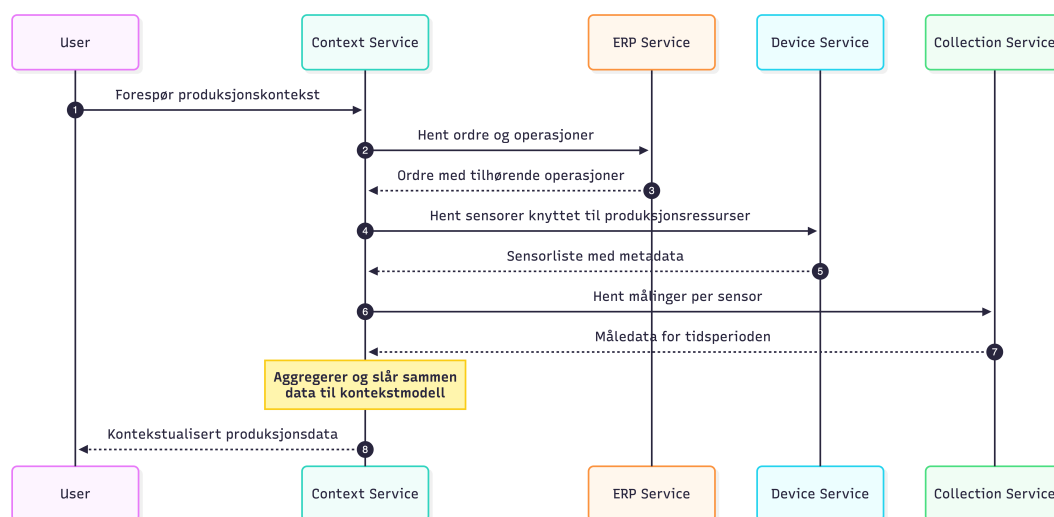


Figure 21: Overordnet dataflyt for Context-Service

For detaljert sekvensdiagram, se vedlegg H.

Dette gir to primære brukstilfeller. Det første er *aggregert sensordata*: gitt én eller flere sensorer og en tidsperiode beregner tjenesten aggregerte verdier fordelt på definerte tidsvinduer. Det andre er *ordrekontekst*: gitt en produksjonsordre beriker tjenesten ordren med sensordata fra de produksjonsressursene som er involvert i ordrens operasjoner, slik at energiforbruk og andre målinger kan knyttes direkte til de konkrete utførte operasjonene ved en ordre.

7.8.2 Integrasjon mot eksterne tjenester

Hver av de tre tjenestene Context-Service kommuniserer med er implementert som en dedikert HTTP-klient. Klientene er enkle og holder kun på basis-URL og en delt HTTP-klient. Operasjonene eksponeres gjennom grensesnitt definert i domenelaget, noe som gjør det enkelt å bytte ut en klient - for eksempel med en mock-klient under utvikling.

Interne porter for interne kall

Device-Service eksponerer to porter: en autentisert port (8080) for eksterne klienter og en intern port (9090) uten autentisering for kall innad i systemet. Context-Service benytter begge disse portene.

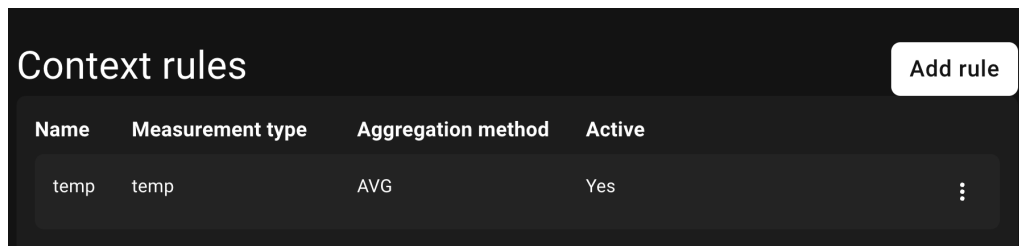
Kall som henter sensorer tilknyttet en gitt produksjonsressurs gjøres mot den autentiserte porten og videresender brukerens auth-headere. Kall som henter sensorens konfigurerte målingstyper gjøres mot den interne porten - her finnes ingen brukerkontekst å videresende, og autentisering er ikke nødvendig siden kommunikasjonen aldri forlater det interne nettverket.

```
1 // Authenticated public port - user context required.
2 func (c *DeviceClient) GetSensorsByProductionResourceID(
3     ctx context.Context, companyID, userID, role string,
4     productionResourceID int64,
5     ) ([]domain.DeviceSensor, error) {
6     url := fmt.Sprintf(
7         "%s/api/v1/device/sensors?production_resource_id=%d",
8         c.baseURL, productionResourceID
9     )
10    // ...
11 }
12
13 // Internal port - no auth headers needed.
14 func (c *DeviceClient) GetSensorMetrics(
15     ctx context.Context, deviceEUI string,
16     ) ([]domain.SensorMetric, error) {
17     url := fmt.Sprintf(
18         "%s/api/v1/device/sensors/%s/metrics",
19         c.internalBaseURL, deviceEUI
20     )
21    // ...
22 }
```

Listing 23: To porter - autentisert ekstern og uautentisert intern

7.8.3 Aggregasjonsregler

For at Context-Service skal vite hvordan råmålinger fra sensorer skal aggregeres, trenger den konfigurasjon: hvilken målingstype som er relevant, hvilken aggregeringsmetode som skal brukes og hvilket tidsvindu resultatet skal beregnes etter. Denne konfigurasjonen er den eneste dataen Context-Service selv eier og holder i sin PostgreSQL-database.



Name	Measurement type	Aggregation method	Active
temp	temp	AVG	Yes

Figure 22: Context regel i frontend

Regelmodellen

En aggregasjonsregel knytter en konteksttype til en målingstype og definerer hvordan verdiene skal aggregeres. Konteksttypen er en semantisk etikett som beskriver hva konteksten representerer - for eksempel strømforbruk eller temperatur. Aggregasjonsmetoden er en av fire verdier: **AVG**, **SUM**, **MAX** eller **MIN**. I tillegg angir regelen en standard tidsvindustørrelse i minutter, som benyttes dersom kalleren ikke selv oppgir en ønsket oppløsning.

```
1  type AggregationRule struct {
2      ID                string
3      CompanyID         string
4      Name              string
5      ContextType       string
6      MeasurementType   string
7      AggregationMethod AggMethod
8      TimeBucketMinutes int
9      IsActive          bool
10     CreatedAt          time.Time
11     UpdatedAt          time.Time
12 }
```

Listing 24: Domenemodell for en aggregasjonsregel

7.8.4 Kalkulator og aggregasjonslogikk

Context-Service inneholder to kalkulatorer som utfører de faktiske beregningene på råmålingene: en generisk aggregeringskalkulator og en watt-time-kalkulator. Begge implementerer et felles `Calculator`-interface.

```
1  type Calculator interface {  
2      Calculate(input Input) (domain.ContextData, error)  
3  }
```

Listing 25: Kalkulator-interfacet — alle kalkulatorer implementerer dette

Input til interfacet er en strukturert pakke med alt som trengs for å utføre beregningen: aggregasjonsregelen som styrer metode og målingstype, råmålingene fra sensoren, tidsvinduet og ønsket tidsvindustørrelse. Watt-time-kalkulatoren trenger i tillegg linjespenning og navnet på payload-nøkkelen (se 7.5.4) som inneholder strømmålingen.

Generisk aggregering

`GenericAggregationCalculator` er standardkalkulatoren og håndterer alle konteksttyper der en payload-nøkkel skal aggregeres med en av de fire metodene nevnt i 7.8.3. Tidsvinduet deles inn i like store intervaller, og aggregasjonsmetoden kjøres separat per bølge. I tillegg beregnes en samlet totalverdi over hele perioden.

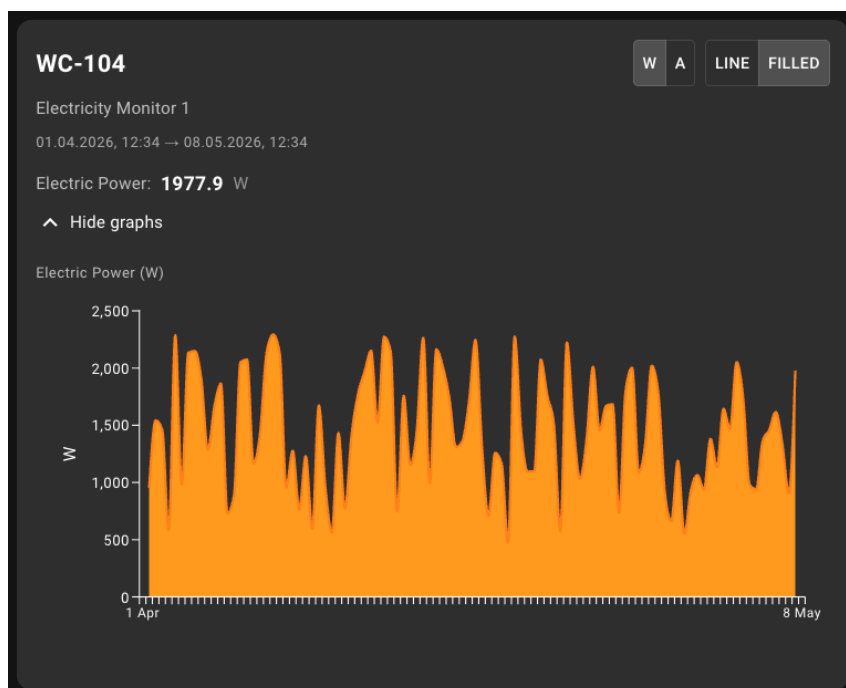


Figure 23: Strømsensor data

```

1   for i := range buckets {
2       inBucket := readingsInBucket(
3           input.Readings,
4           buckets[i].PeriodStart,
5           buckets[i].PeriodEnd,
6           i == lastIdx,
7       )
8       buckets[i].Value, err = aggregate(
9           inBucket,
10          input.Rule.MeasurementType,
11          string(input.Rule.AggregationMethod),
12      )
13  }
14
15  total, err := aggregate(
16      input.Readings,
17      input.Rule.MeasurementType,
18      string(input.Rule.AggregationMethod),
19  )

```

Listing 26: Aggregering per tidsvindu og totalt for hele perioden

Den siste tidsvinduet behandles alltid med inklusiv sluttgrense, slik at målinger som faller nøyaktig på periodeslutt ikke går tapt.

Watt-time-kalkulator

For elektriske sensorer er ikke enkel aggregering tilstrekkelig. Øyeblikksmålinger av strøm (ampere) sier lite i seg selv - det relevante er energiforbruket over tid, målt i watt-timer (Wh). `WattHourCalculator` beregner dette ved å konvertere strømmålinger til watt ved hjelp av ohms lov, og integrerer deretter effektkurven over tid ved hjelp av trapesmetoden¹⁹.

¹⁹<https://hscprep.com.au/hsc-maths-advanced/the-trapezoidal-rule/>

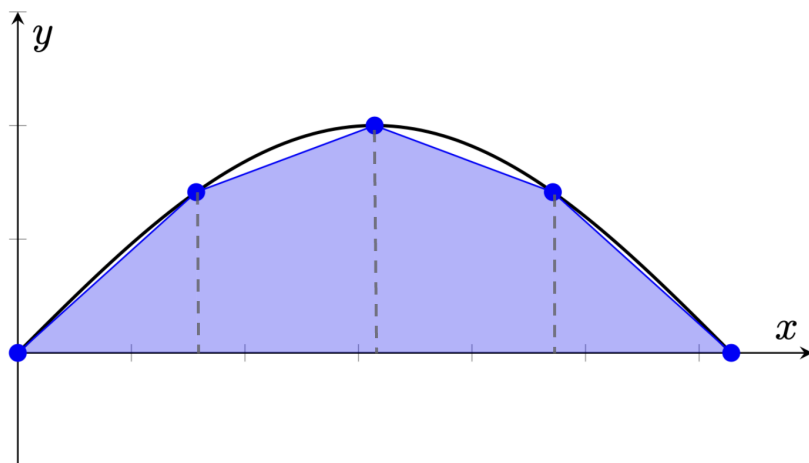


Figure 24: Trapesmetoden illustrert. Figur hentet fra hscprep.com.au[30]

```
1  for i := 1; i < len(readings); i++ {
2      w0 := readings[i-1].Amps * voltageV
3      w1 := readings[i].Amps * voltageV
4      dt := readings[i].Timestamp.Sub(readings[i-1].Timestamp).Hours()
5      energy += (w0 + w1) / 2 * dt
6  }
```

Listing 27: Trapesintegrasjon for beregning av energiforbruk i watt-timer.

Trapesmetoden approksimerer arealet under effektkurven ved å behandle hvert intervall mellom to målinger som et trapes. Dette gir en rimelig god tilnærming så lenge sensoren sender målinger med jevne mellomrom. Dersom et tidsintervall inneholder færre enn to målinger, er det ikke mulig å definere et intervall å integrere over - botten returnerer da 0 Wh og logger en advarsel.

Fallback mellom kalkulatorer

`GetContextData` bruker alltid `GenericAggregationCalculator`, mens `GetOrderContext` instansierer `WattHourCalculator` direkte med sensor-spesifikk metadata. Skillet holder den generelle aggregeringsflyten enkel og uten avhengighet til `Device-Service`.

7.8.5 Beregning av kontekstdata

`GetContextData` er det sentrale endepunktet for aggregert sensordata. Kalleren oppgir én eller flere sensor-EUler, en aggregasjonsregel og et tidsintervall. Tjenesten henter råmålinger og returnerer beregnede kontekstverdier fordelt på tidsvinduer.

Tidsvinduer

Tidsvinduet fra `from` til `to` deles inn i like store intervaller basert på reglens `TimeBucketMinutes`. Kalleren kan overstyre reglens standard tidsvindustørrelse per forespørsel ved å sende `bucket_minutes` som query-parameter, noe som gjør det mulig å hente samme regel med ulik oppløsning avhengig av visningsbehovet.

```
1 func buildBuckets(from, to time.Time, bucketMins int) []domain.ContextBucket {
2     step := time.Duration(bucketMins) * time.Minute
3     var buckets []domain.ContextBucket
4     start := from
5     for start.Before(to) {
6         end := start.Add(step)
7         if end.After(to) {
8             end = to
9         }
10        buckets = append(buckets, domain.ContextBucket{
11            PeriodStart: start,
12            PeriodEnd:   end,
13        })
14        start = end
15    }
16    return buckets
17 }
```

Listing 28: Tidsvinduet deles inn i like store tidsvinduer

Berikelse på lesetid

Context-Service lagrer ikke beregnede resultater. Aggregering skjer på lesetid - det vil si at beregningene utføres hver gang en forespørsel kommer inn, fremfor å mellomlagre resultater som kan bli utdaterte ved neste henting. For hver sensor-EUI hentes råmålinger fra Collection-Service, og kalkulatoren kjøres umiddelbart på det innhentede datasettet. En sensor uten målinger i tidsvinduet betraktes ikke som en feil - tjenesten returnerer et tomt resultat med tomme tidsbøtter, slik at kalleren alltid får et svar selv om en sensor mangler data.

7.8.6 Ordrekontekst og datafusjon

`GetOrderContext` er den mest sammensatte operasjonen i Context-Service. Der `GetContextData` opererer på én målingstype for gitte sensorer, setter `GetOrderContext` en hel produksjonsordre i kontekst: ordren hentes fra ERP-Service, og hver arbeidsoperasjon berikes med sensordata fra den tilknyttede produksjonsressursen. For detaljert sekvensdiagram, se vedlegg H.

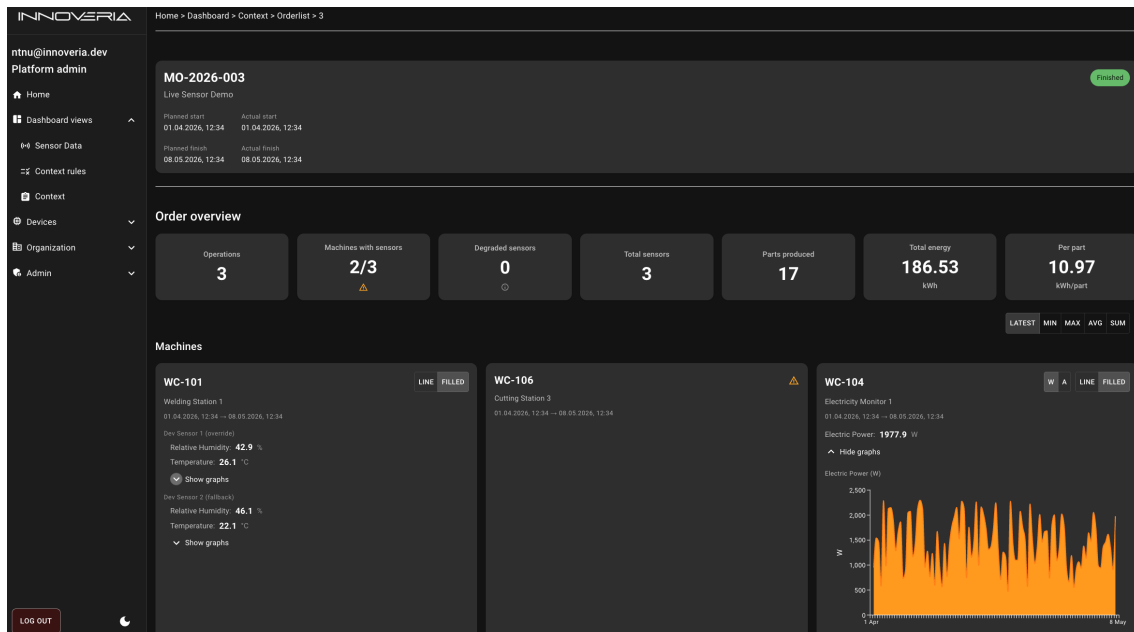


Figure 25: Ordrekontekst i dashboard

ERP-ordre som utgangspunkt

Flyten starter med å hente ordren fra ERP-Service. Ordren inneholder en liste over arbeidsoperasjoner, og hver operasjon er knyttet til en arbeidsressurs (en maskin eller et arbeidssted). Tidsvinduet for målingene styres av ordrens faktiske start- og sluttidspunkt. Dersom ordren ikke har registrert faktisk start eller avslutning, finnes det ingen tidsperiode å hente målinger for, og operasjonene returneres uten sensordata. Kontekst kan således kun hentes for operasjoner som er ferdig utført.

```

1      if found.ActualStartDate == nil || found.ActualFinishDate == nil {
2          for i, op := range found.Operations {
3              ops[i] = domain.OperationContext{
4                  Operation: op,
5                  Sensors: []domain.SensorContext{},
6              }
7          }
8          return &domain.OrderContext{Order: *found, Operations: ops}, nil
9      }

```

Listing 29: Operasjoner uten tidsvindu returneres uten sensordata

Parallell henting av sensordata

Når tidsvinduet er kjent, bygges konteksten for hver arbeidsoperasjon parallelt. For en ordre med mange operasjoner ville sekvensiell henting medføre unødvendig høy ventetid, ettersom hvert kall til Device-Service og Collection-Service er uavhengig

av de andre. Dette løses ved hjelp av goroutines og `sync.WaitGroup`, som lar alle operasjoner hentes samtidig.

```
1     var wg sync.WaitGroup
2     for i, op := range found.Operations {
3         wg.Go(func() {
4             ops[i] = s.buildOperationContext(
5                 ctx, companyID, userID, role, op, from, to,
6             )
7         })
8     }
9     wg.Wait()
```

Listing 30: Operasjonskontekster bygges parallelt med goroutines

Det samme mønsteret gjentas ett nivå ned: innenfor hver operasjon hentes konteksten for hver enkelt sensor parallelt.

Degradert tilstand ved delvise feil

I et distribuert system kan enkeltoperasjoner feile uten at hele forespørselen bør avbrytes. Dersom henting av sensorer for en produksjonsressurs feiler teknisk - for eksempel ved timeout eller 5xx-feil fra Device-Service - returneres operasjonen med `Degraded: true` og tomme sensorer, mens resten av ordren leveres upåvirket. Dersom en produksjonsressurs ikke har noen sensorer registrert, behandles dette som en forventet tilstand og regnes ikke som en feil.

```

1      sensors, err := s.deviceClient.GetSensorsByProductionResourceID(
2      ctx, companyID, userID, role, op.ProductionResource.ID,
3      )
4      if err != nil {
5          if errors.Is(err, domain.ErrNotFound) {
6              // Ingen sensorer på denne ressursen
7              // -> forventet, ikke en feil.
8              return domain.OperationContext{
9                  Operation: op,
10                 Sensors: []domain.SensorContext{},
11             }
12         }
13         // Teknisk feil - returner degradert operasjon.
14         return domain.OperationContext{
15             Operation: op,
16             Sensors: []domain.SensorContext{},
17             Degraded: true,
18         }
19     }

```

Listing 31: Teknisk feil gir degradert operasjon; manglende sensorer er forventet tilstand

For elektriske sensorer beregnes i tillegg totalt energiforbruk i Wh over ordreperioden ved hjelp av `WattHourCalculator`, som beskrevet i seksjon om kalkulatorer (7.8.4). Resultatet lagres direkte på sensorkonteksten og er tilgjengelig for frontend uten videre behandling.

8 Kvalitetssikring

8.1 Gjennomgang av kode

For å sikre kvaliteten av kodebasen ble det lagt vekt på å gjennomføre grundige MR-gjennomganger. Dette var både for å sikre kvaliteten på koden og for å gi utviklingsteamet, bestående av fire medlemmer, en sikring mot å bli for spesialiserte, som nevnt i kapittel 3.5.

Det ble ikke utformet en mal for hvordan en MR skulle gjennomgås, men gruppen var enig om viktigheten av at dette ble gjort. Som et supplement ble KI benyttet for å sikre kodekvaliteten ytterligere. Dette ble gjort gjennom agenter og skills som gjennomgikk kodedifferansen mellom utviklingsbranchen og branchen den skulle merges inn i. Bruken av KI på denne måten ga stor gevinst for prosjektet - blant annet ble det avdekket funksjonalitet som kun virket fordi to felter tilfeldigvis hadde samme navn, noe som kunne ført til feil ved fremtidige endringer.

Ressursbruken har ingen konkrete tall, men med over 220 MRs i prosjektet totalt, og en antakelse om at hver MR gjennomsnittlig tok 45 minutter å gjennomgå, kan man anslå at ressursbruken har vært omtrent 165 timer av utviklingstiden - tilsvarende 14,1% av den totale utviklingstiden. Gjennomsnittlig tid brukt på MR-gjennomgang per utvikler per uke blir da 2,3 timer. Dette er i tråd med funn fra Heander et al., som rapporterer et gjennomsnitt på 4 timer per uke per utvikler med et standardavvik på 2 timer[31], noe som plasserer prosjektets estimat innenfor ett standardavvik fra dette gjennomsnittet.

Med både manuell kodegjennomgang og KI-assistert gjennomgang initiert av utviklerne selv ble kvaliteten og forståelsen av kodebasen ivaretatt, supplert av automatisk kontroll av kodekvalitet.

8.2 Kodekvalitet

Fra prosjektets oppstart ble kodebasen konfigurert med automatisert linting, formatering og testing for å sikre at kildekoden fulgte konsistente standarder gjennom hele utviklingsløpet. Dette ble implementert på bakgrunn av erfaringer fra integrasjonssprosjektet, der kode med byggefeil ved ett tilfelle ble pushet til repoet - noe som medførte at koden aldri ville kjøre. To mekanismer ble benyttet i kombinasjon: en pre-commit git-hook og en Continuous Integration-pipeline.

```
"lint-staged": {
  "*.{ts,tsx}": ["eslint --fix", "prettier --write"],
  "*.{json,css,md,html,yml}": ["prettier --write"]
}
```

Listing 33: lint-staged konfigurasjon

Pre-commit git-hook

Pre-commit-hooken utgjorde det første kvalitetslaget i prosjektet og kjørte lokalt på utviklerens maskin før kode ble registrert i versjonskontroll. Verktøyet som ble brukt var Husky²⁰, som ble valgt fremfor en manuelt plassert git-hook fordi hook-scriptet da versjonskontrolleres sammen med resten av kodebasen og installeres automatisk for alle utviklere via `prepare`-stadiet i `package.json`, vist i listing 32. Denne tilnærmingen sikrer at kodenstandarden håndheves konsekvent uavhengig av hvem som committer, samtidig som enkelte feil fanges opp før de belaster CI-infrastrukturen.

```
"scripts": {
  "dev": "vite",
  "build": "tsc -b && vite build",
  "lint": "eslint .",
  "format": "prettier --write .",
  "formatcheck": "prettier --check .",
  "prepare": "cd .. && husky web/.husky"
}
```

Listing 32: scripts-feltet i `web/package.json`

Hooken var delt inn i tre uavhengige steg, ett per språk i kodebasen:

Frontend - TypeScript

Verktøyene Husky og lint-staged²¹ ble benyttet for frontend-koden. Husky registrerer git-hooken og kaller lint-staged ved hver `git commit`-kommando. lint-staged begrenser kjøringen til kun de filene som er klargjort for commit (*staged*) noe som reduserer kjøretiden betraktelig sammenlignet med å kjøre verktøyet på hele kodebasen. Konfigurasjonen i `package.json` definerte følgende regler:

ESLint²² håndterte statisk analyse av TypeScript-koden, mens Prettier²³ sørget for konsekvent i formateringen. Dersom ESLint rapporterte feil som ikke kunne rettes

²⁰<https://typicode.github.io/husky/>

²¹<https://github.com/lint-staged/lint-staged>

²²<https://eslint.org/>

²³<https://prettier.io/>

automatisk (med `eslint --fix`) ble commit-operasjonen avbrutt og utvikleren får en feilmelding om hva som måtte rettes.

Regel	Beskrivelse
<code>no-console</code>	Advarer ved bruk av <code>console.*</code>
<code>no-restricted-imports</code>	Forbyr direkte import fra <code>@mui</code> toppnivå
<code>jsdoc/require-jsdoc</code>	Krever JSDoc på eksporterte funksjoner og komponenter
<code>jsdoc/require-description</code>	Krever beskrivelsestekst i JSDoc-kommentarer
<code>jsdoc/require-returns</code>	Krever <code>@returns</code> der funksjon returnerer en verdi

Table 8: Konfigurerte regler i `web/eslint.config.js`

Ettersom lint-staged er et JavaScript-verktøy som opererer per fil, lot det seg ikke benytte direkte for Go. Go-verktøykjeden arbeider på pakkenivå fremfor filnivå, noe som innebærer at lintere og formaterere må kjøres mot hele pakken en fil tilhører, ikke mot filen isolert. Av denne grunn ble det etablert egne shell-script-steg i pre-commit-hooken for å utføre tilsvarende oppgaver for Go-koden.

Go

For Go-koden kjørte hooken `gofmt` for automatisk formatering og `golangci-lint` for statisk analyse. For å holde kjøretiden lav ble kun pakkene som inneholdt stagede filer lintet, ikke hele tjenesten. Hooken itererte over samtlige Go-tjenester i monorepoet og kjørte lintingen kun i modulene som var berørt av den aktuelle committen:

```
GO_SERVICES="pkg services/api-gateway services/collection-service
↪ \
  services/device-service services/auth-service ..."

for dir in $GO_SERVICES; do
  if [ -d "$dir" ]; then
    PKGS=$(echo "$STAGED_GO" | grep "^$dir/" | ...)
    if [ -n "$PKGS" ]; then
      echo "  Tidying and linting changed packages in $dir:
↪   $PKGS"
      (cd "$dir" && go mod tidy && git add go.mod go.sum
↪   2>/dev/null; \
        go tool golangci-lint run $PKGS)
    fi
  fi
done
```

Listing 34: Go-linting per pakke i pre-commit-hooken (`web/.husky/pre-commit`)

Regel	Beskrivelse
govet	Mistenkelige konstruksjoner (f.eks. ugyldige printf-argumenter)
staticcheck	Avansert statisk analyse over mange feilklasser
errcheck	Finner uhåndterte feil (f.eks. ignorert retur fra <code>os.Close</code>)
ineffassign	Oppdager tilordninger til variabler som aldri leses
unused	Finner ubrukte konstanter, variabler, funksjoner og typer
gocritic	Samling av sjekker ikke dekket av andre lintere
godoclint	Håndhever doc-kommentarer på eksporterte symboler og pakker

Table 9: Aktiverte lintere i `.golangci.yml`

Skriptet sjekker først om det finnes stagede `.go`-filer. Hvis det er tilfellet, formateres disse automatisk med `gofmt -w` og restages, før `go mod tidy` kjøres for å holde `go.mod` og `go.sum` synkronisert. Skriptet itererer deretter over tjenestene og kjører *golangci-lint* kun på pakkene som inneholder stagede filer, fremfor hele tjenesten — noe som reduserer unødvendig ressursbruk på uendret kode. En klar svakhet med dette skriptet er at tjenestene er listet opp i den hardkodete variabelen `GO_SERVICES`, noe som innebærer at en eventuell skrivefeil stille ville hoppet over en tjeneste uten å gi utvikleren tilbakemelding.

SQL

For migrasjons- og spørrefiler ble `sql-lint`²⁴ kjørt mot stagede `.sql`-filer, konfigurert via `.sql-lint.json` i roten av prosjektet til PostgreSQL-dialekt med ignorerings-av `trailing-whitespace`. Dette representerer en enkel form for linting — verktøyet kontrollerer utelukkende syntaktisk gyldighet og utfører verken semantisk analyse eller formatering. Dersom én eller flere filer ikke bestod sjekken, ble commiten avbrutt.

```
STAGED_SQL=$(git diff --staged --name-only --diff-filter=ACM -- '*.sql')

if [ -n "$STAGED_SQL" ]; then
  FAILED=0
  for file in $STAGED_SQL; do
    OUTPUT=$(cd web && bunx sql-lint --config "../.sql-lint.json" \
      "$file" 2>&1) || true
    if [ -n "$OUTPUT" ]; then
      FAILED=1
    fi
  done
  if [ "$FAILED" -eq 1 ]; then exit 1; fi
fi
```

Listing 35: SQL-linting i pre-commit-hooken (`web/.husky/pre-commit`)

²⁴<https://sql-lint.readthedocs.io/en/latest/index.html>

Type	Delt	Golang	React	SQL
git-hook	Husky		lint-staged	
Linting		golangci-lint	ESLint	sql-lint
Formatering		gofmt	Prettier	

Table 10: Lint- og formateringsteknologi brukt i prosjektet

CI pipeline

Gruppen satte opp en Continuous Integration (CI)-pipeline i GitLab som kjørte ved enhver merge request. I motsetning til pre-commit-hooken, som kun sjekker stagede filer, testes hele kodebasen i CI-pipelen. For at kode skal kunne merges må pipelen gjennomføres uten feil. Pipelen består av tre stadier – *check*, *build* og *test*, der hvert stadium er konfigurert for både frontend og samtlige backend-tjenester i repositoriet.

CI-pipelen fungerte som et sikkerhetsnett for situasjoner der den lokale pre-commit-hooken ikke fanget opp feil. Et praktisk eksempel er dersom en utvikler omgikk den lokale hooken med `git commit --no-verify`, eller jobbet i et miljø der Husky ikke var korrekt installert. I slike tilfeller ville feil først bli oppdaget i CI-pipelen, og koden kunne ikke merges før disse var utbedret. CI-pipelen ga dermed en garanti for at kvalitetskravene ble håndhevet uavhengig av lokalt oppsett.

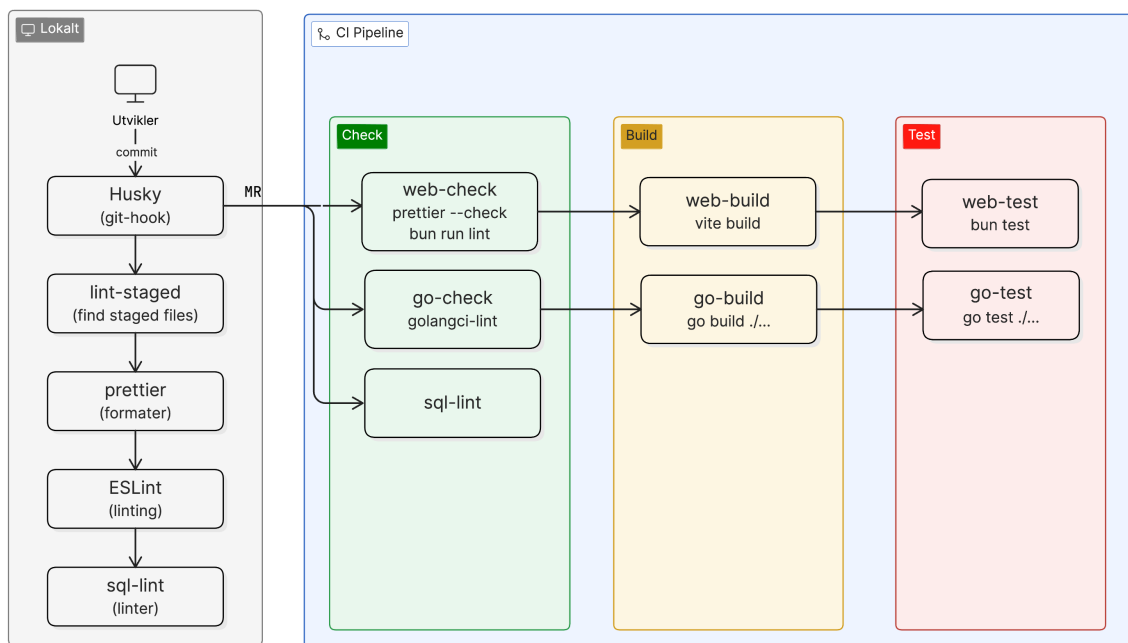


Figure 26: Lokal- og CI-pipeline

8.3 Testing

Teststrategi

Testingen i prosjektet ble ikke prioritert i tilstrekkelig grad. Fokuset gjennom utviklingsperioden lå på å levere funksjonalitet innenfor de gitte tidsrammene, og testing ble i stor grad nedprioritert til fordel for dette. Samtlige tester ble generert av KI. Testdekningen er begrenset og konsentrert om backend - primært kalkulatorlogikk og HTTP-handlere i *context-service* og *device-service* - mens flere tjenester mangler tester fullstendig. Frontend har et minimalt testdekke bestående av et par enhets-tester for isolerte hjelpefunksjoner. En mer systematisk teststrategi, gjerne basert på eller inspirert av Test-Driven Development, ville ideelt sett ha vært fulgt fra prosjektets oppstart.

KI er som nevnt brukt til generering av tester. Mer om dette og hvordan gruppen har brukt KI ytterlig til kvalitetssikring er beskrevet i seksjon Utviklingsprosess 3.6 og Diskusjon 9.3.4)

Enhetstesting

Det ble skrevet enhetstester for å verifisere sentral funksjonalitet i applikasjonen, med hovedvekt på backend. Frontend har et mer begrenset testdekke, noe gruppen anerkjenner som et forbedringsområde. Store deler av testene ble generert med bistand fra KI, da skriving av enhetstester er en repetitiv oppgave som egner seg godt for dette formålet — men samtlige tester ble gjennomgått og kvalitetssikret manuelt før de ble inkludert i kodebasen.

Et eksempel på testet funksjonalitet er `watt_hour.go`, en kalkulator spesialisert for energiberegning som konverterer strømmålinger (ampere) til energi (Wh) ved hjelp av trapesintegrasjon over tid.(se figur 24).

Kodesnutten nedenfor viser en enhetstest som verifiserer at kalkulatoren returnerer korrekt verdi for to avlesninger på 1A over én time. Med en konstant strøm på 1A og en spenning på 230V over én time forventes et resultat på 230 Wh — noe testen bekrefter ved å sammenligne kalkulatorens returverdi mot denne forventede verdien.

```

1  func TestTrapezoidalWh_TwoReadings(t *testing.T) {
2      t0 := time.Date(2026, 1, 1, 0, 0, 0, 0, time.UTC)
3      t1 := t0.Add(1 * time.Hour)
4
5      readings := []domain.MeasurementReading{
6          wattReading(1.0, t0),
7          wattReading(1.0, t1),
8      }
9
10     got, err := trapezoidalWh(readings, "current")
11
12     if err != nil {
13         t.Fatalf("unexpected error: %v", err)
14     }
15     if got != 230.0 {
16         t.Errorf("expected 230.0 Wh, got %v", got)
17     }
18 }

```

Listing 36: Enhetstest for trapesintegrasjon med to avlesninger på 1A over 1 time

Testdekningen i prosjektet som helhet er begrenset, noe som i hovedsak skyldes en bevisst prioritering av funksjonalitet fremfor testing gitt omfanget av det som skulle utvikles innenfor prosjektperioden. I flere tilfeller ble tester skrevet i etterkant av implementert funksjonalitet, fremfor å følge en testdrevet tilnærming. Ideelt sett skulle gruppen ønsket at testingen i større grad hadde fulgt prinsippene fra TDD, der tester skrives før funksjonaliteten implementeres. Dette ville bidratt til bedre testdekning og potensielt avdekket feil tidligere i utviklingsløpet.

Områder som særlig ville hatt nytte av bedre testdekning er blant annet handler-funksjonene i backend, samt et felles testrammeverk som kunne vært gjenbrukt på tvers av de ulike backend-tjenestene i repositoret.

Integrasjonstest

Det ble skrevet integrasjonstester for *context-service* ved hjelp av Go sitt innebygde `net/http/httptest`-pakke. I motsetning til enhetstestene, som tester isolerte funksjoner, verifiserte integrasjonstestene at hele request-respons-syklusen fungerer korrekt. Dette inkluderer alt fra HTTP-ruting gjennom tjenestelag og ned til repository-laget. Nestrømsavhengigheter som *ERP-service*, *device-service* og *collection-service* ble erstattet med stubber HTTP-servere, slik at testene er deterministiske og ikke krever at øvrige tjenester kjører.

To sentrale scenarier som ble dekket var:

- **Degradert operasjon**

Testen `TestIntegration_OrderContext_DegradedOperation` verifiserer at dersom *device-service* returnerer 5xx for én produksjonsressurs, markeres den tilhørende operasjonen som `degraded: true`, mens en søskenoperasjon med vellykket svar forblir upåvirket.

- **Bedriftsisolasjon**

Testen `TestIntegration_Rules_CrossCompanyIsolation` bekrefter at aggregeringsregler opprettet av én bedrift ikke er synlige for en annen bedrift, og at dataene dermed er korrekt isolert på bedriftsnivå.

Repository-laget, det vil si de faktiske SQL-spørringene mot PostgreSQL, er ikke dekket av automatiserte tester. Integrasjonstestene som finnes benytter in-memory-implementasjoner av repository-interfacene for å unngå databaseavhengighet i CI-pipelinen. Dette betyr at selve databaseinteraksjonen aldri verifiseres gjennom integrasjonstesting. Et verktøy som ville muliggjort oppstart av en reell PostgreSQL-instans i CI er testcontainere. Disse ville gitt ekte databasetesting, men ble nedprioritert gitt prosjektets tidsrammer og til fordel for utvikling av funksjonalitet.

```

1 //go:build integration
2
3 func newTestServer(t *testing.T,
4     erpHandler, deviceHandler, collectionHandler http.Handler,
5 ) testServerResult {
6     t.Helper()
7
8     erpStub      := httptest.NewServer(erpHandler)
9     deviceStub   := httptest.NewServer(deviceHandler)
10    collectionStub := httptest.NewServer(collectionHandler)
11
12    repo          := newInMemoryRuleRepo()
13    erpClient      := clients.NewERPClient(erpStub.URL)
14    deviceClient   := clients.NewDeviceClient(deviceStub.URL,
15        ↪ deviceStub.URL)
16    collectionClient :=
17        ↪ clients.NewCollectionClient(collectionStub.URL)
18
19    contextSvc :=
20        ↪ services.NewContextServiceImpl(collectionClient,
21        ↪ erpClient, deviceClient, repo)
22    ruleSvc     := services.NewRuleServiceImpl(repo)
23
24    mux      := server.NewRouter(contextSvc, ruleSvc)
25    mainSrv := httptest.NewServer(mux)
26
27    t.Cleanup(func() {
28        mainSrv.Close()
29        erpStub.Close()
30        deviceStub.Close()
31        collectionStub.Close()
32    })
33
34    return testServerResult{srv: mainSrv, ruleRepo: repo}
35 }

```

Listing 37: Hjelpfunksjon for integrasjonstester i context-service (integration_test.go)

Integrasjonstestene ble isolert bak build-taggen `//go:build integration` slik at de ikke kjøres automatisk med `go test ./...`. Dette er en bevisst organisatorisk avgjørelse for å skille tester som verifiserer samspillet mellom flere lag fra de ordinære enhetstestene, og for å gjøre det eksplisitt når en mer omfattende testjobb kjøres med `go test -tags integration`.

Stresstesting

Stresstesting ble gjennomført for å verifisere at systemet oppfyller de ikke-funksjonelle ytelseskravene som ble satt tidlig i prosjektet. Følgende krav var grunnlaget for testene:

- Systemet skal kunne håndtere 300 samtidige brukere
- Systemet skal kunne håndtere minst 600 sensormålinger per time
- Svartid fra backend under normale forhold skal være under ett sekund

For å gjennomføre testene ble det benyttet to verktøy: prosjektets egen *collection-simulator* for MQTT-last, og k6 [32] for HTTP-belastningstesting. Sensortesten ble kjørt lokalt mot et fullstendig utviklingsmiljø. HTTP-belastningstesten ble kjørt direkte på produksjonsserveren (`prog2900-lorawan.vm.iik.ntnu.no`) mot det kjørende produksjonsmiljøet, med k6 på samme host via Caddy og TLS. Ekstern nettverksforsinkelse fra klienten er dermed ikke inkludert, og reell sluttbrukersvartid vil være noe høyere avhengig av klientens plassering.

Test 1 - Sensordata

For å teste om applikasjonen klarte å håndtere kravet om 600 sensormålinger per time ble prosjektets egen *collection-simulator* brukt. Collection-simulatoren ble kjørt med tre ulike konfigurasjoner for å kartlegge systemets kapasitet utover kravet:

Scenario	Enheter	Intervall	Meldinger/time	Faktor	Tapte
A - Min krav	10	60 s	600	1x	0
B - 5x krav	50	60 s	3 000	5x	0
C - Stressgrense	60	6 s	36 000	60x	0

Table 11: Resultater for sensormeldinger

Ingen av sensormeldingene gikk tapt i noen av scenarioene. Go-rutinene i Collection-service med ti arbeidere og en bufferstørrelse på 1000 meldinger per arbeider ble ikke mettet, selv ved 60x kravet. Scenario C tilsvarer i realiteten et aktivt system med ca. 3 000 sensorer, noe gruppen anser som svært tilfredsstillende gitt at systemet i dag opererer med under 10 sensorer.

Test 2 - HTTP-belastningstesting

For å teste svartid og håndtering av samtidige brukere ble k6 satt opp med virtuelle brukere som utfører realistiske forespørsler mot backend i parallell. Testen dekket

endepunktene `me`, `sensors`, `latest reading`, `production-resources` og `orders`. Mellom hver handling ble det lagt inn en pausetid på 1–4 sekunder. Brukerne ble trappet gradvis opp til kravet på 300 samtidige brukere, holdt der i to minutter, før de ble trappet ned igjen.

Fase	Varighet	Brukere
Opptrapping	1 min	0 – 300
Hold	2 min	300
Nedtrapping	30 sek	300 – 0

Table 12: Virtuelle brukere i HTTP-belastningstest

Testen tok totalt ca. 3,5 minutter og gjennomførte 92 984 HTTP-forespørsler.

Svartid ble målt med percentiler fremfor gjennomsnitt, ettersom gjennomsnittet kan være misvisende for slike tester. Et lite antall ekstremt trege forespørsler kan trekke gjennomsnittet opp uten at det reflekterer den typiske brukeropplevelsen [33]. p95 innebærer at 95 % av forespørslene ble besvart raskere enn den oppgitte verdien, og er et hensiktsmessig mål for et svartidskrav ettersom det sikrer at den store majoriteten av brukere opplever god ytelse.

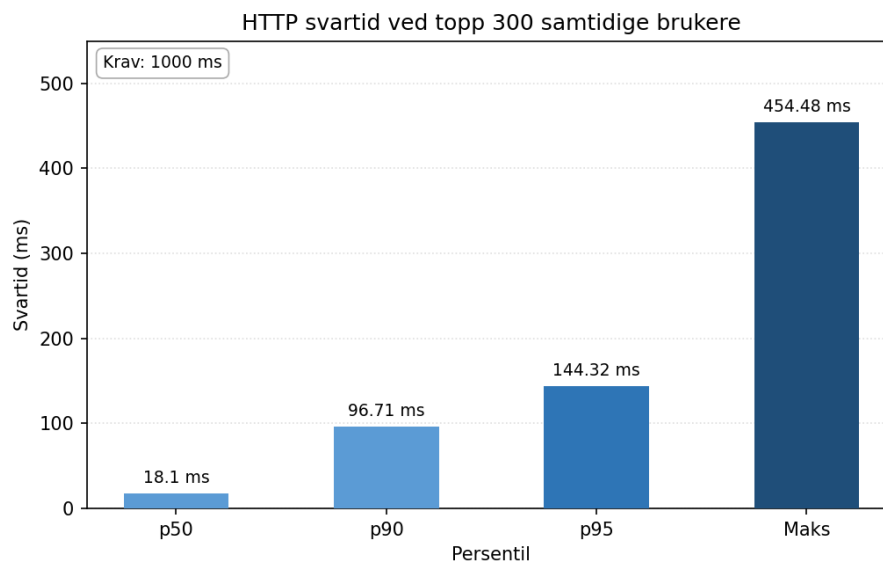


Figure 27: HTTP svartid ved 300 samtidige brukere (produksjonsserver)

Metrikk	Verdi	Krav
Gjennomsnittlig svartid	37,07 ms	-
p50 svartid	18,10 ms	-
p90 svartid	96,71 ms	-
p95 svartid	144,32 ms	< 1 000 ms
Maks svartid	454,48 ms	-
Feilrate	0,00	-

Table 13: HTTP svartid ved 300 samtidige brukere (produksjonsserver)

Samtlige endepunkter returnerte HTTP 200 for 100 % av forespørslene gjennom hele testen. p95-svartiden på 144 ms er nærmere sju ganger lavere enn 1-sekundskravet, og feilraten var 0 %. Gruppen tolker resultatene som svært positive.

Brukertesting

Etter endt sprint 3 ble den første alfa brukertesten gjennomført som en del av sprint review-møtet med oppdragsgiver. På dette tidspunktet besto produktet av et fungerende dashboard med backend-logikk for å legge til gateway og sensor, med tilnærmet umiddelbar statusfeedback. For å simulere et realistisk scenario ble en fysisk gateway og sensor med tilhørende varmeovn tatt med til møtet, slik at oppdragsgiver kunne teste applikasjonen under forhold som lignet en reell installasjon på en ny fabrikk.

Brukertesten var strukturert rundt fire konkrete oppgaver: legge til company, factory, gateway og sensor. Oppdragsgiver gjennomførte samtlige oppgaver uten veiledning og navigerte til riktige knapper og tekstfelter i samtlige tilfeller. Tilbakemeldingene underveis var gjennomgående positive — responstid ble trukket frem som særlig imponerende, og inputfeltene ble oppfattet som intuitive. Et forbedringspunkt som ble avdekket var en for lang profilliste ved registrering av sensor, noe som ble rettet i etterkant av møtet.

Under testingen oppsto det en utfordring knyttet til oppkobling av sensor. Sensoren hadde ikke blitt tilbakestilt til fabrikkinnstillinger, og forsøkte derfor å koble seg til den sist tilknyttede gatewayen fremfor den nye. Dette var en feil i forhåndsoppsettet av enheten, noe som ble bekreftet da sensoren koblet seg opp som forventet etter tilbakestilling — kort tid etter at møtet var avsluttet.

Ut fra denne testen kunne gruppen konkludere med at brukervennligheten er av god kvalitet. Oppdragsgiveren forstod oppgaven og klarte å bruke systemet uten feil. Det må nevnes at dette ikke er en fullverdig test av brukervennlighet, ettersom

oppdragsgiveren allerede var kjent med produktet under utvikling, og systemet er likt det Innoveria allerede hadde. Nyttigheten derimot fikk en bra test og viste til at systemet fungerte for oppgavene det er utviklet for. Brukergrensesnittets følelse ble vist til å være bra, men er ikke like relevant for oppgaven siden frontend er som nevnt ikke hovedfokuset. En av de viktigste lærdommene som kom fra denne testen var en god responstid, knyttet til det operasjonelle kravet om responstid på under 1 sekund.

På grunn av manglende funksjonalitet i bacheloroppgavens natur, ble det heller ikke utført noen beta tester. Gruppen fikk derfor ikke testet applikasjonen på andre operativmiljøer enn sine egne. Dette anbefales å gjøre under videre utviklingen, når systemet er nærmere et ferdig produkt.

9 Diskusjon

9.1 Emneintegrasjon

Under gjennomføringen av prosjektet har gruppen trukket inn kunnskap fra flere emner tilhørende studieløpet. Enkelte av fagene har vært svært avgjørende for gjennomføringen, mens andre har vært mindre relevante, men likevel fungert som et godt utgangspunkt for videre tilegnelse av kunnskap. Emnene som er nevnt under, er de gruppen mener har veid tyngst i prosjektet.

Emner fra studieløpet

PROG2005 - Cloud Technologies er nok faget som har spilt den største rollen i prosjektet. Emnet ga grunnleggende ferdigheter i programmeringsspråket Go og erfaring med å bygge API-er. Dette er selve kjernen i mikroservicearkitekturen vi har utviklet i prosjektet. Uten erfaringen gruppen fikk fra dette faget, ville implementasjonen av de forskjellige tjenestene i backend vært vesentlig mer krevende.

IDATG2204 - Datamodellering og databasesystemer har vært et sentralt fag som utgangspunkt for utformingen av datamodellen. Faget ga oss innsikt i konseptuell modellering, relasjonsdatabaser og SQL. Dette gjorde at gruppen kunne starte raskt med å designe databasestrukturen. I tillegg har vi bygget videre på denne kunnskapen da vi valgte å bruke PostgreSQL, ettersom disse overlapper i stor grad. Gruppen har også gjennom implementering og vedlikehold av systemet hatt god nytte av SQL-spørringer, som vi lærte i faget.

PROG2007 - Mobile Programming introduserte gruppen for komponentbasert UI-utvikling gjennom Jetpack Compose²⁵, og la grunnlaget for hvordan vi tenker rundt oppbygging av brukergrensesnitt. Denne kompetansen ble videreutviklet i *PROG2052 - Integrasjonsprosjekt*, der gruppen utviklet en fullstack Android-applikasjon med Jetpack Compose som frontend og Go som backend. Prinsippene fra disse fagene var direkte overførbare til bachelorprosjektet, der gruppen benyttet React som frontend-verktøy. React og Jetpack Compose deler mange av de samme komponentbaserte prinsippene, og erfaringen med å koble en frontendløsning mot et dedikert API via Axios bygget videre på kompetansen etablert i Integrasjonsprosjektet.

Andre fagområder

Et svært sentralt fagområde for prosjektet, som ikke er dekket av studieløpet, er LoRaWAN-teknologi. Gruppen har måttet tilegne seg mye kunnskap om sensorer,

²⁵<https://developer.android.com/compose>

Gateway, MAC-versjoner, regionale frekvenser, protokoller m.m. selvstendig gjennom prosjektperioden. Dette har gruppen gjort gjennom å lese teknisk dokumentasjon, samt gjennom omfattende eksperimentering og utprøving. Denne delen av kunnskapsinnsamlingen representerer en stor del av den faglige innsatsen i prosjektet, og var mer komplisert enn gruppen hadde sett for seg innledningsvis. Dette synliggjorde også at IIoT-domenet krever betydelig kompetanse utover det mer tradisjonelle programmeringspensumet gruppen fikk gjennom studieløpet.

9.2 Bærekraftsperspektiv

For å vurdere bærekraftsperspektivet til prosjektet har gruppen valgt å bruke Sustainability Awareness Framework (SuSAF) som grunnlag. SuSAF brukes til å analysere programvaresystemer langs fem dimensjoner: teknisk, miljømessig, økonomisk, sosial og individuell [34]. I gruppens vurdering av dette prosjektet har vi valgt å fokusere på de dimensjonene som er mest aktuelle.

Teknisk bærekraft

Gruppen mener systemet scorer godt innenfor teknisk bærekraft. Mikroservicearkitekturen som prosjektet ble bygget rundt, gjør at enkelttjenestene kan vedlikeholdes, oppdateres og skaleres mer eller mindre uavhengig av hverandre. Systemet er derfor enkelt å utvide, og nye sensortyper og tjenester kan integreres uten at kjernen i programvaren må endres.

Gjenbruk og teknologivalg

I løpet av prosjektet skrev gruppen store deler av systemet fra bunnen av. Dette har gitt full kontroll over kodebasen og unngått avhengighet til tredjepartsløsninger. Dette førte imidlertid også med seg en ulempe. Utviklingen gikk saktere enn nødvendig, og gruppen brukte mye ressurser på å løse problemer som allerede er løst. Innsamling og konfigurering av sensorer viste seg å være en mer krevende oppgave enn gruppen hadde sett for seg. Eksisterende plattformer som The Things Stack²⁶ tilbyr nettopp dette og kunne med fordel blitt bygget videre på i stedet for å starte fra bunnen av. I et bærekraftsperspektiv kan det være bortkastede ressurser å gjenskape noe som allerede finnes.

Miljø, klima og økonomi

Et av de sterkeste bærekraftsargumentene for applikasjonen er ikke selve programvaren, men heller hva den muliggjør. Applikasjonen som gruppen utviklet, kan gi

²⁶<https://www.thethingsindustries.com/stack/>

industrien kontinuerlig innsikt i forbruk av strøm og andre ressurser, samt sette dette i kontekst med produksjonen. Dette gir grunnlag for å identifisere og eliminere svinn i produksjonen. Redusert energiforbruk, eller redusert forbruk av andre ressurser, vil kunne gi lavere driftsutgifter for bedriftene og lavere klimautslipp fra produksjonen. Begge deler er viktige gevinster for brukere av systemet og for miljøet. Dette er i tillegg direkte relevant for karbonregnskap slik det er definert i ISO 14067 [5], som spesifiserer krav til beregning og videreformidling av produktenes karbonavtrykk gjennom hele prosessen. Overvåkingsdata fra sensorene, satt sammen med produksjonsdata fra ERP-systemer, vil bli selve grunnlaget for denne beregningen.

På den andre siden har utviklingen av applikasjonen hatt en negativ innvirkning på miljø og klima. Gruppen har brukt KI-verktøy aktivt i utviklingen. Store språkmodeller som Claude²⁷ og ChatGPT²⁸ har et relativt høyt energiforbruk per forespørsel, og gruppen erkjenner at denne bruken kan ha en miljøkostnad som ikke er ubetydelig.

FNs bærekraftsmål

Av FNs 17 bærekraftsmål [35] knytter prosjektet seg naturlig til to av dem:

Mål 12 - Ansvarlig forbruk og produksjon

Ved å redusere forbruk og identifisere ressurstyver gjennom applikasjonen bidrar den til å nå dette målet ved å gjøre det mulig for selskaper å oppnå mer med mindre ressurser, samt stimulere dem til å innføre bærekraftige arbeidsmetoder.

Mål 13 - Klimatiltak

Redusert energi- og ressursforbruk i industrien vil kunne bidra til lavere klimagassutslipp. I kombinasjon med ISO 14067 kan applikasjonen understøtte selskapers arbeid med å kartlegge og redusere sitt klimaavtrykk.

²⁷<https://claude.com/product/overview>

²⁸<https://openai.com/nb-NO/index/chatgpt/>

9.3 Refleksjon

9.3.1 Resultat

Innoveria ønsket en webbasert løsning for administrasjon av LoRaWAN-enheter, innsamling av sensordata og kobling mellom sensormålinger og produksjonsdata. Resultatet ble en velfungerende løsning som oppfyller store deler av prosjektbeskrivelsen i 1.3. Både gruppen og Innoveria var særlig fornøyde med håndteringen av LoRaWAN-enheter, som fremstod som en av de sterkeste delene av sluttproduktet. Onboardingflyten for sensorer og Gateway gjorde det mulig å registrere, konfigurere og ta i bruk nye enheter på én plattform. Innoveria uttrykte spesielt tilfredshet med denne delen av løsningen, ettersom den representerte en betydelig forbedring sammenlignet med den tidligere løsningen beskrevet i 5.1.

Løsningen gjør det mulig å motta sensordata fra LoRaWAN-baserte sensorer og ERP-data, samt å bruke disse videre i systemet for visualisering gjennom det gruppen omtaler som kontekst. Gruppen er også tilfreds med hvor robust systemet ble. Store deler av den planlagte mikrotjenestearkitekturen ble implementert, inkludert separate tjenester for håndtering av sensordata, innhenting av ERP-data og kontekstualisering av sensor- og ERP-data.

Integrasjonen mot Monitor ERP viste seg imidlertid å være mer krevende enn LoRaWAN-integrasjonen. Siden nye produksjonsbedrifter er avhengige av å installere en ERP Agent Service i sitt nettverksmiljø, kan dette gjøre løsningen mer utfordrende å skalere over tid. Dette er samtidig en konsekvens av at mange ERP-systemer driftes som *on-premise*-løsninger med ulike sikkerhetskrav og integrasjon utfordringer. Før løsningen kan tas i bruk i større skala, vil det derfor være nødvendig å videreutvikle ERP-integrasjonen med bedre mekanismer for sikkerhet og skalerbarhet.

9.3.2 Læringsutbytte

Mikroservicearkitektur

Valget om å strukturere systemet som en samling mikrotjenester ga gruppen verdifull innsikt i både fordelene og kompleksiteten ved denne arkitekturen. Et av de mest tydelige utbyttene var opplevelsen av god *separation of concern*. Hver tjeneste hadde et avgrenset ansvarsområde, noe som gjorde kodebasen lettere å forstå og vedlikeholde isolert sett.

Et bevisst valg gruppen tok var å samle alle tjenestene i et monorepo. Dette foren-

klet deling av konfigurasjon som ellers måtte ha ligget i hvert repository dersom tjenestene hadde vært i separate repoer. Verktøyet *Tilt* bidro ytterligere til en god utviklingsopplevelse ved å aggregere logger fra samtlige tjenester på ett sted, noe som gjorde feilsøking langt mer oversiktlig og tilgjengelig.

Gruppen så likevel at arkitekturen bød på utfordringer. Arkitekturen medførte en del *boilerplate*-kode, da mye av oppsettet til tjenestene var nokså likt. Feilhåndtering, logging og lasting av konfigurasjon måtte i prinsippet gjentas i hver enkelt tjeneste.

Monorepo-tilnærmingen er strengt tatt ikke helt i tråd med idealet om løs kobling som mikrotjenestearkitektur bygger på. Ved å samle all kode i ett repository innføres en grad av avhengighet, selv om den tekniske koblingen mellom tjenestene forblir lav. Boilerplate-utfordringen ble delvis bedret gjennom en felles `pkg`-pakke som tjenestene kunne importere delt funksjonalitet fra. For eksempel var `pkg/logger` en felles `NewLogger`-funksjon for å konfigurere strukturert logging via Go sitt innebygde `slog`-bibliotek. Uten denne pakken ville hver enkelt tjeneste måtte sette opp loggformat, loggnivå og tjenesteattributter på egen hånd. Denne løsningen reduserte dupliseringen av kode betraktelig, men representerer samtidig en form for delt avhengighet som strider mot mikroservicearkitekturen [36].

Erfaringer med teamarbeid

Prosjektet ga gruppen verdifull erfaring med tett samarbeid over en lengre utviklingsperiode. Et sentralt læringspunkt var verdien av presis kommunikasjon og tydelig arbeidsfordeling. Ved enkelte anledninger oppstod det situasjoner der to utviklere arbeidet på samme issue samtidig som følge av at SCRUM-boardet ikke reflekterte at arbeidet allerede var påbegynt. Det ble også erfart at sammenslåing av issue ikke alltid ble tilstrekkelig kommunisert i Scrum-boardet, noe som skapte uklarhet rundt ansvarsfordelingen. Disse erfaringene understreker viktigheten av å behandle Scrum-boardet som en entydig, og eneste, kilde til sannhet for arbeidsfordeling i teamet.

De fysiske møtene viste seg å være verdifulle for å løse arkitekturmessige og domenerelevante problemstillinger. På de fleste rommene gruppen benyttet til møter, var det whiteboards som aktivt ble brukt til å visualisere systemoversikt og diskutere løsningsalternativer. Tavlebruken bidro i stor grad til gruppens felles forståelse, ettersom gruppe medlemmene kunne diskutere, peke og tegne alternativer som underbygget de faglige diskusjonene i møtene. Et konkret eksempel på dette var arbeidet med context service, der det var behov for å avklare hvilke datakilder tjenesten skulle aggregere informasjon fra. Slike avklaringer lot seg gjøre effektivt i fellesskap under

de fysiske møtene, mens de resterende dagene ble benyttet til implementasjon av avtalt funksjonalitet. Denne arbeidsrytmen opplevdes som hensiktsmessig og bidro til at gruppen opprettholdt god fremdrift gjennom hele utviklingsperioden.

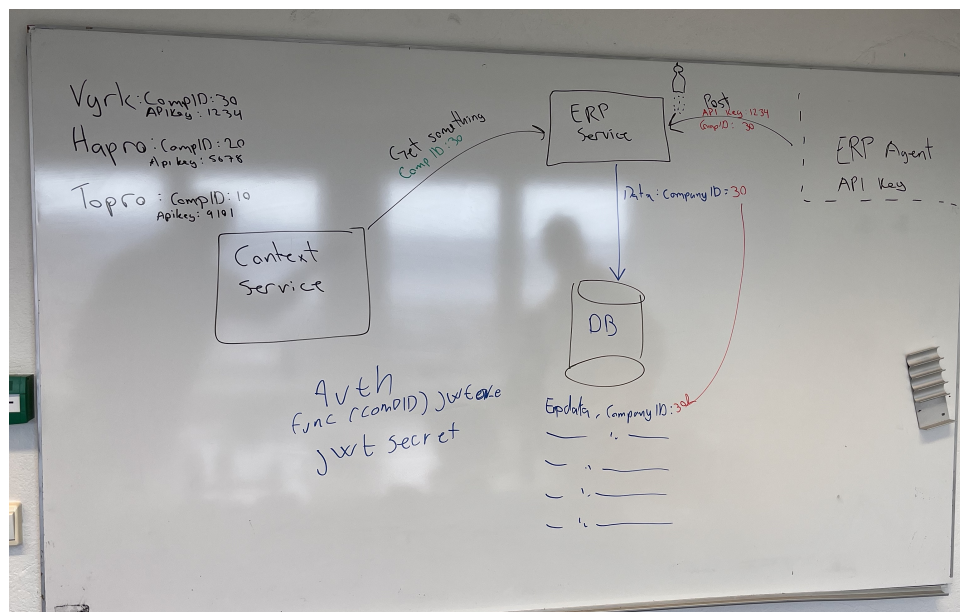


Figure 28: Tavlebruk i fysisk møte

Forholde seg til oppdragsgiver

For alle i gruppen var dette det første prosjektet der man måtte forholde seg til en ekstern oppdragsgiver. Erfaringen ble ansett som både lærerik og givende på flere plan.

Et gjennomgående læringspunkt var utfordringen med å kommunisere teknisk kompleksitet til oppdragsgiver, som ikke hadde bakgrunn innen programvareutvikling. Det å formidle mengden arbeid som inngår i en sprint, for eksempel hvorfor integrasjonen mot ChirpStack krever den tiden den gjør, viste seg å være krevende. Når en ferdig feature ble presentert uten denne konteksten, risikerte man at omfanget av det underliggende arbeidet ikke kom tydelig frem. I etterkant av prosjektet er det naturlig å spørre seg om sprint-reviewene burde vært strukturert annerledes - ikke bare som en demonstrasjon, men også med et innslag av prosessen bak: hvilke utfordringer som ble møtt, hvilke avveininger som ble gjort og hvorfor løsningen ser ut som den gjør. Hvorvidt dette er hensiktsmessig innenfor rammene av et sprint-review er ikke opplagt, men gruppen mener det kunne bidratt til mer realistisk forventningsstyring for prosjektet.

Oppdragsgiver fremstod gjennom hele prosjektet som en engasjert og entusiastisk støttespiller, med tydelig interesse for det gruppen presenterte. Samtidig bar samarbeidet preg av at oppdragsgiver er en iderik person med mange ønsker og

ambisjoner for produktet. Oppdragsgiver var selv tydelig på at gruppen måtte sette grenser dersom arbeidsmengden ble for stor, men i samme åndedrag kom det gjerne nye ideer til overflaten. Det var med andre ord gruppens ansvar å sette på bremsen dersom ønsker og ambisjoner ikke ville vært mulige å oppnå. Et konkret eksempel på dette var nedprioriteringen av tilkobling mot Monitor ERP via agentløsningen, der oppsettstiden ble vurdert som uforholdsmessig høy sett opp mot gevinsten i bachelorprosjektet, gitt prosjektets begrensede tidsrammer og ressurser. Gruppen anser denne erfaringen som svært verdifull med tanke på forventningstyring i et reelt prosjektforhold.

Undervurdering av domene og integrering av ERP

I den innledende fasen med research ble ikke hele domenet, spesielt veien mot en eventuell abstrahering av sensor og gateway, utforsket i tilstrekkelig grad. Det viste seg at dette problemområdet rommet langt flere nyanser enn først antatt. Blant annet medførte leverandørspesifikke konfigurasjoner og protokolldetaljer at gruppen brukte betydelig mer tid på flere iterasjoner av abstraksjonsimplementeringen, ettersom kunnskapen om domenet gradvis utviklet seg underveis i prosjektet. Med etterpåklokskapens lys ville en grundigere domeneanalyse, med mer tid avsatt til research, trolig spart tid i selve implementasjonen.

Tilsvarende ble kompleksiteten knyttet til integrasjon av ERP-systemer undervurdert. Gruppen hadde innledningsvis antatt at tilgang til data fra Monitor ERP ville løses gjennom et enkelt API-kall med tilhørende nøkkel. Det viste seg imidlertid at *on-premise*-løsninger er utbredt i industrien, noe som innebærer at data ikke er tilgjengelig utenfor fabrikkens lukkede nettverk. Dette medførte at gruppen måtte bruke en til to uker ekstra på å utforme og implementere en løsning tilpasset slike omgivelser. Løsningen som ble utviklet var en agenttjeneste som kjøres innenfor det lukkede nettverket og videresender data til Fabrikkpuls, slik at systemet kan hente reelle data fra et *on-premise*-ERP-system. Sikkerhetsperspektivet ved denne tilnærmingen er nærmere omtalt i kapittel 4.4.

Grunnet tidsbegrensninger i prosjektperioden ble det ikke gjennomført reell testing av ERP-agent-løsningen mot et faktisk *on-premise*-system. Gjennom utviklingsperioden ble det i stedet benyttet syntetiske testdata generert av en innebygd mock-klient i agenttjenesten, slik at dataflyten kunne valideres uten tilgang til et reelt *on-premise*-system.

9.3.3 Timebruk

Fordeling mellom temaer

Fordeling	Sum	Prosent
Research	288,5	13,0%
Møte	440,75	19,8%
Utvikling	1161	52,2%
Skriving	304,5	13,7%
Testing	30,5	1,4%

Figure 29: Prosentandelvis fordeling av tid mellom temaer

Som forventet ble mesteparten av tiden brukt på utvikling. Denne tiden besto primært av utvikling av funksjonaliteter, men det ble også brukt mye tid på MR-er. Siden gruppen organiserte arbeidet slik at hver MR måtte gjennom en grundig gjennomgang (mer detaljer i kapittel 2), fikk all utvikling en kvalitetssikring som bidro til at problemer ble oppdaget før det var for seint. I senere prosjekter burde denne arbeidsmetoden videreføres.

Noe som ikke vises på timeloggen, er spesialiseringen av tidsbruken. Med unntak av én person har gruppen primært jobbet enten med backend eller frontend, bortsett fra en kort periode mot slutten av prosjektet. Dette har ført til at læringsutbyttet har vært svært spredt, og at enkelte går ut av prosjektet med mindre kunnskap om spesifikke deler av systemet enn de burde hatt, som for eksempel databasestrukturen eller generell TypeScript-basert frontendutvikling.

Møter utgjorde 19,8% av tidsbruken, og skyldtes regelmessige møter. Dette er i hovedsak tid som ikke kunne blitt satt av til andre oppgaver av hensyn til struktur. Mengden møter med gruppen kan diskuteres, men det føltes hensiktsmessig med to møter i uken. Det å få en oppdatering gjennom daily standup hjalp betydelig med å holde oversikt over fullført arbeid, samtidig som det ikke ville vært nok å snakke om dersom gruppen møttes hver arbeidsdag. Ved en lignende oppgave ville samme mengde møter blitt opprettholdt, men dette er generelt svært avhengig av oppgavens karakter og antall gruppemedlemmer.

Forskning var nødvendig for utviklingsarbeidet. En andel av forskningen ble brukt på implementasjoner som ikke kom frem i sluttproduktet. I ettertid hadde det vært bedre å fokusere kun på det gruppen hadde tid til, men realistisk sett er det umulig å vite hva man rekker.

Det som skiller seg mest ut, er at testing bare utgjorde 1,4% av tidsbruken. Mer tid burde blitt brukt på testing, spesielt tidligere i prosjektet. Tallet gir ikke et

fullstendig bilde, siden all testing under utvikling for å forsikre at implementasjonene fungerte som de skulle, er kategorisert som utvikling. Likevel burde mer tid vært satt av til testing eksklusivt, særlig brukertesting.

Utfordringer

Gruppen hadde erfaring med tilsvarende timelogging fra Integrasjonsprosjektet. Den gangen var det ikke satt like strenge forventninger knyttet til å oppnå timekravet. I dette prosjektet derimot var motivasjonen for å oppnå timene mye sterkere, ikke bare for å oppfylle kravene, men også for å sikre at nødvendig arbeid faktisk ble gjennomført. Dette kunne til tider oppleves som stressende, ettersom fokuset i enkelte perioder gikk fra hvilke oppgaver som skulle utføres til hvor mange timer som måtte registreres. Det oppsto tanker om at man måtte arbeide til et viss tidspunkt for å nå målet. Mengden tidsbruk var nødvendig, da prosjektet ikke kunne blitt gjennomført med mindre innsats. Selve loggingen la til et ekstra lag med stress.

Ut fra timeloggen kan man se at tidsfordelingen mellom gruppemedlemmene varierer betydelig. Dette skyldes at alle logger timer ulikt, basert på egne vurderinger av hva som skal kategoriseres hvor. Dette er tydeligst ved møter. For et slikt prosjekt er forskjeller i loggføring ikke spesielt problematiske.

Totalt sett har timebruken vært god. Alle holdt seg til arbeidskravene, møtene bidro til god struktur, og MR-er har i høy grad vært nødvendige. I ettertid ville det vært bedre å fordele ulike typer oppgaver i større grad slik at alle i det minste fikk en forståelse av hele prosjektet, samt starte med testing tidlig og gjennomføre dette kontinuerlig gjennom prosjektperioden.

9.3.4 Kunstig intelligens som verktøy

Bruken av KI gjennom prosjektet ga gruppen erfaringer som strekker seg utover det rent tidsbesparende. KI fungerte godt som en bred kunnskapsbase som gjorde det mulig å orientere seg raskt i ukjente teknologier og domener, noe som var særlig verdifullt gitt omfanget av prosjektet og den begrensede tidsrammen en bacheloroppgave setter. Det er gruppens vurdering at prosjektets omfang i liten grad ville vært gjennomførbart innenfor disse rammene uten aktiv bruk av KI. Dette handler ikke utelukkende om hastighet, men om at KI lot gruppen rette oppmerksomheten mot arkitektur og løsningsdesign fremfor syntaks og tekniske detaljer i selve implementasjonene.

Samtidig er det viktig å nyansere dette bildet. KI er for det meste trent på offentlig tilgjengelig kode [37, avsnitt 1.1.1, s. 8], noe som innebærer at store, proprietære og

moderne produksjonssystemer i liten grad er representert i treningsdataen. Dette betyr i praksis at løsningsforslagene fra KI ikke nødvendigvis reflekterer gjeldende standarder som benyttes i profesjonelle og robuste systemer. Gruppen erfarte at KI tidvis genererte forslag til løsninger som teknisk sett fungerte, men som ikke nødvendigvis var de mest hensiktsmessige på sikt.

En tilleggsdimensjon ved KI-assistert utvikling er at kodebasen kan vokse raskere enn utviklernes forståelse av den. Når generert kode fungerer som forventet, er terskelen for å akseptere den noe lavere, selv om de underliggende mekanismene ikke er fullt ut forstått. Over tid kan dette svekke eierskapet til koden, noe gruppen anser som problematisk både for motivasjon og for evnen til å videreutvikle og feilsøke systemet. Et konkret eksempel som illustrerer dette tveeggende aspektet, er en TCP-tilkoblingsfeil i Go-klienten som ble avdekket av KI under en MR-gjennomgang. Feilen burde ideelt sett blitt oppdaget av gruppen selv, men ble fanget opp av KI da gruppen ikke var klar over at dette var en potensiell feilkilde. Eksemplet viser at KI kan fungere som et sikkerhetsnett, men understreker samtidig at gruppen ikke hadde full oversikt over samtlige deler av kodebasen.

Spørsmålet om i hvilken grad man bør stole på og akseptere KI-generert kode, er ikke trivielt. Gruppen mener at en kritisk holdning er avgjørende, ikke bare til om KI kan generere fungerende kode, men også om koden er forståelig, vedlikeholdbar og i tråd med systemets overordnede design.

Samkjøring av KI-verktøy

En utfordring som ble tydelig mot slutfasen av prosjektet var fraværet av felles retningslinjer for KI-bruk på tvers av teamet. Gruppen benyttet ulike KI-verktøy, (Claude Code, ChatGPT, OpenCode) uten en felles konfigurasjon eller et felles sett med instruksjoner. Dette medførte at KI-verktøyene fikk ulik kontekst om prosjektet, og dermed kunne tilnærmingen variere mellom utviklerne.

Gruppen hadde en `docs`-mappe i repoet som dokumenterte blant annet FSD-strukturen i frontend og andre sentrale systemvalg. Denne var i utgangspunktet ment for menneskelige lesere, og ble ikke konsekvent matet inn i KI-verktøyenes kontekst. I ettertid er det tydelig at denne dokumentasjonen ville vært godt egnet som grunnlag for en felles systemkontekstdatabase for KI-verktøy med tilgang til repoets filer.

Verktøy som Claude Code og OpenCode støtter prosjektspesifikke konfigurasjonsfiler (`claude.md` og `agents.md`), hvor man kan definere kontekst, konvensjoner og retningslinjer for hvordan KI-en skal utføre oppgaver i prosjektet. I dette prosjektet ble ingen av disse filene pushet til repoet, noe som innebar at selv utviklerne som brukte samme verktøy, for eksempel Claude Code, ikke var samkjørte. En slik overordnet

fil burde i retrospekt vært sentralisert i repoet og inneholdt referanser til overordnet systemdokumentasjon, som for eksempel `docs`-mappen i dette prosjektets tilfelle.

For verktøy uten støtte for konfigurasjonsfiler, som ChatGPT brukt via nettleser, kunne en tilsvarende effekt blitt oppnådd ved å utarbeide en felles instruksjonstekst som ble limt inn ved oppstart av hver samtale. Dette er ikke en elegant løsning, men illustrerer at samkjøring av KI-bruk ikke nødvendigvis krever avansert verktøystøtte - det krever først og fremst bevisst koordinering i teamet.

Det er samtidig verdt å reflektere rundt de praktiske rammene for et bachelor-prosjekt. Full samkjøring forutsetter at alle teammedlemmer bruker de samme verktøyene, noe som i praksis kan innebære kostnader. OpenCode er gratis å installere, men bruk av underliggende modeller er ikke alltid kostnadsfritt. Å pålegge studenter betalte løsninger i et akademisk prosjekt er verken rimelig eller hensiktsmessig. Dette er en reell begrensning som ikke er unik for dette prosjektet, men som vil gjelde for de fleste team der KI-verktøy brukes ulikt på tvers av medlemmene.

Kritisk bruk av KI-genererte forslag

Et gjennomgående læringspunkt ved bruk av KI i utviklingen var viktigheten av å opprettholde en kritisk holdning til genererte forslag. KI-verktøy opererer utelukkende på bakgrunn av det de eksplisitt presenteres for: de har ikke deltatt i interne diskusjoner, kjenner ikke til avveininger rundt arkitektur som er gjort underveis, og har ingen kjennskap til planlagt fremtidig funksjonalitet. Dette var særlig synlig ved bruk av KI under MR-gjennomganger, hvor forslagene ofte var teknisk velfunderte isolert sett, men ikke nødvendigvis den mest hensiktsmessige løsningen gitt prosjektets bredere kontekst. Erfaringen understreker at KI egner seg godt som et teknisk supplement, men at menneskelig skjønn forblir avgjørende for å vurdere forslagenes relevans i en større sammenheng.

Et relatert spørsmål er når KI er det rette verktøyet, og når andre tilnærminger ville vært mer hensiktsmessige. I dette prosjektet ble KI blant annet brukt til å generere boilerplate-kode ved oppsett av nye mikrotjenester. I et lengre prosjektforløp, der de samme mønstrene gjentas over tid, ville det trolig vært mer hensiktsmessig å etablere ferdige templates for mikroservicearkitekturen, da templates gir mer forutsigbarhet, er enklere å vedlikeholde og ikke krever at utvikleren vurderer KI-output kritisk ved hver iterasjon. Dette er ikke en svakhet ved KI som verktøy, men snarere en påminnelse om at valg av verktøy bør tilpasses prosjektets varighet og kontekst, i stedet for å lene seg fullt og helt på KI.

9.4 Forslag til videre utvikling

Gruppen er som tidligere nevnt godt fornøyd med resultatet, men flere områder peker seg ut for videre utvikling dersom systemet skal videreutvikles:

ERP Agent Service er ikke distribuert i et Monitor ERP-nettverksmiljø. Dataene som kommer inn i løsningen er mock-data, noe som ble valgt for at gruppen skulle kunne fokusere på systemet fremfor å sende bedriftsdata over internett. Før tjenesten tas i bruk må det implementeres nødvendige sikkerhetsmekanismer for håndtering av ERP-data.

Fabrikkområder er delvis implementert i databaseskjemaet, men sensorer og Gateway er ennå ikke fullt koblet til disse. En fullstendig implementering av dette systemet vil gi en bedre visuell oversikt over sensorer og gateways, spesielt i større fabrikker.

Rollebasert tilgang er grunnleggende implementert i applikasjonen med et skille mellom administrator og bruker, men rollen *FactorySuperUser* per fabrikk er ikke implementert slik den var tiltenkt innledningsvis i prosjektet. En slik rolle bør implementeres for å begrense hvilke brukere på fabrikk som har tilgang til å administrere sensorer og gateways. I tillegg kan denne rollen brukes til å håndtere egne brukere innenfor fabrikk.

Offboarding mangler i sin helhet. Per nå finnes det ingen måte for administrator å fjerne selskaper på uten å manuelt gå inn i databasen. Brukere, sensorer og gateways kan slettes gjennom frontend, men det bør implementeres en offboardingmekanisme som sletter hele selskapet med tilhørende data og målinger når et selskap ikke lenger ønsker å være kunde av plattformen. GDPR stiller krav om at personopplysninger tilknyttet selskapet, slik som brukerdata, skal kunne slettes uten ugrunnet opphold [38], noe som understreker behovet for en slik mekanisme.

Sikkerhetskopi av database er foreløpig implementert med et script som kan kjøres manuelt og lagrer en kopi på disk. I et produksjonsmiljø bør scriptet settes opp til å kjøre som en cron-jobb for å automatisere sikkerhetskopieringen. I tillegg bør det vurderes å lagre kopien utenfor servermiljøet for å sikre seg mot feil på disken.

Kontinuerlig utrulling (CD) er ikke implementert. Per nå skjer deployering manuelt. En naturlig utvidelse av den eksisterende CI-pipelinen er å legge til et deploy-steg som automatisk oppdaterer produksjonsmiljøet når kode blir merget til hovedgrenen, *main*.

10 Kilder

- [1] Ian Sommerville. *Engineering Software Products – Testing*. Pearson Education, 2021.
- [2] Craig Douglas. *What is a High-Fidelity Wireframe and When to Use It?* URL: <https://www.visily.ai/blog/what-is-a-high-fidelity-wireframe> (visited on 6th May 2026).
- [3] Innoveria. *About*. URL: <https://innoveria.no/about/> (visited on 20th Jan. 2026).
- [4] Prosess21. *Digitalisering i prosessindustrien 2025: Hovedrapport*. Se spesielt s. 9 for omtale av data i siloer og utfordringer med integrasjon og sporbarhet. URL: <https://www.prosess21.no/siteassets/ekspertgrupper/nye-rapporter/250318-digitalisering-i-prosessindustrien-2025-hovedrapport.pdf> (visited on 20th Jan. 2026).
- [5] International Organization for Standardization. *ISO 14067: Carbon footprint of products*. International standard ISO 14067. International Organization for Standardization, 2018. URL: <https://www.ditan.com/static/upload/file/20240522/1716345585128585.pdf> (visited on 30th Jan. 2026).
- [6] The-Things-Network. *What are LoRa and LoRaWAN?* URL: <https://www.thethingsnetwork.org/docs/lorawan/what-is-lorawan/> (visited on 23rd Jan. 2026).
- [7] Torgeir Dingsøy. *Smidig utviklingsmetoder*. 2025. URL: https://snl.no/smidige_utviklingsmetoder (visited on 15th May 2026).
- [8] Theuri Karue. *The Art of Writing Meaningful Git Commit Messages*. URL: <https://medium.com/@iambonitheuri/the-art-of-writing-meaningful-git-commit-messages-a56887a4cb49> (visited on 5th Jan. 2026).
- [9] Anomaly Innovations Inc. *OpenCode*. URL: <https://opencode.ai/> (visited on 6th Apr. 2026).
- [10] Anthropic. *Claude Code overview*. URL: <https://code.claude.com/docs/en/overview> (visited on 6th Apr. 2026).
- [11] Jakob Nielsen. *Response Times: The 3 Important Limits*. URL: <https://www.nngroup.com/articles/response-times-3-important-limits/> (visited on 5th Apr. 2026).
- [12] Google. *GoLang vs. Node.js: Which One to Choose for 2025*. URL: <https://www.geeksforgeeks.org/blogs/golang-vs-node-js/> (visited on 6th Apr. 2026).

-
- [13] Matt Carroll. *The React Foundation: A New Home for React Hosted by the Linux Foundation*. URL: <https://react.dev/blog/2026/02/24/the-react-foundation> (visited on 9th Apr. 2026).
 - [14] Stack Overflow. *most-popsular-techologies-webframe-webframe*. URL: <https://survey.stackoverflow.co/2025/technology#most-popular-technologies-webframe-webframe> (visited on 9th Apr. 2026).
 - [15] Timescale, Inc. *TimescaleDB – Real-Time Insights, Scalable Time-Series*. URL: <https://timescaledb.org> (visited on 12th May 2026).
 - [16] owasp. *Zero Trust Architecture Cheat Sheet*. URL: https://cheatsheetseries.owasp.org/cheatsheets/Zero_Trust_Architecture_Cheat_Sheet.html (visited on 23rd Apr. 2026).
 - [17] EMQ Technologies Inc. *What Is MQTT and Why Is It the Best Protocol for IoT?* URL: <https://www.emqx.com/en/blog/what-is-the-mqtt-protocol> (visited on 30th Apr. 2026).
 - [18] Ian Craggs. *Understanding the Differences between MQTT and WebSockets for IoT*. URL: <https://www.hivemq.com/blog/understanding-the-differences-between-mqtt-and-websockets-for-iot/> (visited on 1st May 2026).
 - [19] Orne Brocaar. *ChirpStack*. URL: <https://www.chirpstack.io/> (visited on 7th Apr. 2026).
 - [20] Chris Richardson. *Microservice Architecture pattern*. URL: <https://microservices.io/patterns/microservices.html> (visited on 3rd May 2026).
 - [21] <https://microservices.io/>. *Pattern: Saga*. URL: <https://microservices.io/patterns/data/saga.html> (visited on 8th Apr. 2026).
 - [22] Feature-Sliced Design. *FSD Concepts*. URL: <https://feature-sliced.design/docs/get-started/overview#concepts> (visited on 9th Apr. 2026).
 - [23] Inc Meta Platforms. *Synchronizing with Effects – Learn React*. URL: <https://react.dev/learn/synchronizing-with-effects%5C#fetching-data> (visited on 6th May 2026).
 - [24] auth0. *JSON Web Tokens*. URL: <https://auth0.com/docs/secure/tokens/json-web-tokens> (visited on 11th May 2026).
 - [25] auth0. *Hashing in Action: Understanding bcrypt*. URL: <https://auth0.com/blog/hashing-in-action-understanding-bcrypt/> (visited on 12th May 2026).
 - [26] Microsoft. *Circuit Breaker pattern*. URL: <https://learn.microsoft.com/en-us/azure/architecture/patterns/circuit-breaker> (visited on 2nd May 2026).

-
- [27] ChirpStack. *chirpstack-device-profiles*. URL: <https://github.com/chirpstack/chirpstack-device-profiles> (visited on 2nd May 2026).
- [28] <https://microservices.io/>. *Pattern: Event-driven architecture*. URL: <https://microservices.io/patterns/data/event-driven-architecture.html> (visited on 2nd May 2026).
- [29] FP Team. *Data reconciliation: the great data jigsaw*. URL: <https://www.future-processing.com/blog/data-reconciliation-the-great-data-jigsaw/> (visited on 10th May 2026).
- [30] HSC Prep. *The Trapezoidal Rule*. 2025. URL: <https://hscprep.com.au/hsc-maths-advanced/the-trapezoidal-rule/> (visited on 15th May 2026).
- [31] Lo Gullstrand Heander, Emma Söderberg and Christofer Rydenfält. ‘Code review as decision-making - building a cognitive model from the questions asked during code review’. In: *Empirical Software Engineering* 31.54 (2026). DOI: 10.1007/s10664-025-10791-2. URL: <https://link.springer.com/article/10.1007/s10664-025-10791-2>.
- [32] Grafana Labs. *Grafana k6 – Open Source Load Testing Tool*. URL: <https://k6.io> (visited on 3rd May 2026).
- [33] Anil Gudigar. *Mastering Latency Metrics: P90, P95, P99*. URL: <https://medium.com/javarevisited/mastering-latency-metrics-p90-p95-p99-d5427faea879> (visited on 12th May 2026).
- [34] SUSO Academy. *Sustainability Awareness Framework (SUSAF)*. URL: <https://www.suso.academy/en/sustainability-awareness-framework-susaf/> (visited on 9th May 2026).
- [35] FN. *FNs Bærekraftsmål*. URL: <https://unric.org/no/baerekraftsmalene/> (visited on 9th May 2026).
- [36] Chris Richardson. *Pattern: Microservice Architecture*. 2025. URL: <https://microservices.io/patterns/microservices.html> (visited on 7th May 2026).
- [37] Anthropic. *System Card: Claude Sonnet 4.6*. Se avsnitt 1.1.1: Training data and process, s. 8. 2026. URL: <https://www-cdn.anthropic.com/bbd8ef16d70b7a1665f14f306ee88b5.pdf> (visited on 9th May 2026).
- [38] Ben Welford. *Everything you need to know about the “Right to be forgotten”*. 2018. URL: <https://gdpr.eu/right-to-be-forgotten/> (visited on 16th May 2026).

11 Vedlegg

- A Prosjektplan
- B Gruppekontrakt
- C Wireframes
- D Møtereferater
- E Timeføring
- F Use-Cases
- G Commit markdown
- H Detaljert sekvensdiagram context-service
- I API Endepunkter

A Prosjektplan



INSTITUTT FOR DATATEKNOLOGI OG INFORMATIKK

PROG2900 - BACHELOR

PROSJEKTPLAN

Fabrikkpuls

Samspill mellom sensordata og ERP-systemer for kontekstuell forståelse
av produksjonsprosesser

Forfattere:

Vinjar Berland
Magnus Dybdal
Jonas Nygaard
Marius Eilertsen

1st April 2026

Table of Contents

1	Mål	1
1.1	Bakgrunn	1
1.2	Prosjektmål	1
2	Omfang	3
2.1	Fagområde	3
2.2	Avgrensning	3
2.3	Oppgavebeskrivelse	3
2.4	Videre utvikling av funksjonalitet	4
3	Prosjektorganisering	5
3.1	Ansvarsforhold og roller	5
3.2	Gruppereregler	6
4	Planlegging	7
4.1	Arbeidsmetodikk	7
4.2	Versjonkontroll og GitLab bruk	7
4.3	Milpæler	8
4.4	Plan for statusmøter	8
5	Organisering av kvalitetssikring	9
5.1	Dokumentasjon, verktøybruk og kildekode	9
5.2	Møte-rutiner	9
5.3	Test og kvalitetssikring	10
5.4	Risikoanalyse	10
6	Plan for gjennomføring	12
6.1	Gantt-skjema	12
6.2	Product backlog	14
7	Utviklingsmiljø og teknologi	15
	Kilder	16

Ordliste

gateway En [LoRaWAN](#)-gateway er en enhet som mottar trådløse [LoRa](#)-signaler fra sensorer og videresender dem til en sentral server over IP-basert nettverk.. [ii](#), [3](#), [4](#), [10](#), [11](#), [15](#)

LoRaWAN Nettverks Server Programvare som håndterer tilkoblinger, behandling og oversikt over [gateway](#) og sensorer.. [3](#), [15](#)

SCRUM Et smidig rammeverk for utviklingsmetodikk. [5](#), [7](#)

sprint En kort, tidsbestemt periode hvor et scrum team jobber med å fullføre en bestemt mengde jobb Atlassian 2026.. [7](#)

Forkortelser

API Application Programmable Interface. [1](#), [3](#)

ERP Enterprice Resource Planning. [1](#), [3](#)

LoRa Long Range. [ii](#)

LoRaWAN Long Range Wide Area Network. [ii](#), [3](#)

MVP Minimum Viable Product. [4](#)

PBL Product Backlog. [5](#), [7](#)

PO Product Owner. [5](#)

SBL Sprint Backlog. [7](#)

SM Scrum Master. [5](#)

1 Mål

1.1 Bakgrunn

Innoveria AS¹ er et teknologiselskap som arbeider med å forenkle innsamling, bruk og visualisering av data i industrien. Selskapet har som mål å gjøre produksjon mer effektiv ved å motivere fabrikker til å ta i bruk teknologiske løsninger for overvåking av produksjonsprosesser. Innoveria ønsker å redusere kompleksiteten og tidsbruken knyttet til datainnsamling, slik at produksjonsbedrifter kan ta bedre beslutninger basert på både sanntids- og historiske data.

Produksjonsdata i dagens industrimiljøer er ofte fragmentert og krevende å benytte i operasjonelle sammenhenger. Det finnes i begrenset grad felles standarder for hvordan slike data samles inn, struktureres og visualiseres (Prosess21 2026). Samtidig stilles det i økende grad krav til at fabrikker kan dokumentere og rapportere miljøavtrykket fra egne produksjonsprosesser. Et eksempel på dette er ISO 14067 (ISO 2018), som legger til rette for rapportering av klimaavtrykk helt ned på per-enhet-nivå. Disse utfordringene skaper et behov for en helhetlig løsning som kan samle data fra maskiner, sensorer og eksisterende systemer, som for eksempel ERP-systemer, og sette disse i kontekst av produksjonsmiljøet gjennom visualisering av sanntids- og historiske data.

Innoveria ønsker derfor å utvikle en helhetlig løsning bestående av en database for lagring av tidsseriedata med tilhørende metadata, et API-lag for datainnhenting og spørringer, samt et moderne dashboard med rollebasert tilgang for både fabrikkene og Innoveria. Løsningen skal støtte håndtering av både sanntids- og historiske data, og legge til rette for at data kan settes i kontekst av produksjonsmiljøet.

Vi valgte oppgaven fordi den gir oss mulighet til å arbeide med ny teknologi vi ikke har erfaring med fra før. Sammenhengen mellom programvareutvikling og praktisk anvendelse er tydelig, og prosjektet har potensial til å skape reell verdi i faktiske produksjonsmiljøer. Samarbeidet med Innoveria AS og deres tilknytning til lokale bedrifter og fabrikker gjør prosjektet ekstra relevant og motiverende.

1.2 Prosjektmål

Effektmål

Målet med prosjektet er å gi Innoverias kunder enklere tilgang til produksjonsdata og kunne sette denne i kontekst med produksjonen, slik at de kan overvåke produksjonsprosesser og ta bedre og mer informerte beslutninger. Kundene vil ha ulike behov basert på sine produksjonsmiljøer, og effekten av applikasjonen vil derfor variere mellom de ulike fabrikkene, blant annet avhengig av hvilke sensorer som er installert. Selv om effekten kan variere, har prosjektet definert noen overordnede effektmål.

Applikasjonen skal gjennom innhenting, analyse og visualisering av produksjonsdata bidra til å:

- Bevisstgjøre fabrikkene om eget ressursbruk ved å sette sensordata i kontekst av produksjonsdata.
- Bidra til redusert forbruk av ressurser i produksjonslinjen, som for eksempel strøm eller gass.
- Redusere behovet for manuell loggføring av produksjonsprosesser gjennom automatisk innsamling og visualisering av data.
- Redusere tiden fra installasjon av sensor og/eller gateway til bekreftet og synlig dataflyt i systemet ved å forenkle oppsett og igangsetting av sensorer og gateways.

¹Innoveria 2026

Resultatmål

Etter endt prosjektperiode skal prosjektgruppen levere en fullstack-løsning i form av en webapplikasjon som fungerer godt på både små og store skjermer. Løsningen skal inkludere komponenter for innsamling, overvåking og visualisering av sensordata, samt innhenting av produksjonsdata fra ERP-systemer. Løsningen skal sørge for at sensordataene settes i kontekst av produksjonsprosessen, noe som gir fabrikker innsikt som få andre systemer tilbyr.

Videre skal systemet ha rollebasert tilgang med autentisering, slik at kun autoriserte brukere får tilgang til riktig data. Siden konteksten kan variere mellom fabrikker, vil løsningen i første omgang fokusere på noen utvalgte use-cases, men den skal designes slik at det enkelt kan utvides til andre kontekster for fremtidige kunder. Løsningen skal kunne overtas og settes i drift av Innoveria etter endt prosjektperiode.

Læringsmål

Læringsmålene for prosjektperioden er fokusert på følgende punkter:

- Videreutvikle ferdigheter knyttet til teamarbeid og utviklingsprosess, med særlig fokus på SCRUM som arbeidsmetodikk
- Tilegne oss erfaring med utvikling av en fullstack-løsning, inkludert samspill mellom database, API og webgrensesnitt
- Utvikle kompetanse innen autentisering og rollebasert tilgangskontroll i webapplikasjoner

2 Omfang

2.1 Fagområde

Prosjektets fagområde faller inn under industriell digitalisering. Dette fagområdet omfavner mye, men vårt prosjekt fokuserer spesielt på innsamling, lagring og visualisering av produksjonsdata fra egne ERP-systemer og LoRaWAN-sensorer. Det er sensorer som benytter lavenergi og langdistansekommunikasjon for datainnsamling i industrielle miljøer. Sensorene har som rolle å fange opp måledata som temperatur, fuktighet, avstand og mer, før de sender data over LoRaWAN-nettverket til gateway, som viderefører til datainnsamling og analyse.

LoRa og LoRaWAN utgjør en sentral rolle for teknologi i dette fagområdet. I følge The-Things-Network (2026) beskrives LoRa som:

Trådløs modulasjonsteknikk som er avledet fra Chirp Spread Spectrum (CSS)-teknologi. Den koder informasjon på radiobølger ved hjelp av chirp-pulser på samme måte som delfiner og flaggermus kommuniserer. LoRa-modulert overføring er robust mot forstyrrelser og kan mottas over svært lange avstander (egen oversettelse).

Videre beskriver The-Things-Network (2026) LoRaWAN som:

Media Access Control (MAC)-lagsprotokoll bygget oppå LoRa-modulasjon. Protokollen fungerer som et programvarelag som definerer hvordan enheter benytter LoRa-maskinvare, blant annet når enheter kan sende data, samt hvordan meldinger struktureres og formateres (egen oversettelse).

LoRaWAN passer godt i produksjonsbedrifter ettersom teknologien gjør det mulig å overvåke maskiner og produksjonsmiljøer over et stort areal med minimalt energiforbruk. Det egner seg også veldig godt i miljøer med tykke vegger ettersom signalet har sterk penetreringsevne (The-Things-Network 2026).

Produksjonsbedrifter benytter ERP-systemer for å samle data om produksjonsprosesser, som produksjonsorder, maskinbruk og tidsintervaller. For at sensorerdata skal tolkes på meningsfull måte, må disse måleverdiene settes i kontekst av produksjonsdata. Det vil gi en bedre forståelse av hva en produksjonsprosess koster i utslipp og muliggjør en mer informert beslutning.

2.2 Avgrensning

Prosjektet vil ikke utvikle en egen LoRaWAN Nettverks Server, men benytte eksisterende tjenester for å forenkle oppsett av sensorer og gateways. Fokus vil ikke ligge på å lage avansert grafisk fremstilling av data, men på systemets evne til å lagre, behandle og visualisere sensordata, samt å sette denne dataen i kontekst med produksjonsdata.

Siden det finnes mange kilder til produksjonsdata, vil prosjektet fokusere på én av de større leverandørene av ERP-systemer, Monitor ERP², for innhenting av slik data. Videre begrenses prosjektet til å støtte et begrenset utvalg av sensorprodusenter og gateways, primært Milesight³ og RAK⁴, for å sikre at prosjektets omfang er håndterbart.

2.3 Oppgavebeskrivelse

Oppgaven er å utvikle en fullstack-løsning for mottak, lagring, behandling og visualisering av sensordata i kontekst av produksjonsdata fra industrielle sensorer tilknyttet et LoRaWAN-nettverk, samt produksjonsdata fra ERP-systemer. Fullstack-løsningen skal bestå av et webgrensesnitt for visualisering av data, et backend-API for behandling av data, og en database for lagring.

²Monitor 2026.

³Milesight 2026.

⁴RAK 2026.

Den ferdige løsningen skal ha følgende funksjonalitet - funksjonalitet knyttet til MVP er markert med (MVP)

- Legge til gateway og sensorer.
- Motta og dekode sensordata. (MVP)
- Lagre sensordata i databasen for videre behandling.
- Behandle data for analyse og visualisering. (MVP)
- Innhente produksjonsdata fra ERP-systemer.
- Rollebasert autorisasjon for å sikre riktig tilgang til data.
- Visualisere sensordata satt i kontekst av produksjonsdata gjennom webgrensesnittet, inkludert:
 - Fremstilling av historiske sensordata.
 - Fremstilling av sanntidsdata.
 - Fremstilling av sensordata kombinert med produksjonsdata. (MVP)

I MVP-en inngår det bruk av mockdata for blant annet produksjonsdata fra ERP-systemer. Dette skyldes at tilgang til fabrikkens ERP-system ennå ikke er etablert, og at det ikke er avklart når slik tilgang vil være tilgjengelig. På bakgrunn av dette er funksjonalitet for direkte innhenting av produksjonsdata fra ERP-systemer ikke inkludert i MVP-en.

Likevel er funksjonalitet for fremstilling av sensordata kombinert med produksjonsdata inkludert i MVP-en, ettersom bruk av mockdata gjør det mulig å utvikle, teste og demonstrere visualiserings- og analysefunksjonalitet uten avhengighet til eksterne systemer.

2.4 Videre utvikling av funksjonalitet

Utover funksjonaliteten som er beskrevet i dette kapittelet, finnes det flere utvidelser som vurderes som relevante, gitt tilstrekkelig tid og ressurser i prosjektperioden. Avgrensningen og oppgavebeskrivelsen er bevisst utformet for å prioritere kjernefunksjonalitet som gir størst mulig verdi for både fabrikkene og Innoveria, samtidig som det sikrer gjennomførbarhet innenfor prosjektets rammer.

Dersom tidsplanen tillater det, vil prosjektet utvides med ytterligere funksjonalitet for å gjøre løsningen mer robust og fleksibel. Disse utvidelsene er tenkt implementert på en modulær måte, slik at de kan fungere som selvstendige byggeklosser i systemarkitekturen. Aktuelle utvidelser inkluderer blant annet:

- Et modulært dashboard som gir brukere mulighet til å tilpasse visning og presentasjon av data etter egne behov.
- Eksponering av et API for deling av data med tredjepartssystemer.

3 Prosjektorganisering

3.1 Ansvarsforhold og roller

Scrum-teamet er organisert og inndelt etter tradisjonelle scrum roller:

- **Scrum Master** - Marius Eilertsen
Scrum Master (**SM**) er overordnet ansvarlig for at utviklingsteamet skjønner verdien av **SCRUM**, og at anvendelsen av rammeverket blir benyttet på en effektiv måte. SM vil også avholde og sørge for at møter relatert til SCRUM-metodikken blir gjennomført på en hensiktsmessig måte.
- **Product Owner** - Vinjar Berland
Bindeleddet mellom oppdragsgiver (Innoveria) og utviklingsteamet. **PO** har ansvar for å forstå Innoveria sine ønsker og visjon slik at det kan videreføres til utviklings-teamet gjennom blant annet **PBL** slik at denne blir riktig prioritert. Vi valgte en Product Owner fra utviklingsteamet for å sikre rollen hos en person med god teknisk forståelse av produktet. Det er for å ikke pålegge Innoveria ansvaret som påfaller PO, som f.eks. kontinuerlig endringer av PBL.
- **Utviklere** - Magnus Dybdal og Jonas Nygaard
Hovedfokus for utviklere ligger på utvikling av programvare. I og med at scrum-teamet består av kun fire medlemmer, vil det si at alle fire medlemmer i praksis anses som utviklere.

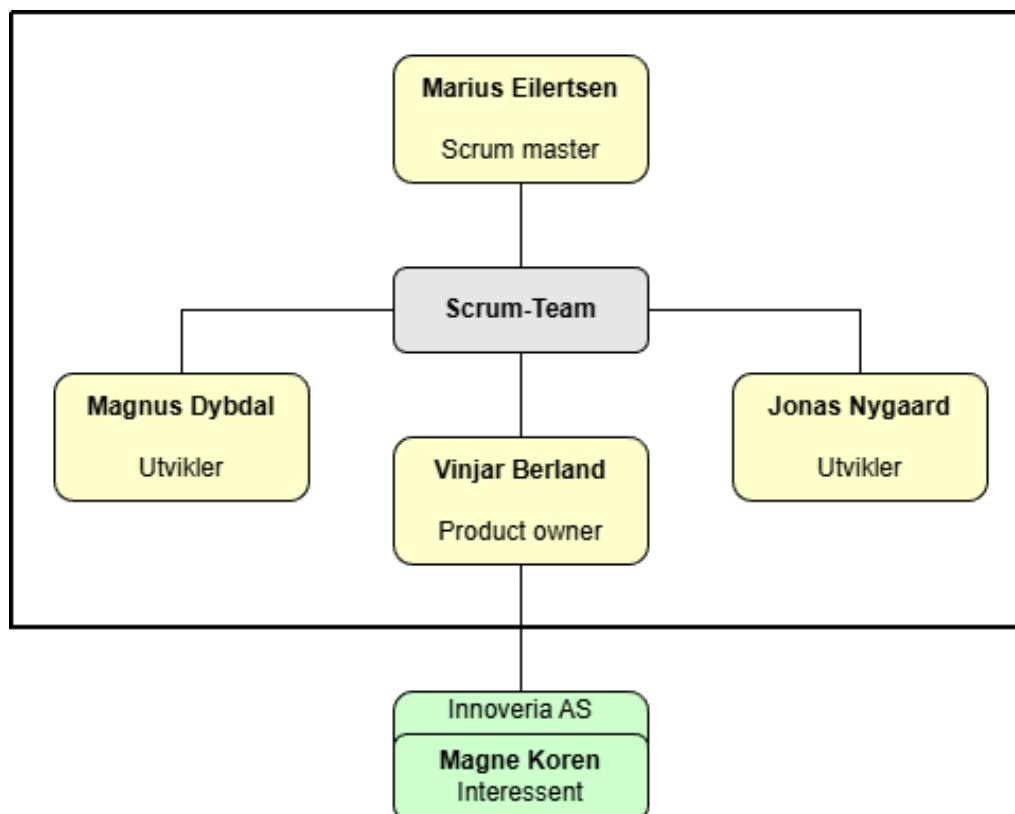


Figure 1: Organisasjonskart

3.2 Grupperegler

Grupperegler i sin helhet er blitt lagt ved som vedlegg, men en oppsummering av gruppereglene ser slik ut:

- Hovedkommunikasjon utenom fysiske møter skjer på gruppens Discord-server.
- Arbeidstimer skal loggføres og hvert gruppemedlem skal jobbe minst 30 timer pr. uke med oppgaver knyttet til prosjektet.
- Det er innført fast kjernetid⁵ mellom kl. 10:00 til 14:00 hverdager.
- Det er lagt opp til to faste arbeidsdager med fysisk oppmøte: mandager og torsdager. Det forventes at alle gruppemedlemmer møter på disse ved mindre annet er kommunisert.
- Gruppemedlemmer kan som siste utvei bli kastet ut av gruppen, ved grove og gjentakende brudd på kontrakt.

⁵Tid der medlemmer av teamet er tilgjengelige og jobber med oppgaven

4 Planlegging

4.1 Arbeidsmetodikk

I diskusjoner rundt arbeidsmetodikk ble det tidlig fremmet et ønske om [SCRUM](#). Dette er fordi alle gruppemedlemmene har gode erfaringer med scrum fra tidligere prosjekter, og vi er godt vant og liker med å jobbe i et scrum-team. Det er dermed ikke selvsagt at dette er den beste metodikken for dette prosjektet. Før vi bestemte oss for metodikk diskuterte vi alternative modeller som fossefall- og andre iterativ-modeller.

En modell som er blitt diskutert i sammenheng med SCRUM, er Kanban-modellen. Denne ville stilt prosjektet noe friere med tanke på struktur, men det er nettopp på grunnlag av dette at valget vårt falt på SCRUM. Vi mener at strukturen som kan anvendes med SCRUM er en styrke av rammeverket, som bidrar til en smidig og tilpasningsdyktig utvikling.

Under planleggingsfasen i prosjektet ble det lagt fram flere inkremitter som kan bygges rundt selve hovedoppgaven om vi skulle få tid og muligheten. Dette var blant annet et av de gode argumentene som ga fordel til å anvende scrum i dette prosjektet. Om vi skulle anvendt en fossefallsmodell gir dette liten fleksibilitet til å legge til nye inkremitter, og forutser at vi blir helt ferdig med hovedoppgaven før vi kan starte på en ny inkrement.

I diskusjonen vår rundt å anvende en iterativ modell, ble gruppen fort gjort oppmerksom på at den iterative modellen er et mer generelt utviklingsprinsipp, og mangler mye av rammeverket rundt selve teamet som skal utvikle.

Sprinter

I planleggingsperioden i januar vil det ikke følges faste sprinter. Denne fasen kan likevel anses som en *sprint 0*, ettersom planlegging og avklaring av prosjektets rammer utgjør en essensiell del av utviklingsarbeidet. I denne perioden vil det være mindre struktur og oversikt over konkrete utviklingsoppgaver, og arbeidsoppgaver vil derfor fordeles fortløpende. Dette gir en mer dynamisk arbeidsform enn det en ordinær sprintstruktur normalt legger opp til. Det vil likevel gjennomføres jevnlige møter internt i gruppen, samt med oppdragsgiver, veiledere og andre relevante interessenter.

Etter avsluttet planleggingsfase (sprint 0) legges det opp til to-ukers sprinter for selve utviklingsarbeidet i prosjektet. Totalt er det planlagt ni sprinter à to uker. Sprintlengden på to uker anses som hensiktsmessig, da prosjektet strekker seg over flere måneder og gir rom for jevnlig evaluering, tilpasning og tett samarbeid i teamet gjennom regelmessige møter, arbeidsøkter og digital kommunikasjon.

4.2 Versjonkontroll og GitLab bruk

Branch-logikk

Repositoriet er delt opp i to hovedbrancher: *main* og *dev*, der *main* fungerer som en stabil release-branch, og *dev* benyttes til utvikling av ny funksjonalitet.

Videre benytter vi oss av en *feature-branch*-basert arbeidsflyt, der ny funksjonalitet og feilrettinger utvikles i egne brancher før de merges inn i *dev*-branchen. Arbeid knyttet til en issue skal leveres som en merge request, som må godkjennes av minst ett annet gruppemedlem før den merges inn i *dev*-branchen. Dette gjør det enklere å isolere endringer og opprettholde god versjonskontroll. *dev*-branchen merges inn i *main*-branchen ved slutten av en sprint.

Scrum board

Vi bruker gitlab sitt issue board som vårt scrum-board. Issueene er fordelt i 6 kolonner:

- [Product Backlog \(PBL\)](#)
- [Sprint Backlog \(SBL\)](#) (issue i nåværende [sprint](#), men ikke påbegynt)

-
- In progress (i pågående arbeid)
 - Blocked (vidre utvikling er avhengig av funksjon som ikke enda er utviklet)
 - In review (ferdig issue som skal sees gjennom av et annet medlem)
 - Closed (fullført og godkjent)

Hver issue er markert med grad av prioritet/arbeidsmengde og type (funksjon/administrativ).

4.3 Milpæler

- 29. Januar: Prototype av datapassasje fra sensor til backend.
- 2. Februar: Innlevering av prosjektplan.
- 2. Mars: Ferdigstille MVP.
- 13. April: Gjennomført brukertester.
- 27. April: Utgivelse av programvare - fokus på rapport
- 20. Mai: Innlevering rapport.
- 02. Juni: Presentasjon av prosjekt

4.4 Plan for statusmøter

Oppdragsgiver

Det er satt opp et fast møte med arbeidsgiver annenhver mandag, på slutten av en sprint. Formålet med dette møtet er å vise arbeid som er gjort og planlegge videre arbeid som er i tråd med oppdragsgiver interesse. I scrum-teori vil dette møtet består av en *sprint review* og *sprint planning*.

Veiledning

Det er planlagt veiledningsmøter med vår veileder fra NTNU, Tom Røise. Veiledningsmøter skal kalles inn av gruppen med en klar agenda. Hyppigheten av møtene bestemmes kontinuerlig i samarbeid med veileder, og holdes kun når gruppen ser et behov for veiledning. Det satt av tid hver torsdag kl 14:15-14:45 til slike møter.

5 Organisering av kvalitetssikring

5.1 Dokumentasjon, verktøybruk og kildekode

Utifra det vi har lært og erfart fra tidligere prosjekter må vi ha god tilgjengelighet på all dokumentasjon og kildekode til når enn vi trenger det. Dette krever god struktur på filer og dokumenter.

Dokumentasjon og lagring

- Alle notater, møte-referater og diverse filer som ikke kan kategoriseres som verken kildekode eller rapport skal lagres på en felles Google disk.
- Kildekode relatert til prosjektet skal ligge i sin respektive mappe på Gitlab basert på monorepo mappe struktur. Det vil si at all kildekode ligger i samme kodebase.
- Rapporter skrives og lagres på overleaf.

Verktøy

Program	Beskrivelse
Google Drive	Dokumentlagring
Google Sheets	Timeføring
Google Docs	Dokumentasjon og notater
Google Slides	Presentasjon
Overleaf Latex	Prosjektbeskrivelse og rapporter
Miro	Idémyldring
Discord	Intern og ekstern kommunikasjon
Slack	Kommunikasjon med øvrige interessenter
GitLab	VCS og CI/CD
TeamUp	Felles kalender for utviklingsteamet

Table 1: Verktøy

5.2 Møte-rutiner

- I løpet av hvert møte, både innenfor gruppen og sammen med veileder eller ekstern virksomhet, skal det skrives et referat. Det skal inneholde alt vi snakker om og oppgaver vi løser.
- Notater direkte relatert til sprinter skal følge en bestemt mal.
 - Alle maler inneholder tilstedeværende medlemmer.
 - Daily scrum skal for hver person si hva som ble utført, videre arbeid og blokkerte oppgaver.
 - Sprint planning inneholder sprintmål, prioriterte oppgaver, estimer, avhengigheter og risiko, kapasitet og definisjon av ferdig.
 - Sprint retrospective inneholder hva fungerte bra, hva fungerte ikke, hva kan forbedres, tiltak til neste sprint og eventuelle åpne tema.
 - Sprint review inneholder mål for sprinten, hva ble levert, demo / gjennomgang, hva fungerte bra, hva fungerte ikke, tilbakemeldinger og neste steg.

5.3 Test og kvalitetssikring

- **Enhetstesting:** Det skal implementeres enhetstester for å sikre sentrale funksjoner og komponenter har korrekt oppførsel.
- **Systemtesting:** Det skal implementeres systemtesting for å verifisere at samspillet mellom komponenter fungerer som forventet. Det blir spesielt lagt fokus på å teste systemet som kalkulerer sammenhengen mellom data fra ERP systemer og sensordata.
- **Kodekvalitet:** Det skal brukes linting for å sikre lesbar og vedlikeholdbar kode samtidig som all kode skal ha meningsfulle kommentarer før commit. Dette er for å sikre en smidigere overføring til Innoveria for videreutvikling.
- **AI-støtte:** AI-baserte verktøy som OpenCode og Claude Code benyttes som støtte men ikke direkte utvikling i produktet. Det brukes til blant annet kodegjennomgang, forslag til forbedringer og gir en økt oversikt av sammenhenger i systemet.

5.4 Risikoanalyse

Den følgende tabellen analyserer de ulike scenarione som kan hende under prosjektet. Hver risiko er analysert basert på sannsynlighet og konsekvens. Sjanse er vurdert som usannsynlig, sannsynlig eller veldig sannsynlig. Konsekvens er vurdert som uproblematisk, problematisk eller kritisk.

Risikotabell

Konsekvens		Uproblematisk	Problematisk	Kritisk
Sjanse	Veldig sannsynlig			
	Sannsynlig			
	Usannsynlig			

Table 2: Risikomatrise

Nr	Risiko	Sjanse	Konsekvens	Tiltak
1	En av medlemmene forlater gruppen i en lengre periode eller permanent.	Usannsynlig	Kritisk	Ja
2	Kunden gjør store kravspesifisering i løpet av prosjektet	Sannsynlig	Uproblematisk	Nei
3	Prosjektet blir ikke fullført før deadline, grunnet undervurdering av tid/kompleksitet, tap av data/filer, teknologiske utfordringer .	Usannsynlig	Kritisk	Ja
4	Tap av kildekode/rapport	Sannsynlig	Problematisk	Ja
5	Sluttproduktet er for komplekst for fabrikkarbeiderne	Sannsynlig	Problematisk	Ja
6	Fabrikk(e) er ikke lenger interessert i utviklingen av produktet	Usannsynlig	Problematisk	Nei
7	Produksjonen overgår Innoverias budsjett	Usannsynlig	Problematisk	Nei
8	Feil med hardware (sensor og gateway) så programvaren ikke får reell data fra fabrikk	Sannsynlig	Kritisk	Ja

Table 3: Risikoanalyse

Tiltak

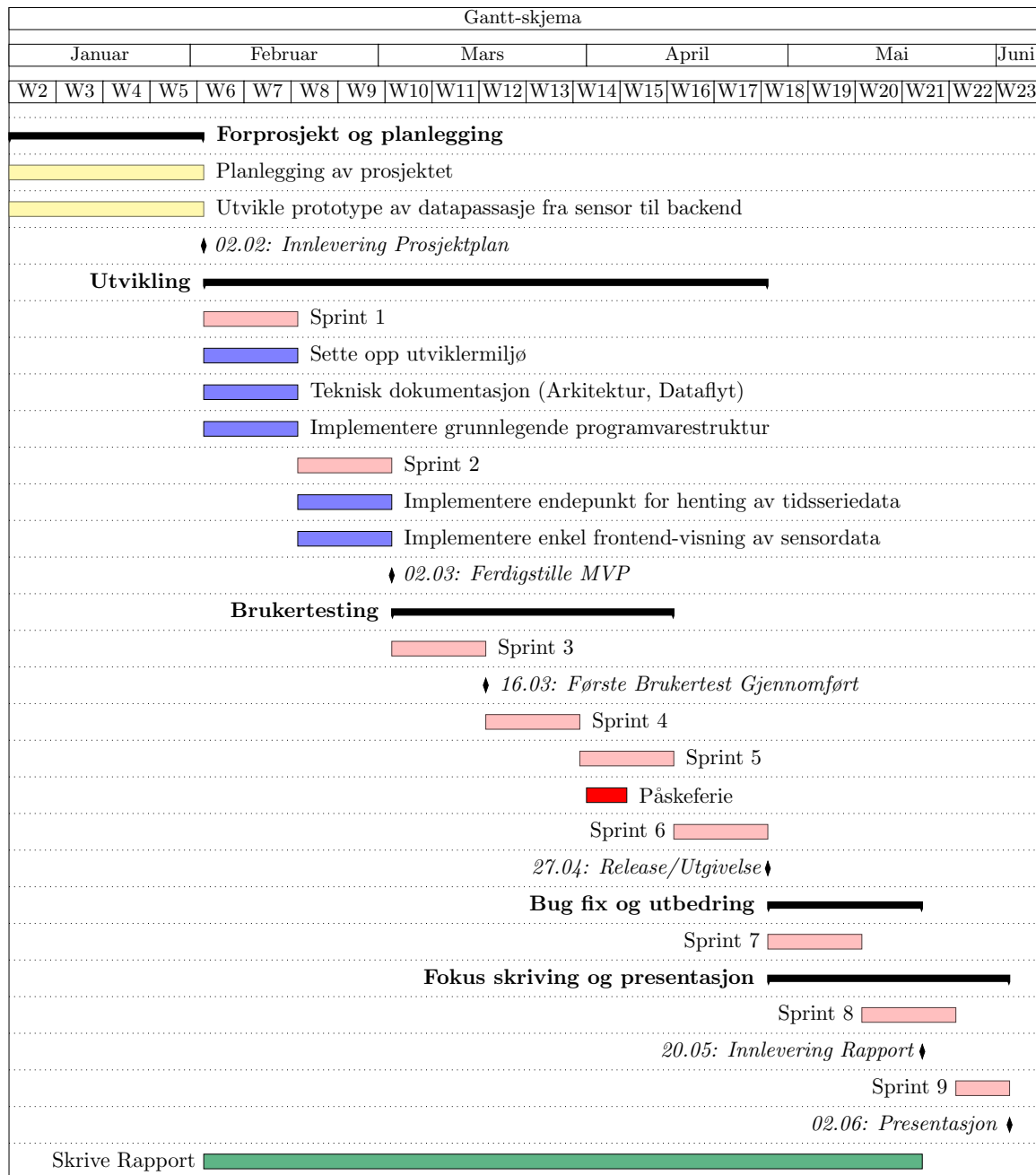
5 av de mulige risikoene ble valgt ut til å kreve en diskusjon om hvilke tiltak som kan gjøres. Disse tiltakene er ment for å redusere sannsynligheten og skaden som risikoen bringer.

Prioritet	Nr	Tiltak
Medium	1	Resten av gruppen må fortsette videre og fordele arbeidet som det manglede medlemmet hadde ansvar for. Alt arbeide skal dokumenteres, kildekode skal kommenteres og utført arbeid gjennomgås hver sprint, slik at vi vet hvilke oppgaver som skal fordeles.
Medium	3	Ved eventuelle forsinkelser av produktet må det kommuniseres med produkteieren. Vi må deretter vurdere om forsinkelsen er grunnet planleggingen av utviklingen (om så må planen framover revideres) eller kompleksiteten/størrelsen på produktet. Om det er det sistnevnte, må det diskuteres hvilke kravspesifikasjoner som må fjernes for at vi kan utvikle et fullført produkt.
Medium	4	All skriving av rapport blir gjort via overleaf, hvor teksten og filene automatisk lagres. Rapportene vil alltid være tilgjengelig så lenge vi kan logge oss inn. All kildekode lagres på en felles gitlab med regelmessige commits. Hver medlem har en lokal versjon på egen maskin.
Medium	5	De periodiske møtene med Innoveria og deres kunder skal forsikre at produktet utvikles slik de ønsker, med antagelsen om at kundene til Innoveria forstår hva deres arbeidere forstår/trenger. Brukertestene skal også sørge for at brukergrensesnittet er forståelig for Ola Nordmann.
Høy	8	Ved eventuelle problemer rundt data innhenting fra sensorer og gateway tilhørende fabrikker, så går vi over til mer manuell oppsett av gateway med produktene tilsendt av oppdragsgiver.

Table 4: Tiltak

6 Plan for gjennomføring

6.1 Gantt-skjema



i	Name/title	Start date	End date
0	Forprosjekt og planlegging	05.01.2026	02.02.2026
0.1	Planlegging av prosjektet	05.01.2026	02.02.2026
0.2	Utvikle prototype av datapassasje fra sensor til backend	05.01.2026	02.02.2026
0.3	<i>Milepæl: Innlevering Prosjektplan</i>	02.02.2026	
	Periode: Utvikling	03.02.2026	27.04.2026
1	Sprint 1	03.02.2026	16.02.2026
1.1	Sette opp utviklermiljø	03.02.2026	16.02.2026
1.2	Teknisk dokumentasjon (Arkitektur, Dataflyt)	03.02.2026	16.02.2026
1.3	Implementere grunnleggende programvarestruktur	03.02.2026	16.02.2026
2	Sprint 2	17.02.2026	02.03.2026
2.1	Implementere endepunkt for henting av tidsseriedata	17.02.2026	02.03.2026
2.2	Implementere enkel frontend-visning av sensordata	17.02.2026	02.03.2026
2.3	<i>Milepæl: Ferdigstille MVP</i>	02.03.2026	
	Periode: Brukertest	03.03.2026	13.04.2026
3	Sprint 3	03.03.2026	16.03.2026
3.1	<i>Milepæl: Første Brukertest Gjennomført</i>	16.03.2026	
4	Sprint 4	17.03.2026	30.03.2026
5	Sprint 5	31.03.2026	13.04.2026
5.1	Påskeferie	01.04.2026	06.04.2026
6	Sprint 6	14.04.2026	27.04.2026
6.1	<i>Milepæl: Release/utgivelse</i>	27.04.2026	
	Periode: Bug fix og utbedring	28.04.2026	20.05.2026
7	Sprint 7	28.04.2026	11.05.2026
	Periode: Fokus skriving og presentasjon	28.04.2026	02.06.2026
8	Sprint 8	12.05.2026	25.05.2026
8.1	<i>Milepæl: Innlevering Rapport</i>	20.05.2026	
9	Sprint 9	26.05.2026	02.06.2026
9.1	<i>Milepæl presentasjon</i>	02.06.2026	03.06.2026

Table 5: Gantt-skjema i tekst format

6.2 Product backlog

Oppgave	Prioritet	Størrelse
Skrive prosjektplan	høy	medium
Liste minimum-requirements for en fungerende app	høy	liten
Gantt skjema	høy	medium
Bestemme tech stack	høy	liten
UI Scetch / Wireframe	høy	medium
Tegne arkitekturskisse	medium	liten
Sette opp servermiljø	høy	liten
Utvikle API Gateway	høy	stor
Sette opp dokumentasjon for Swagger docs	medium	liten
Utvikle Chirpstack integrasjon tjeneste (Mikrotjeneste)	høy	stor
Utvikle Sensor innhenting tjeneste (Mikrotjeneste)	høy	stor
Utvikle Monitor ERP innhenting tjeneste (Mikrotjeneste)	høy	stor
Utvikle Kontekst tjeneste (Mikrotjeneste)	høy	stor
Utvikle rollebasert tilgang	høy	stor
Research konkurrenter	medium	liten
Utvikle dashboard for visning av enkel sensor data	høy	medium
Grensesnitt for legge til sensor og gateway	høy	stor
Sette opp databaser	høy	liten
Lage database skjema	høy	medium
Lage enhetstester	medium	stor
Utvikle mock data for erp og sensor data	høy	medium
Grensesnitt for å vise kontekst data	høy	stor
Brukertesting	medium	medium
UML Diagram	høy	medium
Presentasjon	høy	medium
Sikkerhet/GDPR	lav	medium
lage grensesnitt for visuelle grafer	lav	medium
Skrive prosjektrapport	høy	stor

Table 6: Oversikt over Product backlog

7 Utviklingsmiljø og teknologi

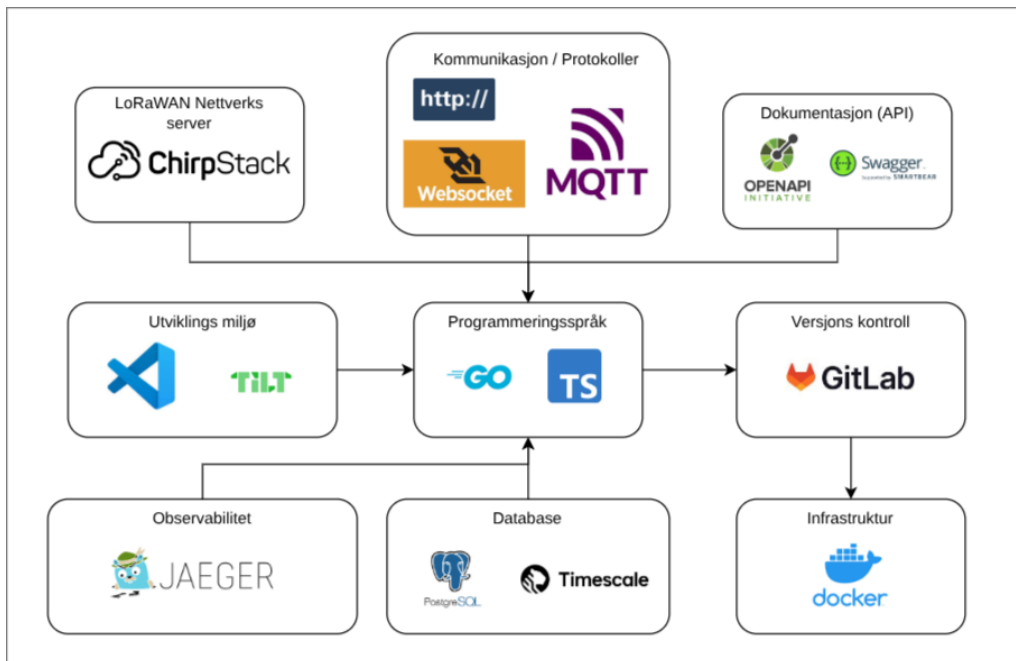


Figure 2: Utkast av teknologivalg

Figur 2 viser et foreløpig og ikke endelig utkast av teknologivalg basert på utforskning og prototype i tidelig fase av prosjektet. Enkelte deler av foreslått teknologivalg er særlig knyttet til sensor og gateway kommunikasjon som gruppen har begrenset erfaring med. Disse områdene krever mer testing og utforskning før endelig teknologi beslutning tas.

Systemet forventes å bestå av flere teknologiske lag, inkludert sensorer for datainnsamling, en mellom liggende gateway for videre formidling av data, en LoRaWAN Nettverks Server, samt en fullstack-løsning for behandling og presentasjon av data.

Kilder

- Atlassian (2026). *Scrum sprints*. URL: <https://www.atlassian.com/agile/scrum/sprints> (visited on 22nd Jan. 2026).
- Innoveria (2026). *About*. URL: <https://innoveria.no/about/> (visited on 20th Jan. 2026).
- International Organization for Standardization (2018). *ISO 14067: Carbon footprint of products*. International standard ISO 14067. International Organization for Standardization. URL: <https://www.ditan.com/static/upload/file/20240522/1716345585128585.pdf> (visited on 30th Jan. 2026).
- Milesight (2026). *Milesight*. URL: <https://www.milesight.com/> (visited on 23rd Jan. 2026).
- Monitor (2026). *Monitor-ERP*. URL: <https://www.monitorerp.com/no/> (visited on 30th Jan. 2026).
- Prosess21 (2026). *Digitalisering i prosessindustrien 2025: Hovedrapport*. Se spesielt s. 9 for omtale av data i siloer og utfordringer med integrasjon og sporbarhet. URL: <https://www.prosess21.no/siteassets/ekspertgrupper/nye-rapporter/250318-digitalisering-i-prosessindustrien-2025-hovedrapport.pdf> (visited on 20th Jan. 2026).
- RAK (2026). *RAK*. URL: <https://www.rakwireless.com/en-us> (visited on 30th Jan. 2026).
- The-Things-Network (2026). *What are LoRa and LoRaWAN?* URL: <https://www.thethingsnetwork.org/docs/lorawan/what-is-lorawan/> (visited on 23rd Jan. 2026).

B Gruppekontrakt



INSTITUTT FOR DATATEKNOLOGI OG INFORMATIKK

PROG2900 - BACHELOROPPGAVE

Gruppekontrakt

Forfattere:

Vinjar Berland
Magnus Dybdal
Jonas Nygaard
Marius Eilertsen

26th January 2026

1 Gruppekontrakt

1.1 Formål

Gruppekontrakten er en formell avtale mellom gruppemedlemmene for arbeidet med bacheloroppgaven ved NTNU Gjøvik våren 2026. Formålet er å tydeliggjøre felles forventninger, roller og arbeidsform, kommunikasjon, møtestruktur og arbeidsmengde. Kontrakten beskriver også hvordan avvik og eventuelle tvister skal håndteres, og hvilke konsekvenser som kan følge ved brudd på avtalte retningslinjer.

1.2 Organisering av gruppen og roller

- Magnus - Gruppeleder
- Vinjar - Nestleder (Sekretær)
- Marius - Kommunikasjon
- Jonas - Rom-ansvarlig

1.3 Bestemmelse av Kommunikasjons- og arbeidsplanform(er), og dokumentasjon

Som hovedkommunikasjon bruker gruppen en egen discord kanal. Her vil gruppen kunne avtale å møtes utenom faste tider og melde ifra om fravær.

Alle medlemmene i gruppen skal føre timer for arbeid de gjør i eget excel skjema som ligger i discord-gruppen. Dokumentasjon av utført arbeid vil være at man fører timer og kommenterer på timelisten. I tillegg vil det gå frem av issues man arbeider med i gitlab.

1.4 Møteorganisasjon, møtetider og møtestruktur

Gruppen skal jobbe minst to dager fysisk på skolen, og to dager remote. Dette betyr derimot ikke at man ikke kan jobbe fysisk på skolen hele uken. Alle skal være tilgjengelige i kjernetiden mellom kl 10:00-14:00, men forventes også å jobbe mer utover dette. Gruppen har valgt denne modellen for å ha forutsigbarhet for å møtes og kommunisere om oppgaven, samtidig som det gir fleksibilitet for medlemmene i gruppen.

Gruppen har avtalt å ha fysisk veilednings-møte på skolen hver torsdag kl. 14:30. Møtet er i utgangspunktet på 30 minutter, og skal forberedes med agenda og møteinnkalling som blir sendt til veileder minimum 48 timer før møtet. Daily standup avholdes mandager og torsdager kl 10:00 og avgrenses til 15 minutter.

1.5 Gruppens forventninger

Gruppen forventer at alle gruppemedlemmer deltar aktivt i utviklingsprosessen av prosjektet. Det forventes at gruppens medlemmer har god og hyppig kommunikasjon med de andre gruppemedlemmene. Om det skulle oppstå uforutsette utfordringer eller lignende som tilsier at man ikke får jobbet eller fullført tiltenkt arbeid (for eksempel i en scrum-periode), forventes det at medlemmet det gjelder sier fra i god tid til de andre medlemmene slik at det arbeidskraft kan fordeles deretter.

1.6 Arbeidsmengde

Tirsdag og Torsdag brukes til fysiske oppmøte og arbeid.

Det forventes 30 timer arbeid i uken og timer skal føres i excel ark for timeføring.

1.7 Konsekvenser/sanksjoner for brudd på gruppereglement

Hvis et av medlemmene bryter reglene i kontrakten vil de først få en advarsel fra resten av gruppen. Ved gjentatte brudd uten tydelig tegn på forbedring vil resten av gruppen ta kontakt med fagansvarlig for å informere om problemet. Gruppemedlemmet kan da som siste utvei bli fjernet fra gruppen.

1.8 Tvister/andre utfordringer

Ved tvister skal det kalles inn til et møte der tvisten skal tas opp i gruppen. Alle i gruppen må være til stede ved et slikt tvist-møte. Om det skulle låse seg helt, det vil si at det er likt antall for og mot, bruker vi en mynt for å avgjøre utfallet av tvisten om det ikke skulle være mulig å finne en fornuftig løsning for ”*begge parter*”.

C Wireframes

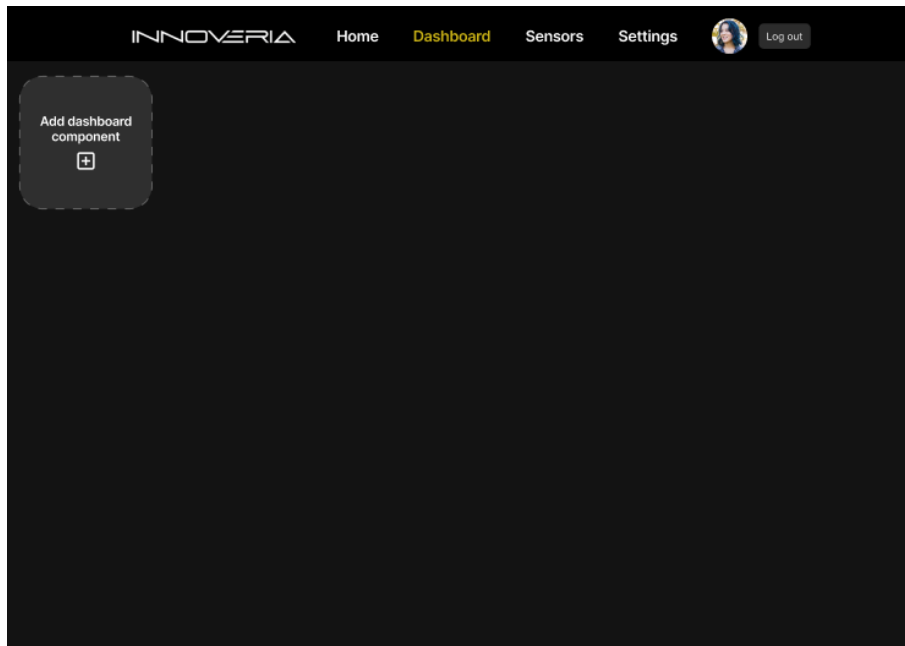


Figure 30: Tomt dashboard

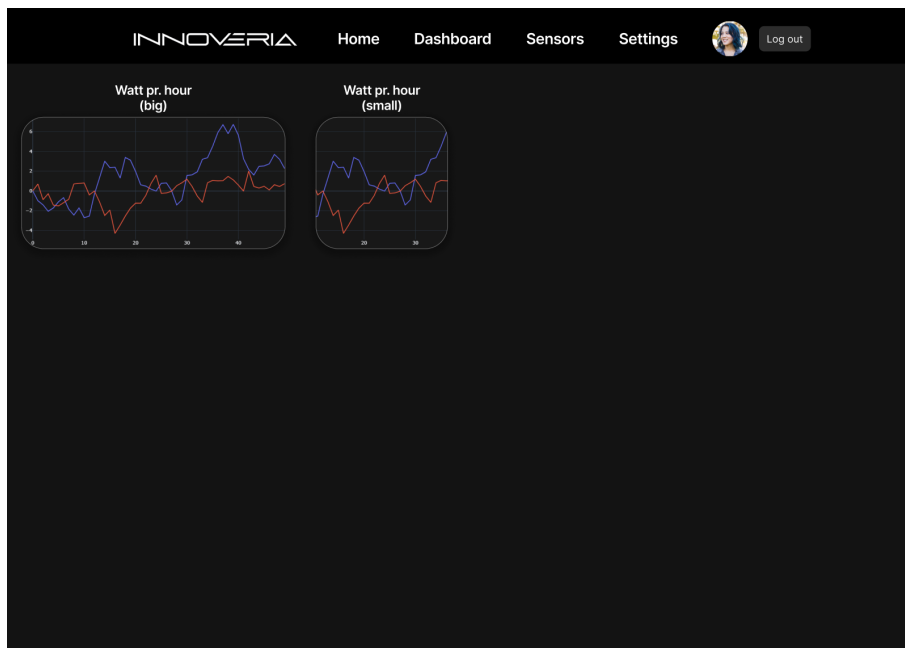


Figure 31: Med komponenter

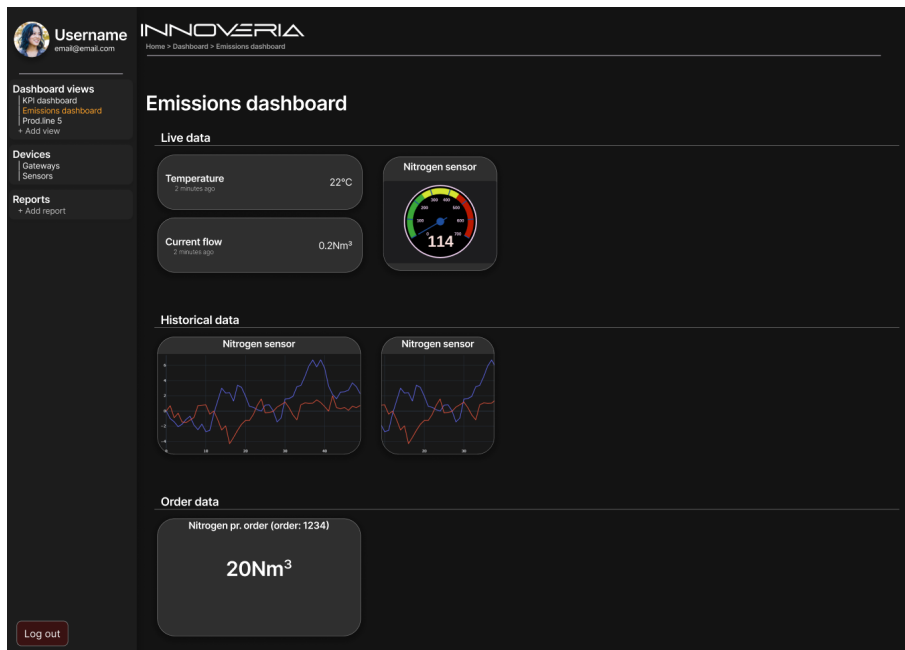


Figure 32: CO2 dashboard v2

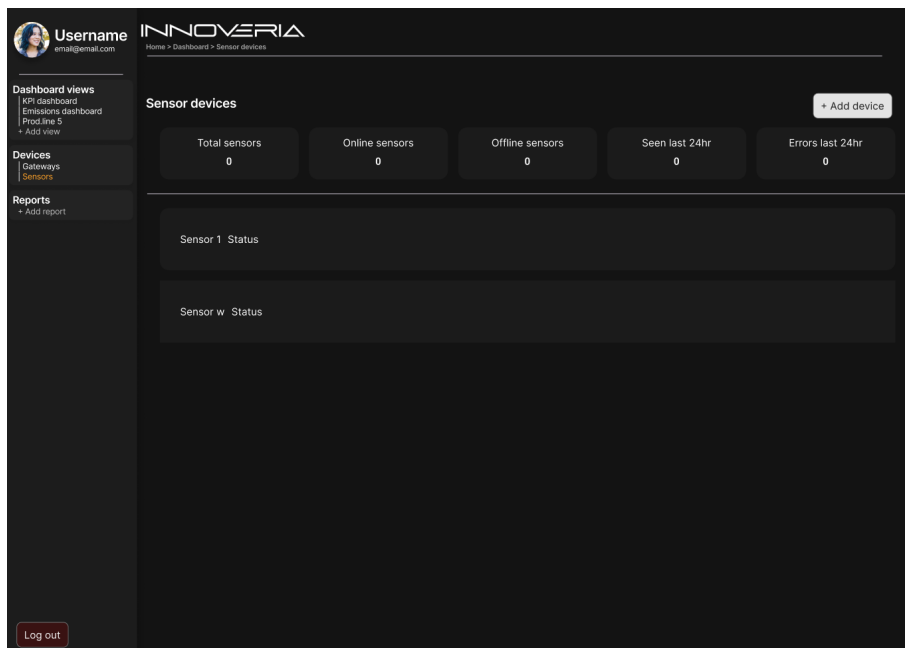


Figure 33: Sensors v2

D Møtereferater

Sprint Planning – Sprint 1 (2026-02-02)

Deltakere

Navn	Tilstede
Marius	Ja
Magnus	Ja
Jonas	Ja
Vinjar	Ja

Sprintmål

- Sette opp utviklingsmiljø
- Sette oss inn i teknologi
- Utvikle innhenting tjeneste og sette opp data passasje fra sensor til frontend.

Prioriterte oppgaver

- Velge hvilke mikro tjenester vi skal lage
- Må velge teknologi stach, har språk. Kan være vi må

Definisjon av ferdig

- Vi har satt opp en enkel http server og data passasje fra innhentings tjenste til api gateway.

Figure 34: Sprint 1 planning

Daily Scrum

Deltakere

Navn	Tilstede
Marius	Ja
Magnus	Ja
Jonas	nei
Vinjar	Ja

Hva jobber jeg med?

Marius

Har kodet list på go og react javascript

- sett på tailwind
- lagd et lite prosjekt fullstack
- og lest på monitor erp api dokumentasjon
- lest på isp14067
- lest på git message standard og bestemt git commit format

Magnus

Jobbet med IØ lagd use case diagram lagd discord gitlab bot

Vinjar

Vinjar har lagd api gateway tjeneste (boilerplate ish)

- lagd sekvens diagram for dataflyt
- lagd overordnet system arkitektur

Figure 35: Daily 5/2

Daily Scrum

Deltakere

Navn	Tilstede
Marius	Ja
Magnus	Ja
Jonas	Ja
Vinjar	Ja

Hva jobber jeg med?

Marius

frontend og material ui, og lagd en fint dashboard.

Jonas

Har jobbet med frontend og gjort seg kjent med tailwind og materual ui. Sittet og prøvd seg frem med

Magnus

fiksa domene usecase modell og lest database og microservice arkitetur. gjort mye research på kubernetes og databaser.

Vinjar

har lagd api gateway og en collection tjeneste som lytter på alle topics fra mqtt brokerern.

Figure 36: Daily 12/2

Sprint Retrospective – Sprint 1 (2026-02-16)

Deltakere

Navn	Tilstede
Marius	Ja
Magnus	Ja
Jonas	Ja
Vinjar	Ja

Hva fungerte bra

- møtene har vært bra kommer frem til mye bra
- godt oppmøte og god struktur
- generelt er kommunikasjon bra

Hva fungerte ikke

- blir for store issues, må være mer spesifikke

Hva kan forbedres

- Passe på at vi ikke gjør for mye i mvp tanken.
- fokusere på de enkle modulene så vi har noe som faktisk funker
- vi må samkjøre utviklingsspråk, vi tar alt på englesk
- til neste gang få struktur på retrospective

Tiltak til neste sprint

- bli flinkere til å spesifisere issuer

Figure 37: Sprint 1 retrospective

Sprint Review – Sprint 2 (2026-03-02)

Deltakere

Navn	Tilstede
Marius	Ja
Magnus	Ja
Jonas	Ja
Vinjar	Ja

Mål for sprinten

- live sensor i dashbordet
- og gateway og sensor status

Hva ble levert

- 70% mangler sensor status per nå i stedet for sensor så får vi det riktig

Demo / gjennomgang

- innoveria har kun bruk avg amp

Neste steg

Må høre med Carla fra vyrc om tilgang til erp tjenester

Vi ønsker å gjøre en intern onboarding, så vi kan lage til selskap, sensor og gateway.

Må se på autensitering så ikke vyrc kan se på hapro sin data.

Snakke med folk for å forstå hva de ønsker med context.

operatør stempler utifra status, riktig, indirekte tid. De har ulike status som blir stempla

support@getonnet.agency \$admin123

Figure 38: Sprint 2 review

Daily Scrum

Deltakere

Navn	Tilstede
Marius	Ja
Magnus	Ja
Jonas	Ja
Vinjar	Ja

Hva jobber jeg med?

Marius

Kan velge maskin in context og sensor. Har integret med erp sin pkg dto

Jonas

har gjort ferdig payload schema i frontend

Magnus

har fiksa login og logout, og fiksa så api gateway viser korrekt endepunkter som finnes.

Vinjar

Har lagde erp agent service registrasjon, så en bruker

Hva skal jeg gjøre i dag?

Jobbe med MR conflicts

Figure 39: Daily 27/4

Sprint Retrospective – Sprint 4 (2026-03-30)

Deltakere

Navn	Tilstede
Marius	Ja
Magnus	Ja
Jonas	nei
Vinjar	Ja

Hva fungerte bra

- ting flyter bedre
- bedre arbeidsfordeling
- folk forstår domene mye bedre

Hva fungerte ikke

- for dårlig research på sprint mål
- så undervurderte erp og hvordan det er satt opp
- at braching logikken ikke funka. Så backend implmentasjon ble puhsa før frontend var klar.

Hva kan forbedres

Tiltak til neste sprint

Eventuelle åpne tema

Figure 40: Sprint 4 retrospective

Sprint Planning – Sprint 4 (2026-03-16)

Deltakere

Navn	Tilstede
Marius	Ja
Magnus	Ja
Jonas	Ja
Vinjar	Ja

Sprintmål

- Bygge et system som kan hente collection, erp data og slå sammen til en forenkla kontekst

Prioriterte oppgaver

- starter med erp og kontekst tjenesten
- nedprioriterer RBAC (men med enkel tilkobling)

Estimerer

Avhengigheter og risiko

Kapasitet

Definisjon av ferdig

- Som bruker kan man legge til maskin
- legge til sensor til maskin

notater

fokuser på datainnsamling per maskin

Figure 41: Sprint 4 planning

Daily Scrum

Deltakere

Navn	Tilstede
Marius	Ja
Magnus	Ja
Jonas	Ja
Vinjar	Ja

Hva jobber jeg med?

Marius

har gjort en del mr og fikse noe bug fixes.

Jonas

har prøvd på edit sensor. fikk noen problemer kan nå endre gateway.

Magnus

lagd en to port splitt i device service så det finnes en del for interne kall og en til frontend og satt opp company seperation

Vinjar

- lagd en større mr sette erp token systemet.
- skrevet rapport nettverk og litt på database.

Hva skal jeg gjøre i dag?

sette opp merge request for de store branchene

- håndtering av company seperation
- håndtering av erp token generation

Figure 42: Daily 30/4

Sprint Planning – Sprint 5 (2026-03-30)

Deltakere

Navn	Tilstede
Marius	Ja
Magnus	Ja
Jonas	Nei
Vinjar	Ja

Sprintmål

- erp agent service og erp service

Prioriterte oppgaver

- backend skal hente (mock) data fra erp agent service
- sende til erp service som sender data til context.
- frontend må fikse mapping på payload schema. og jobbe med brukergrensesnittet med context.

Definisjon av ferdig

- en bruker skal kunne velge en sensor i kontekst dashbordet. Og se sensor data satt opp mot (mock) produksjons data.
 - Skal man sette opp til hvilken som helst produksjons data?

Figure 43: Sprint 5 planning

E Timeføring

DATO		Hjalmar :)					Marius					Jonas					Magnus				
		R	M	U	S	T	R	M	U	S	T	R	M	U	S	T	R	M	U	S	T
Uke 2	05.01.26																				
	06.01.26		2,5					2,5					2,5					2			
	07.01.26	2					4										4,25				
	08.01.26	3	6				5,25	1,25		1		6	1,25	4			3	4,75			
	09.01.26	5						0,5				6	0,25								
	10.01.26							0,25				4,5			0,75						
	11.01.26							0,25				4			0,75		4				
	SUM		18,5				15					30					18				
Uke 3	12.01.26	6					3,5			1,5		6		0,25			5				
	13.01.26	4										6,25					4				
	14.01.26		4				2,75	4				0,25	4	1,75	0,25			4			
	15.01.26	6	2				3,5	3,5				2,5	4,5				5,5	1,5			
	16.01.26	2				3						3,25									
	17.01.26					4	1					0,5			0,5					2,25	
	18.01.26			1		4														2	
	SUM		36				19,75					30					24,25				
Uke 4	19.01.26	1			2		2			5		5	0,5		0,5		4				
	20.01.26											3,5			2,5		2	0,5		2	
	21.01.26		6				1	5,5		0,5		0,5	6					6			
	22.01.26				1,5	3	1,5	1		3,5		1	2		3,25					4,25	
	23.01.26				4							3			2,25						
	24.01.26																				
	25.01.26						1,5													1	
	SUM		17,5				21,5					30					19,75				
Uke 5	26.01.26		2,25		2,25			2,25		2,25		2,5	1,75		1,75			2,25		2,25	
	27.01.26	4					6					1,25			3,75		4,5				
	28.01.26	2,5	2				4,5	2				1	2	2,25			3	3			
	29.01.26		7,5			1	1,5	6					7,5		0,25			7,5			
	30.01.26			3	2							4			2					4	
	31.01.26			2	1		0,5			0,25											
	DEADLINE 01.02.26				1					1,25											
	SUM		30,5				26,5					30					26,5				
Uke 6	02.02.26		5		1		1,5	1,75	2,75	2,25		2	2		2			3		1	
	03.02.26	2		2			5												1	3	
	04.02.26						3,75													2	
	05.02.26		4				4,75											4		1,5	
	06.02.26	1		3													2			2	
	07.02.26								1			0,75		0,5							
	08.02.26			4					1,25					1,5			2				
	SUM		22				24					8,75					21,5				
Uke 7	09.02.26		4				2,25	4				4	4					4			
	10.02.26			4			2,25					3,75		2,5			4			2	
	11.02.26			5			6,75					3		3			5,75				
	12.02.26		2	2			1	3,75	3,5			0,5	2	3,5			2	3			
	13.02.26			6					2					3,75			2				
	14.02.26								2,5												
	15.02.26			3					1,5	1							3				
	SUM		26				30,5					30					25,75				
Uke 8	16.02.26		5,25	2				4,75	2,75				3,5	4,5			3,5	3,75			
	17.02.26			1					8					6,5			2		3,5		
	18.02.26			4					6,5			0,75		5,5			5		1		
	19.02.26		1,5	4,5				1,5	4,5				2	4,75				1,5	3,5		
	20.02.26			5										3							
	21.02.26			5						1,5				0,25					5		
	22.02.26			2			1							0,5				1,5			

DATO		Hjalmar :)					Marius					Jonas					Magnus				
		R	M	U	S	T	R	M	U	S	T	R	M	U	S	T	R	M	U	S	T
SUM		30,25					30,5					31,25					30,25				
Uke 9	23.02.26			3				1	7,25					4			4		3		
	24.02.26			4					5,75					6					4		
	25.02.26		10,25					10,25						6					6		
	26.02.26			6															1,5		
	27.02.26		4	5				4	3,25				1,5	3,75				4	2		
	28.02.26			6			0,25		1					6					6		
	01.03.26			3					0,25			1,5		1,25					5,5		
SUM		41,25				33					30					36					
Uke 10	02.03.26		7,5					8	0,25				3,25	6,5				8			
	03.03.26			7			1		7					5					4		
	04.03.26	2		1					2,5				1	4,5				1	2		
	05.03.26	2		4				0,5	9				0,5	7,25				0,5	7,5		
	06.03.26			3					0,25		1,5			0,5					2,5		
	07.03.26			5					3,5					2,75					5		
	08.03.26			6					2					0,5					3		
SUM		37,5				34					33,25					33,5					
Uke 11	09.03.26		1	5				1	6,5					7,5				1	9		
	10.03.26			7					7					6,25					6,5		
	11.03.26		4	5				2,5	6					6,25					2		
	12.03.26			7				1,5	7,5			1,5	7,75		2,5	1,5	5,5				
	13.03.26			3		1,5			1				1,25							3	
	14.03.26			2					0,5					0,5					0,5		
	15.03.26			2															1		
SUM		36				35					31					32,5					
Uke 12	16.03.26		5	2				5	2,75				3	4,5				4	2		2
	17.03.26	2		2					8,25					6,75					4		
	18.03.26			7					7					6							
	19.03.26		1,5	6				1,5	3,5				0,75	5,75				1,5	6,25		
	20.03.26			2										3,25							
	21.03.26			2					4,5					1,75					2		
	22.03.26			3					4					7					5		
SUM		32,5				36,5					38,75					26,75					
Uke 13	23.03.26		1	6										6				1	5		
	24.03.26		1	5				1	7,5				0,5	5,5				2	6,25		
	25.03.26		5						6,5					6					6,25		
	26.03.26		1	4				1	7					9,5				1	4		
	27.03.26			2																	
	28.03.26								3					3,75							
	29.03.26								2,5					3					4		
SUM		25				28,5					34,25					29,5					
Uke 14	30.03.26		1,5	6				1,5	7					4,5	1,75			1,5	6		
	31.03.26		1	3	1			1	6,75				1	0,25	8,25			1	4		1
	01.04.26	1		3	0,5									2							
	02.04.26			2	3				1,75	3,25				0,25	2,75						
	03.04.26			2	1					5									1	2	
	04.04.26			1	2					2		2,75			0,25				1	3	
	05.04.26			2	1				1	1,75					0,5					2	
SUM		31				31					24,25					22,5					
Uke 15	06.04.26			1	4					2,5		1			2,5					3	
	07.04.26		5	0,5	3			5					3	0,25	4,25			5		2,5	
	08.04.26	0,5	0,25	3	3			0,5		3			0,25	3,25	2,5				3	4	
	09.04.26		2	4	0,5			2,5	4	1,5			1,25	7,75	0,25			2,5	4	1	
	10.04.26			0,25					8					6,5							1,5
	11.04.26								4					5,25					6		

DATO		Hjalmar :)					Marius					Jonas					Magnus				
		R	M	U	S	T	R	M	U	S	T	R	M	U	S	T	R	M	U	S	T
	12.04.26			7					3					4					3		
SUM		34					34					42					35,5				
Uke 16	13.04.26		6	2				6,25	2,25				3	3				6	2		
	14.04.26			3					4					7					4		
	15.04.26			8					8					7,5					7		
	16.04.26			5					8,5					5					3		
	17.04.26						1							4,5					3		
	18.04.26			3					3					0,5							
	19.04.26			3	1				4,5					1							
SUM		31					37,5					31,5					25				
Uke 17	20.04.26			7				0,75	8,75					8					4,5		
	21.04.26			1					3,5										4		
	22.04.26			4					8,75					2					8,25		
	23.04.26		3	3	1			3	5			0,5	3	0,5				3	5		
	24.04.26			7					2					8,5					9		
	25.04.26			5					2,5					10,5					9		
	26.04.26			8					7,5					6,75					8,25		
SUM		39					41,75					39,75					51				
Uke 18	27.04.26		3	4				3	3,25				2	5,25				3	3		
	28.04.26			4	2				4					8,75	0,25				7		
	29.04.26								1,5					4,25						4	
RELEASE	30.04.26		5,5					5,75					2,5	3,5	2,5			5,5	1		
	01.05.26	2			2				6						2				4		
	02.05.26				5				5,75					0,25	3,75				1	4	2
	03.05.26				4				1,5	8					1				1,5	2,5	
SUM		31,5					38,75					36					38,5				
Uke 19	04.05.26	1			3												3			2	1
	05.05.26		5		3			2,5	5				1	3,25	4,25			5	2,5		4
	06.05.26				7				2,25	7,75		2			5				4		1
VEIL. TOM	07.05.26		4		2			4		4			2,5		6			4	1		
	08.05.26				5					3					6						
	09.05.26				3				0,5	11,75					2,75					5	
	10.05.26				4				1,5	5,5		4			1,5					5	
SUM		37					47,75					38,25					37,5				
Uke 20	11.05.26																				
	12.05.26																				
SOFT DL	13.05.26																				
	14.05.26																				
	15.05.26																				
	16.05.26																				
	17.05.26																				
SUM		0					0					0					0				
Uke 21	18.05.26																				
	19.05.26																				
DEADLINE	20.05.26																				
SUM		0					0					0					0				
SUMSUM		557					565,5					569					534				
UKE AVG		30,9					31,4					31,6					29,7				

F Use Case Beskrivelser

High-level use case beskrivelser

Use case: Administrere fabrikker og fabrikkområder

Aktører: Bruker, Administrator

Mål: Legge til, slette eller endre fabrikker og/eller fabrikkområder

Beskrivelse: Superbrukeren legger til eller endrer fabrikker og fabrikkområder via en egen side på nettsiden ved å registrere eller oppdatere påkrevd informasjon. Videre kan superbrukeren også slette eksisterende fabrikker og/eller fabrikkområder ved å trykke på en knapp og bekrefte slettingen.

Use case: Administrer måletyper

Aktører: Administrator

Mål: Velge hvilke måletyper systemet skal håndtere

Beskrivelse: Administratoren administrerer hvilke måletyper systemet skal håndtere ved å legge til, endre eller slette navngitte måletyper, hvor administratoren selv bestemmer navnene.

Use case: Merk sensor-datafelter

Aktører: Administrator

Mål: Merke sensor-datafelter med forhåndsbestemte måletyper

Beskrivelse: Administratoren velger en sensorprofil som ikke er konfigurert. Deretter får vedkommende oversikt over hvilke datafelter sensoren sender, og kan merke disse med måletyper som er lagt til i biblioteket.

Use case: Se status på gateway/sensor

Aktører: Bruker, Administrator

Mål: Se tilkoblingsstatus på gateways og/eller sensor

Beskrivelse: Brukeren navigerer til sensor eller gateway siden. Her vil brukeren kunne se alle enheter som er koblet til systemet samt en indikator som er rød, gul eller grønn og representerer statusene offline, never seen og online.

Use case: Konfigurere dashboard

Aktører: Bruker, Administrator

Mål: Velge hvilke sensorer og hvilken data fra disse som vises i dashboardet

Beskrivelse: Brukeren velger hvilken sensor og hvilke data fra sensoren som skal vises i dashboardet, samt hvilken type graf den skal vise. Brukeren kan også legge til flere visninger på samme enhet.

Use case: Administrere brukere av plattformen

Aktører: Administrator

Mål: Administrere brukerkontoer tilknyttet et selskap

Beskrivelse: Administratoren får oversikt over alle selskapene registrert i systemet. Administratoren velger hvilket selskap sine brukere hen ønsker å administrere. Vedkommende kan legge til nye brukere, redigere eksisterende brukerinformasjon og fjerne brukere som ikke lenger skal ha tilgang til systemet.

Use case: Konfigurere aggregeringsregler

Aktører: Bruker, Administrator

Mål: Definere regler for hvordan sensordata aggregeres og presenteres i kontekstvisningen

Beskrivelse: Superbrukeren oppretter aggregeringsregler som bestemmer hvilke måletyper fra sensorene som skal beregnes og vises i produksjonskonteksten. Hver regel knyttes til en måletype og en aggregeringsmetode. Regler kan aktiveres, deaktiveres og slettes etter behov.

Use case: Legg til gateway

Aktører: Bruker, Administrator

Mål: Registrere en ny LoRaWAN-gateway i systemet

Beskrivelse: Brukeren registrerer en ny gateway ved å oppgi navn, DeviceEUI, beskrivelse samt hvilken fabrikk og fabrikkområde gatewayen er plassert i. Gatewayen provisjoneres i ChirpStack, og systemet begynner å motta sensordata via denne (så fremt en sensor er innen rekkevidde og lagt til).

Use case: Administrere selskaper

Aktører: Administrator

Mål: Opprette og administrere fabrikksselskapers tilgang til plattformen

Beskrivelse: Administratoren kan opprette nye selskaper i systemet, som videre tilrettelegger for at all infrastrukturen som trengs for å koble opp sensorer og gateways skjer under prosessen. Eksisterende selskapers informasjon kan oppdateres, og brukere kan legges til selskapet.

Use case: Se Sensordata

Aktører: Bruker, Administrator

Mål: Se data fra sensor som er lagt til i systemet

Beskrivelse: Brukeren navigerer til sensor data dashbordet. Deretter kan hen velge hvilke sensorer man vil se dataen fra ved å gi visningen et navn, tidsintervall og fra og til dato. Denne handlingen forutsetter at sensoren er konfigurert og har mottatt data allerede.

G Commit markdown

Hvorfor standardisere?

For å opprettholde oversikten over tid ved hjelp av enkel kategorisering av commit-meldinger.

Commit-typer

type	beskrivelse
feat	Ny feature du legger til i applikasjonen
fix	En bugfix
style	Feature og oppdateringer relatert til styling
refactor	Refaktoring av en spesifikk seksjon av kodebasen
test	Alt relatert til testing
docs	Alt relatert til dokumentasjon
chore	Vedlikehold av kode

Om en av disse ikke passer til din spesifikke commit, velg den som passer best.

Eksempel: `git commit -m "feat: logg inn funksjon" -m "Logger bruker inn ved hjelp av autentisering av bruker i database for så å videresende brukeren til hjemmesiden for bruker."`

eller via *nano*:

```
feat: logg inn funksjon
```

```
Logger bruker inn ved hjelp av autentisering av bruker i database for så å videresende brukeren til hjemmesiden for bruker.
```

Metoder

Nedenfor er metoder for hvordan man kan strukturere enkle commit-meldinger, men legge til en lengre beskrivelse om ønskelig.

Editor metoden

Konfigurer standard editor med kommando: `git config --global core.editor nano`

Da vil nano-editoren bli commit-melding editoren din med kommando: `git commit`

Format: (i dette tilfellet i nano):

```
<Subject>
```

```
<Description>
```

NB!! Det er veldig viktig at det er en *newline* mellom subjekt og description.

CLI-metode

```
git commit -m "<Subject>" -m "<Description>"
```

Mer detaljert info finnes [her](#).

H Sekvensdiagram context-service

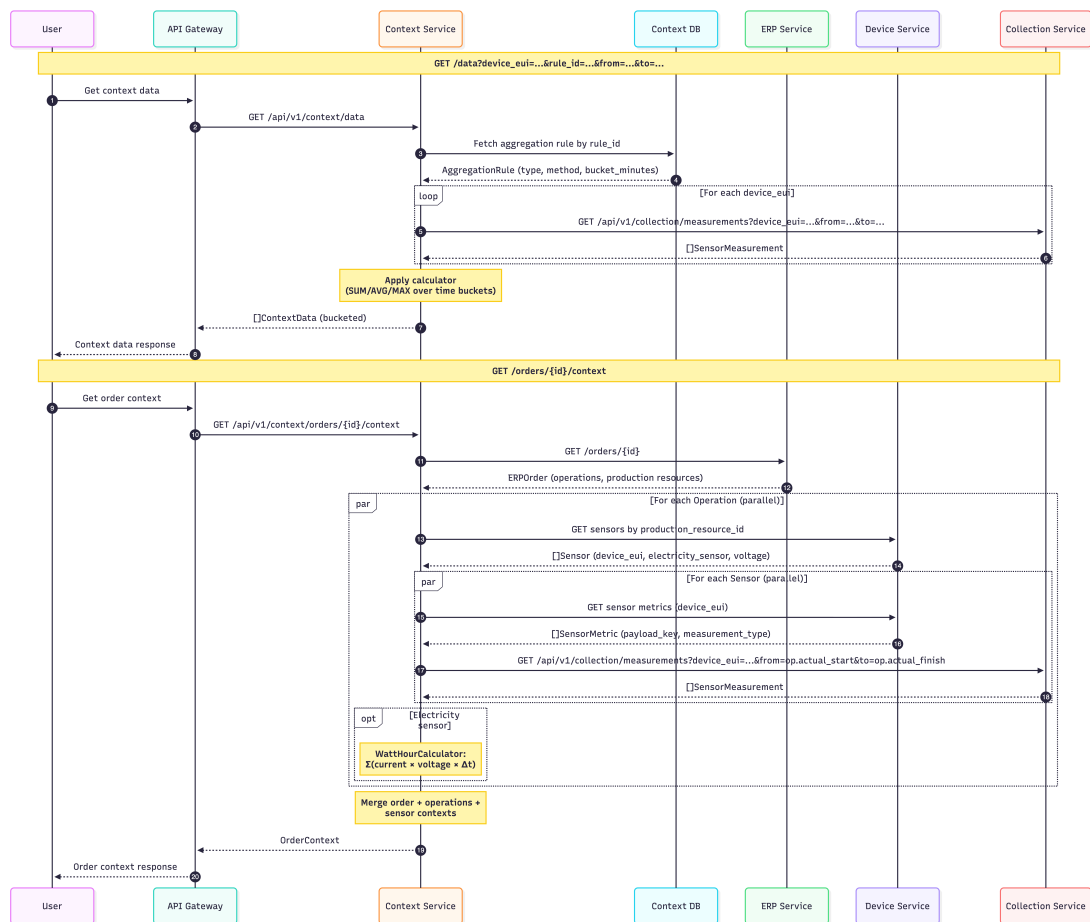


Figure 44: Detaljert sekvensdiagram

I API Endepunkter

Table 14: API-endepunkter eksponert av systemet

Tjeneste	Ressurs	Metode	Beskrivelse
auth	/auth/login	POST	Logg inn
	/auth/logout	POST	Logg ut
	/auth/refresh	POST	Roter access token
	/auth/me	GET	Innlogget bruker
	/auth/users	GET, POST	List/opprett brukere
	/auth/users/{id}	PATCH, DELETE	Oppdater/slett bruker
	/auth/companies	GET, POST	List/opprett selskaper
	/auth/companies/{id}	GET, DELETE	Hent/slett selskap
	/auth/companies/{id}/erp-agent-token	POST	Utsted ERP agent-token
	/auth/factories	GET, POST	List/opprett fabrikker
	/auth/factories/{id}	GET, DELETE	Hent/slett fabrikk
	/auth/factory-areas	GET, POST	List/opprett områder
	/auth/factory-areas/{id}	GET, DELETE	Hent/slett område
device	/device/sensors	GET, POST	List/opprett sensorer
	/device/sensors/{id}	PATCH, DELETE	Oppdater/slett sensor
	/device/sensors/sample-eui	GET	Hent eksempel-EUI
	/device/sensors/{eui}/metrics	GET, PUT	Målingstyper per sensor
	/device/gateways	GET, POST	List/opprett gateways
	/device/gateways/{id}	PATCH, DELETE	Oppdater/slett gateway
	/device/sensor-profiles	GET	List sensorprofiler

Fortsettes på neste side

Fortsettelse fra forrige side

Tjeneste	Ressurs	Metode	Beskrivelse
	/device/sensor-profile-config/{id}	GET, PUT	Hent/sett profilkonfig
	/device/payload-schema/{id}	GET, PUT	Hent/sett payload-schema
	/device/measurement-types	GET, POST	List/opprett målingstyper
	/device/measurement-types/all	GET	List alle inkl. utgåtte
	/device/measurement-types/{slug}/deprecate	PATCH	Deprecer målingstype
	/device/company-config	POST	Opprett company config
	/device/company-config/{id}	DELETE	Slett company config
collection	/collection/measurements	GET	Målinger for tidsrom
	/collection/latest	GET	Siste måling
	/collection/export	GET	Eksporterer som CSV
	/collection/payload-tags	GET	Payload-nøkler for sensor
	/collection/company-config	POST	Opprett tenant-mapping
context	/context/orders	GET	List produksjonsordrer
	/context/orders/{id}	GET	Hent ordre
	/context/orders/{id}/context	GET	Hent ordre med kontekst
	/context/rules	GET, POST, DELETE	Aggregeringsregler
	/context/data	GET	Kontekstualisert sensordata
	/context/production-resources	GET	List produksjon-sressurser

Fortsettes på neste side

Fortsettelse fra forrige side

Tjeneste	Ressurs	Metode	Beskrivelse
onboarding	/onboarding/company	POST	Onboard nytt selskap

J sekvensdiagram

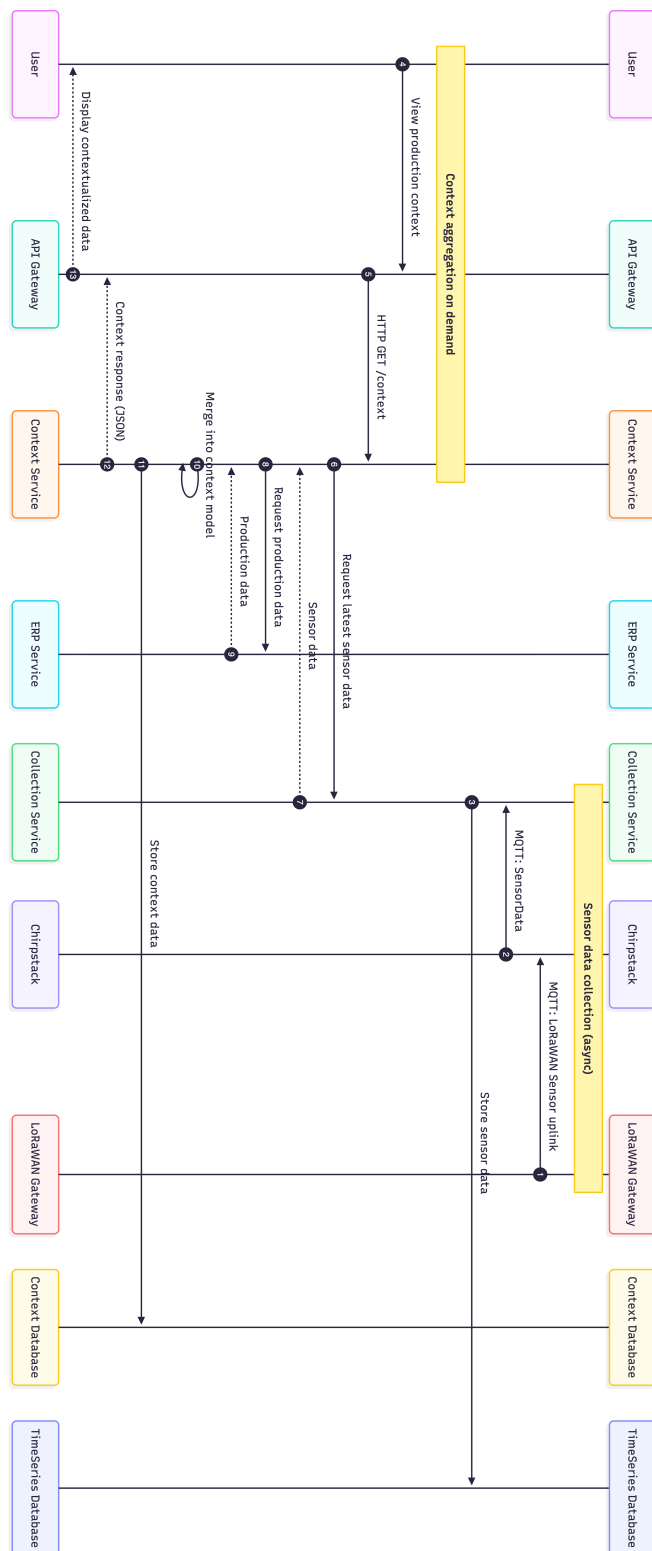


Figure 45: Context data flow